BOOMVALLEY Research Toolkit

Researcher Manual

English Edition

Final edition for Toolkit 1.0.0 (2026-10-16). Earlier: first edition 2026-10-13 (rc2), revision 2026-10-16 (rc5)

BOOMVALLEY Research Toolkit 1.0.0 (install location C:\Program Files\BOOMVALLEY Research Toolkit)

Clean Anonymous Edition

Imprint

Document	BOOMVALLEY Research Toolkit — Researcher Manual — English Edition
Copyright holder	The BOOMVALLEY Lab authors (names withheld for anonymous review)
Edition	Final edition for Toolkit 1.0.0 (2026-10-16). Earlier: first edition 2026-10-13 (rc2), revision 2026-10-16 (rc5)
Applies to	BOOMVALLEY Research Toolkit 1.0.0 (install location C:\Program Files\BOOMVALLEY Research Toolkit)
Checked on	A Toolkit 1.0.0-rc6 installation on Windows 11 Home, 2026-10-16. Every command in Part 5 and every template in Part 4 was run (section 0.2)
PDF published	2026-10-23
Software licence	Apache License 2.0 — BOOMVALLEY Lab, GNDCRW, FULLMT and the sample run code (LICENSE)
Document licence	CC BY 4.0 (Creative Commons Attribution 4.0 International) — the researcher manuals and their PDF editions, the guides, explanatory documents and diagrams (LICENSE-DOCS)

This PDF carries the content of the source Markdown unchanged, in a publication layout. Collapsible blocks (<details>) are printed open. Relative links between the source Markdown files keep their text only.

Contents

BOOMVALLEY Research Toolkit — Researcher Manual (English) 6
0. Read this first 6
0.1 What this document is based on 6
0.2 Verification level markers 7
0.3 Six things to know first 8
0.4 Installation and runtime environment 8
0.5 Version check (`bvt --version`) 10
Part 1. BOOMVALLEY Lab concepts 13
1.1 Purpose 13
1.2 Applicable research types 13
1.3 Core concepts 13
Organization is not a schema key 14
1.4 How to define organizational structure and information access rights 14
1.5 GNDCRW → BVL → MANIFEST-4 → FULLMT 15
1.6 Two ways GNDCRW is connected 15
Differences by installation 16
1.7 Supported and unsupported features 16
Supported 16
Not supported 17
Part 2. Files the researcher must create 18
2.1 Map of all files 18
2.2 `study.json` (`bvl-study-1`) 18
2.2.1 Common rules 18
2.2.2 Top-level keys 19
2.2.3 `protocol` — procedure, rounds, termination 19
2.2.4 `roles`, `profiles`, `actors` 21
2.2.5 `information` — sources and access rights 22
2.2.6 `proposals` — proposal schemas and parsers 23
2.2.7 `decisions[]` — decision rules 24
2.2.8 `world` and `initial_world_state` 25
2.2.9 The message an actor actually receives 25
2.2.10 `analysis` — pre-bound aggregation 26
2.2.11 `design` — experimental conditions 27
2.2.12 `plugin_config` 28
2.2.13 `schedule`, `humans`, `tools` 28
2.2.14 Common errors while writing `study.json` 29
2.3 `sources\*.json` — fixed source files 29
2.4 GNDCRW — inside a BVL study (generated automatically) 30
2.5 GNDCRW — standalone use (written by the researcher) 30
2.5.1 File layout and cross-references 31
2.5.2 `catalog.json` — `gndcrw-target-catalog/v0.1` 31
2.5.3 `base.json`, `candidate.json` — `gndcrw-surface/v0.1` 32
2.5.4 `pair.json` — `gndcrw-pair/v0.1` 33
2.5.5 `declaration.json` — `gndcrw-declaration/v0.1` 34
2.5.6 `manifest.json` — `gndcrw-manifest/v0.2` 35
2.5.7 Computing SHA-256 and size 35
2.5.8 Common errors 36
2.5.9 Building the files from scratch — worked example 37

2.6 FULLMT — input package 40
2.6.1 What you give it 40
2.6.2 MANIFEST-4.json 40
2.6.3 What the researcher creates and what is created automatically 40
2.6.4 FULLMT result JSON 41
Part 3. Order of writing files 42
3.1 Steps 42
3.2 Studies with two or more conditions 43
Part 4. Templates you can copy and use 45
4.1 List of registered components 45
Provider (Adapter) — profiles[].provider 45
Renderer — profiles[].renderer 45
Parser — proposals.contracts[].parser 45
Decision rules — decisions[].rule 45
Termination rules — protocol.round.termination.rule 45
Memory transform — transform of memory_update 45
Information transform — information.access[].transform 45
World — world.plugin, config.world of world_transition 46
4.2 How to read the templates 47
4.3 Template A — minimal study (1 actor, scripted responses) 48
4.4 Template B — using a real model 49
4.5 Template C — a design with two conditions 50
4.6 Template D — the official 4-agent B2B negotiation demo (negotiation between two organizations with manager approval, real model) 52
4.7 Template for using GNDCRW alone 58
Part 5. Windows CMD Run Guide 59
5.1 Preparation 59
5.2 Common points for bvl 60
5.3 bvl validate 60
5.4 bvl check 61
5.5 bvl run 61
5.6 bvl status and bvl resume 63
5.7 bvl package 63
5.8 bvl verify 64
5.9 bvl analyse 65
5.10 gndcrw capabilities and gndcrw check 65
5.11 fullmt 67
5.12 The whole flow at a glance 68
5.13 Reading the run record — who received what, who answered what, what was decided 68
Part 6. Research Interpretation and Limits 72
6.1 Distinguish six things 72
6.2 What a GNDCRW RUN guarantees and what it does not 72
6.3 What FULLMT VERIFIED guarantees and what it does not 73
6.4 Relationship between bvl verify and fullmt 74
6.5 Points to note in studies with a real model 74
6.6 Wording you can use in a report 75
Part 7. Troubleshooting 76
7.1 Schema errors in input JSON 76
7.2 Actor / Information Source reference errors 77
7.3 Gate refusal (gate-2 REFUSED, exit code 8) 77

7.4 GNDCRW Manifest errors (standalone use, exit code 3) 78
7.5 GNDCRW `BLOCK` / `UNVERIFIABLE` (standalone use) 79
7.6 AI Provider connection errors 80
7.7 Model output parse failures 81
7.8 BVL run failures 81
7.9 FULLMT `REJECTED` / `ERROR` 83
Appendix A. Check results **85**
Appendix B. Unconfirmed items **87**
Appendix C. Main source files **88**
Appendix D. Related documents **89**

BOOMVALLEY Research Toolkit — Researcher Manual (English)

Intended readers: researchers who are installing BOOMVALLEY Research Toolkit for the first time and want to run their own research in BOOMVALLEY Lab (BVL).

The question this document answers: **"To run my research in BOOMVALLEY Lab, which files must I create, and what do I write in each one, and how?"**

Item	Value
Edition	**Final edition for Toolkit `1.0.0`** (2026-10-16). Earlier: first edition 2026-10-13 (rc2), revision 2026-10-16 (rc5)
Applies to	BOOMVALLEY Research Toolkit `1.0.0` (install location `C:\Program Files\BOOMVALLEY Research Toolkit`)
Checked on	A Toolkit `1.0.0-rc6` installation on Windows 11 Home, 2026-10-16. Every command in Part 5 and every template in Part 4 was run (section 0.2)
BOOMVALLEY Lab	Release `v1.0.0` (source commit `db21f42`, package self-reported version `1.0.0rc3`)
GNDCRW	`0.2.0`
FULLMT	`0.12.0`
Source baseline	BOOMVALLEY Lab development repository (withheld for anonymous review) `main` (commit `628e18c`, Toolkit `1.0.0-rc6` merged, 2026-10-16)
Location of this document	`boomvalley-lab` repository `docs/manual/BOOMVALLEY_RESEARCHER_MANUAL_EN.md`
Operating principles	Internal project documentation (not included in this edition)

0. Read this first

0.1 What this document is based on

This document was written by reading **the source code of the installed implementation, the contract identifiers in it, and the official examples included in the installation**, and then by **running the commands and templates on an installed Toolkit**. Retired `bvl-experiment-2` input is not treated as a current feature.

The first edition (2026-10-13) was written by reading the Toolkit `1.0.0-rc2` installation. This final edition was checked by running on a Toolkit `1.0.0-rc6` installation. The release candidates relate as follows (`packaging/toolkit/README.md`, `packaging/toolkit/VERSIONING.md`).

- The BOOMVALLEY Lab and GNDCRW component bundles are byte-identical from rc1 through rc6. The FULLMT bundle is identical from rc2 onward. Therefore content based on source under `PKG`, `GND`, and `FMT` is the same in every one of them.
- rc3 also installs the GNDCRW package into the BOOMVALLEY Lab runtime environment (section 1.6).
- rc4 added a Toolkit version check command, and rc5 named that command `bvt` (section 0.5).
- rc6 installs this manual under `docs\` and adds two Start menu shortcuts (section 0.4).

- rc7 asks for acceptance of the Apache License 2.0 in Setup, installs the licences and notices, the PDF editions of this manual and of the getting-started guide, and the official sample (`samples\manual_negotiation_demo\`), and adds `bvt gndcrw-digest` (sections 0.4 and 2.5.9). The component bundles did not change.

Toolkit `1.0.0` and the check results. Toolkit `1.0.0` is to carry the same three components as rc6 (BOOMVALLEY Lab `1.0.0rc3`, GNDCRW `0.2.0`, FULLMT `0.12.0`). The run results in this manual therefore apply to `1.0.0` as long as `bvt --version` shows those component versions (section 0.5). If it shows other component versions, the run results of this manual do not carry over.

Evidence paths are written with the following abbreviations.

Abbreviation	Actual path
`PKG`	`C:\Program Files\BOOMVALLEY Research Toolkit\components\BOOMVALLEY-Lab\venv\Lib\site-packages\boomvalley_lab`
`EX`	`C:\Program Files\BOOMVALLEY Research Toolkit\components\BOOMVALLEY-Lab\examples`
`GND`	`C:\Program Files\BOOMVALLEY Research Toolkit\components\gndcrw\venv\Lib\site-packages\gndcrw`
`FMT`	`C:\Program Files\BOOMVALLEY Research Toolkit\components\fullmt`

Examples in the anonymous review edition. This edition includes one example only: the buying center study `examples\bcp_demo\`, a design with two conditions in the form of section 4.5 (a purchasing director who sees one or all four functional evaluations; guide: `examples\bcp_demo\SAMPLE_GUIDE_EN.md`). The negotiation demo `examples\manual_negotiation_demo\` that this manual uses as its worked example (sections 2.5.9, 4.6 and 5.13) and the other official examples that it cites as `EX\...` (for example `EX\k0_study\`, `EX\conformance\`, `EX\longitudinal\`, `EX\external\`, `EX\provider\`, `EX\human\`, `EX\tool\`, `EX\bvl_input_v2\` and `EX\b2b_frontline_internal_exchange_qwen_100_158\`) are part of the Toolkit installation and the development repository and are not included in this edition. Where this manual quotes their content, the quotation is the reference; the run results reported for them are from the Toolkit installation (section 0.2, Appendix A). In this edition, GNDCRW runs inside `bvl check` and `bvl run` on the two cells of `examples\bcp_demo\` (section 2.4); no set of standalone GNDCRW input files is included. The commands of section 5.13 for reading a run record are applied to this example in its sample guide, step 2.

0.2 Verification level markers

For this final edition, the commands and templates were **actually run** on a Toolkit `1.0.0-rc6` installation (2026-10-16, Windows 11 Home, local Ollama `qwen2.5:7b-instruct`). The evidence level of each item is marked as follows.

Marker	Meaning
(no marker)	Confirmed by reading the installation's source code or official examples. Not confirmed by running it.
[Run-confirmed]	Actually run on the Toolkit `1.0.0-rc6` installation on 2026-10-16, with the output shown or described here.
[Unverified]	Only the form was confirmed in the source, or the content is inferred. Check it yourself before use.

What was run (details in Appendix A):

- Every command of Part 5: `bvt`, `bvl validate`, `check`, `run`, `status`, `resume --plan`, `package`, `verify` (all level options), `analyse`, `gndcrw --version`, `capabilities`, `check` (text, JSON, `--gate`, `--output`), and `fullmt` (`--quiet`, `--out`, `--locale`).
- Templates A, B, C (both examples, both cells each) and D of Part 4, each through `validate` → `check` → `run` → `package` → `verify` → `fullmt` → `analyse`. B and D called the real model.
- All 31 official `bvl-study-1` examples in `EX` with `validate` and `check`. The 27 cells of the 24 that need no real model, human or tool were also run, packaged and checked with FULLMT.
- The error cases of Part 7 that can be produced on purpose (Appendix A).

Not run: Toolkit Setup screens, uninstall, a second Windows user account, human and tool actor procedures, and the items in Appendix B.

0.3 Six things to know first

1. **The only file the researcher must create is `study.json`.** Source files in the `sources/` folder are added to it as needed. All other files are made by the tools.
2. **In `bvl-study-1` research, you do not write GNDCRW input files by hand.** If there are two or more experimental conditions, BVL derives comparison edges from the `design` in `study.json` and itself creates the five documents to pass to GNDCRW (section 2.4). You write the six files such as `declaration.json` yourself only when you use the `gndcrw` command standalone, separately from BVL (section 2.5).
3. **A design with two or more conditions requires Toolkit `1.0.0-rc3` or later.** In rc2, the BVL runtime environment did not have the `gndcrw` Python package, so such designs were rejected with `BVL-GATE-004`. From rc3, Setup also installs that package into the BVL environment (section 1.6). On the rc6 installation, the official factor designs passed the gate with a GNDCRW `RUN` verdict and ran to `VERIFIED` **[Run-confirmed]**.
4. **`VERIFIED` and `RUN` are not judgments of scientific validity.** What they guarantee and what they do not guarantee is in Part 6.
5. **Check the version with `bvt --version`.** There is no `bvl --version` (`USAGE_ERROR OPTION_NOT_SUPPORTED`, exit code 2; `packaging/toolkit/KNOWN_NONBLOCKERS.md` N-32).
6. **Keep the study folder path short**, for example `C:\work\my_study`. Windows limits a path to 260 characters unless long paths are enabled, and the files inside a run folder add about 115 characters to the run folder's path. A run folder deep inside the user profile fails at once with `execution: [Errno 2] No such file or directory: '...\.<hash>.json.partial'` **[Run-confirmed]** (section 5.1).

0.4 Installation and runtime environment

Evidence: `packaging/toolkit/README_INSTALLER.md`, `packaging/toolkit/INSTALL_CONTRACT.md`, `packaging/toolkit/TOOLKIT_RELEASE_MANIFEST.json`. What was confirmed on the rc6 installation is marked **[Run-confirmed]**. The Setup screens, Setup exit codes other than 0 and uninstallation were not run for this manual.

Installer file. It is `packaging/toolkit/release/BOOMVALLEY-Research-Toolkit-v1.0.0-Setup.exe` in the `boomvalley-lab` repository (the file name follows the `-v<version>-Setup.exe` rule of every release candidate). Its SHA-256 is in the `.sha256` file next to it. This manual is itself installed by that Setup, so it cannot print the Setup's own hash; compare the downloaded file with the `.sha256` file. There is no code

signature, so running the downloaded file shows a SmartScreen warning, and the UAC window shows the publisher as unknown.

Item	Requirement
Operating system	Windows (64-bit install mode)
Privileges	Administrator privileges. Setup requests them via UAC. There is no way to install with lower privileges.
Python	**3.11 or later must already be installed.** Setup looks for `py -3`, `python`, `python3` in that order, or uses only the interpreter specified with `/PYTHON=`. It does not download or install Python. On a PC shared by several people, Python installed for all users is better.
Install location	Fixed at `C:\Program Files\BOOMVALLEY Research Toolkit`. `/DIR=` is rejected.
Network	Not needed for installation.
Real model	Optional. If you need an OpenAI-compatible server (e.g. Ollama `http://127.0.0.1:11434/v1`), prepare it separately. The official demo was run with Ollama `0.35.1` and `qwen2.5:7b-instruct`.

How to run.

```
certutil -hashfile BOOMVALLEY-Research-Toolkit-v1.0.0-Setup.exe SHA256        compare with the
.sha256 file
BOOMVALLEY-Research-Toolkit-v1.0.0-Setup.exe                                    interactive
BOOMVALLEY-Research-Toolkit-v1.0.0-Setup.exe /VERYSILENT /SUPPRESSMSGBOXES      unattended install
(from an administrator window)
BOOMVALLEY-Research-Toolkit-v1.0.0-Setup.exe /PYTHON="C:\...\python.exe"        specify Python
```

On the check PC, Setup was run as an upgrade from rc2 to rc6 with `/PYTHON=` pointing to Python `3.13` (64-bit); it ended with exit code 0 and the state `installed (upgrade)` **[Run-confirmed]**. The interactive screens were not checked.

- If a per-user rc1 Toolkit, or a separately installed BOOMVALLEY Lab / GNDCRW / FULLMT, is in its default location, Setup refuses. Remove them first with their own uninstallers.
- Running it again on a PC where the Toolkit is already installed is a repair or upgrade. Installing a lower version over the install state of a higher Toolkit version is rejected (`PF-09`).
- A Setup log is always kept (`%TEMP%\Setup Log <date> #nnn.txt`, or `/LOG=<file>`).
- When installation is finished, open a **new** terminal. Windows opened before installation do not pick up the PATH.

What is installed.

```
C:\Program Files\BOOMVALLEY Research Toolkit\
  TOOLKIT_RELEASE_MANIFEST.json   toolkit-state.json   toolkit-version.ps1   logs\   unins000.exe
  bin\          bvl.cmd  boomvalley-lab.cmd  gndcrw.cmd  fullmt.cmd  bvt.cmd  boomvalley-toolkit.cmd
<- system PATH
  components\   BOOMVALLEY-Lab\  gndcrw\  fullmt\
  docs\         BOOMVALLEY_RESEARCHER_MANUAL_EN.md   <- this manual
```

Only one entry, `C:\Program Files\BOOMVALLEY Research Toolkit\bin`, is added to the system PATH. The three components do not touch PATH themselves. The layout above, the six commands in `bin\` and the single PATH entry were confirmed on the rc6 installation **[Run-confirmed]**.

This manual on the installed PC. Setup copies this manual into `docs\` and adds a Start menu folder `BOOMVALLEY Research Toolkit` for all users with shortcuts to it. Each shortcut opens the file in Notepad

(`%WINDIR%\notepad.exe`), so it works on any Windows without a Markdown program. Notepad shows the Markdown source as plain text (`#` headings, `|` tables). To read it formatted, open the file in `docs\` with any Markdown viewer; Setup changes no file association. The shortcuts and the text in Notepad were confirmed for the administrator account that ran Setup **[Run-confirmed]**; a second user account was not checked.

From rc7. The Start menu folder has five shortcuts. The four shortcuts of the manual and the getting-started guide open the PDFs in `docs\` with the PDF program of the computer; `Sample Demo` opens the sample folder. The Markdown sources stay in `docs\`. An interactive Setup shows the Apache License 2.0 after the welcome page and goes on only when it is accepted. `LICENSE.txt`, `LICENSE-DOCS.txt`, `NOTICE.txt` and `THIRD_PARTY_NOTICES.txt` are installed in the toolkit directory. Not yet checked on an rc7 installation **[Unverified]**.

Setup exit codes.

Code	Meaning
0	Install or repair complete
1	Rejected before writing: no administrator privileges, `/DIR=`, preflight (`PF-01` .. `PF-10`) failure
2, 5	Cancelled by the user (a first install is rolled back)
7	A component installer or a Toolkit-own step (shims, PATH) failed. A first install is rolled back; for a repair or upgrade, `status: failed` is recorded in the state
3, 4	Fatal error in Inno Setup itself

Uninstall. Remove it from "Installed apps" or run the following. Python and files outside the Toolkit folder (research folders, run results) remain.

```
"C:\Program Files\BOOMVALLEY Research Toolkit\unins000.exe"
```

Python is still needed to run. If you delete or move the Python that Setup chose, the commands do not work. Running Setup again repairs it.

0.5 Version check (`bvt --version`)

Evidence: `packaging/toolkit/VERSIONING.md` section 4, `packaging/toolkit/INSTALL_CONTRACT.md` section 16, `packaging/toolkit/installer/toolkit-version.ps1`. All behavior below was run on the rc6 installation **[Run-confirmed]**.

```
bvt --version
bvt --json
bvt --help
```

- `bvt` is the Toolkit's own command. It reads the Toolkit product version and the installed versions of each component (BOOMVALLEY Lab, GNDCRW, FULLMT, the GNDCRW package inside the BOOMVALLEY Lab environment), and compares them with the versions declared in the release manifest.
- `boomvalley-toolkit` is the name from rc4 and remains as an alias. Its output and exit codes are the same.

- Running `bvt` alone with no arguments is the same as `--version`. `--json` prints the same content as a single JSON document (`"schema": "bvl-toolkit-version/1"`, top-level keys `toolkit`, `components`, `links`, `result`).
- It only reads. It does not write files or change state.

Actual output on the rc6 installation (on `1.0.0` the first three version lines read `1.0.0`, and the two manifest SHA-256 lines show the hash of the `1.0.0` release manifest):

```
> bvt --version
BOOMVALLEY Research Toolkit 1.0.0-rc6
  C:\Program Files\BOOMVALLEY Research Toolkit

Toolkit: the product as a whole (one product version)
  product version, installed      1.0.0-rc6  (toolkit-state.json)
  product version, declared       1.0.0-rc6  (TOOLKIT_RELEASE_MANIFEST.json)
    match                         yes
  release manifest, SHA-256       ad85b86241dad8435adf1f1d1554ce570440459e231458de841ea612f91b521f
(the installed file)
  release manifest, recorded      ad85b86241dad8435adf1f1d1554ce570440459e231458de841ea612f91b521f
(toolkit-state.json)
    match                         yes
  installation status             installed (upgrade, 2026-10-16T06:42:31Z)

Components: each has its own version; none of them is the toolkit product version
  what                                 installed  declared  match  how the installed version was
read
  BOOMVALLEY Lab package               1.0.0rc3   1.0.0rc3  yes    boomvalley-lab --version
(package_version)
  GNDCRW package                       0.2.0      0.2.0     yes    gndcrw --version
  FULLMT package                       0.12.0     0.12.0    yes    READER_VERSION in the installed
app\epc_reader.py (FULLMT has no version option)
  gndcrw package inside boomvalley-lab  0.2.0      0.2.0     yes    the gndcrw distribution in the
environment of boomvalley-lab

Result: CONSISTENT
  installed = read from this installation now; declared = what the release manifest says.
  Versions are compared as text, exactly. UNKNOWN = could not be read; it is never a match.
```

What to look at. The `Result` line, and the four component lines. The run results in this manual hold for the component versions shown above (section 0.1).

Result	Exit code	Meaning
`CONSISTENT`	0	All comparisons match
`MISMATCH`	1	One or more mismatches (including when a repair or upgrade failure is recorded in the state)
`UNKNOWN`	3	No mismatch, but some values could not be read
—	2	Unknown argument

Do not confuse the version labels (`VERSIONING.md` section 2).

What	Version
BOOMVALLEY Research Toolkit (product, Setup)	`1.0.0` (checked on `1.0.0-rc6`)
BOOMVALLEY Lab release tag	`v1.0.0`

What	Version
BOOMVALLEY Lab Python package `boomvalley`	`1.0.0rc3`
GNDCRW	`0.2.0`
FULLMT reader	`0.12.0`

The Toolkit's `1.0.0` and the BOOMVALLEY Lab package's `1.0.0rc3` are numbers of different products. It is intended that the package in the `v1.0.0` release reports `1.0.0rc3`. Comparison is literal, so `1.0.0rc3` and `1.0.0-rc3` are different.

Each component's own version command: `gndcrw --version` (`gndcrw 0.2.0`), `boomvalley-lab --version` (`package_version` in the JSON). FULLMT has no version option.

Part 1. BOOMVALLEY Lab concepts

1.1 Purpose

BOOMVALLEY Lab is an experiment runner that executes, as declared, **a process in which multiple actors (AI models, people, tools) view information, make proposals, and make decisions according to a fixed procedure, and those decisions change an environment**, and that leaves that process behind as evidence that can be recomputed later.

The researcher does not write a program. The whole study is declared as **one data file (`study.json`)**. In the source's words, "A study is data only" (`PKG\lifecycle\study.py`). The declaration cannot contain code or module paths; it refers to components registered in the installation by name only.

1.2 Applicable research types

What can be declared with the current implementation:

- Effects of role division and decision authority within an organization (single decision, majority, plurality, veto)
- Effects of information access rights (who sees what, the difference between originals and summaries)
- Effects of model replacement (changing only one actor's model in the same procedure)
- Effects of decision-making and memory across multiple rounds
- Price negotiation with a counterpart outside the organization
- Procedures in which a person takes part as an actor, and procedures in which a tool takes part as an actor

What the current implementation **cannot do** is listed separately in section 1.7. You must read it before designing.

1.3 Core concepts

Concept	Location in `study.json`	Meaning
Actor	`actors[]`	One actor called within the procedure. One of a model, a person, or a tool responds.
Role	`roles[]`, `actors[].role`	A role name attached to an actor. **It is only a label and grants no authority.** Authority is set separately in Information and Decision.
Profile	`profiles[]`, `actors[].profile`	"What" the actor responds with: provider, model name, renderer, options.
Information	`information`	Sources (source) and access rules (access). Access is **default-deny**. An actor sees something only if an allow rule exists.
Proposal	`proposals`	A structured proposal obtained by interpreting an actor's response. Defined by a schema and a parser.
Decision	`decisions[]`	A rule that makes one decision from proposals, and its list of authority members.

Concept	Location in `study.json`	Meaning
World	`world`, `initial_world_state`	The environment to which decisions are applied. It has schemas for state and actions, and rules that judge actions.
Protocol	`protocol`	A node graph that sets what happens in what order within one round, plus round repetition and termination rules.
Memory	`memory_update` in `protocol.nodes[]`	An explicit record that carries results that happened in the World to later rounds.
Design	`design`	Experimental factors and levels. A combination of conditions is a design cell.
Analysis	`analysis`	Aggregate metrics fixed in advance before the run.

Organization is not a schema key

`study.json` has **no** key named `organization` (the `REQUIRED` and `OPTIONAL` lists in `PKG\lifecycle\study.py`). The researcher **expresses** an organization as a combination of the following three.

- Who is there: `actors` and `roles`
- Who sees what: `information.access`
- Who decides: `decisions[].authority` and `rule`

For example, to distinguish a "purchasing organization" from a "supplying organization", name the actors accordingly, allow each organization's internal sources only to that organization's actors, and put only that organization's actors as authority members in each organization's decision node. BVL itself does not know that two actors belong to the same organization.

1.4 How to define organizational structure and information access rights

Organizational structure is set by the node connections of the Protocol and the authority members of the Decision.

- Reporting structure: declare a subordinate's `proposal_parse` result as an `actor_output` source and allow it to the superior.
- Decision authority: set by who is in the decision node's `authority` and what the `rule` is.
- Call order: set by the node's `depends_on`.

There is one point to watch. `depends_on` sets **only order**. To let another actor **see** one actor's output, you must separately declare an `actor_output` source and an `ALLOW` rule (`EX\k0_study\README.md`).

Information access rights follow these principles (`PKG\kernel\information.py`).

- Default-deny: no actor sees any source without an `ALLOW` rule.
- If both `ALLOW` and `DENY` apply to the same actor and source in one condition, this is not resolved by order; it is rejected (`BVL-INFO-003`).
- Before a model call, BVL checks whether the content of a denied source got into the request (`BVL-INFO-005`).

There is one exception. The only renderer, `bvl.json-renderer:1`, shows **the entire World state to all actors** (`JsonRendererV1` in `PKG\lifecycle\plugins.py`). Values placed in the World state cannot be

hidden by access rules. Information that only specific actors should see goes in a `frozen_resource` source, not in the World state.

1.5 GNDCRW → BVL → MANIFEST-4 → FULLMT

Stage	Tool	What it does	What it does not do
Preflight	GNDCRW `0.2.0`	Before the run, checks whether two experimental conditions **differ only in what is declared and are otherwise the same**, and returns `RUN` / `BLOCK` / `UNVERIFIABLE`.	Does not run the experiment. Does not fix anything. Does not claim scientific validity.
Run	BVL `bvl run`	Runs a declaration that passed the gate and records requests, responses, proposals, decisions, and World transitions as evidence.	Does not retry failed calls. Does not rewrite responses.
Seal	BVL `bvl package`	Writes `MANIFEST-4.json` into the run folder, binding all evidence files by hash and size.	Cannot make a scientific evidence package (`SCIENTIFIC`). Makes only `CONFORMANCE`.
Independent verification	FULLMT `0.12.0`	Recomputes from the package bytes alone, without using BVL code, and returns `VERIFIED` / `REJECTED` / `ERROR`.	Does not call models again. Does not re-run World code. Does not re-run GNDCRW.

`MANIFEST-4.json` is the list file of a sealed run folder. Its schema identifier is `boomvalley-lab-manifest-4`, and it holds the path, role, SHA-256, and size of every evidence file in the folder (`PKG\kernel\manifest4.py`).

The difference between BVL and FULLMT is **who computes**. `bvl verify` is the tool that ran the study recomputing by itself; `fullmt` is a separate program that shares no code with BVL recomputing the same bytes (header of `FMT\app\manifest4_reader.py`).

1.6 Two ways GNDCRW is connected

This distinction is the easiest part of this manual to confuse.

Mode A — automatically inside BVL (part of the study run)

- If the `design` in `study.json` has factors, BVL derives comparison edges between adjacent levels.
- For each comparison edge, **BVL generates GNDCRW's five documents (declaration, target catalog, base surface, candidate surface, pair context) directly from the compiled plan** (`edge_documents` in `PKG\kernel\coverage.py`).
- BVL loads the `gndcrw` Python package **inside its own process** and gets the verdict (`_gndcrw_engine` in `PKG\cli\bvl.py`). It does not run the `gndcrw` command.
- The gate opens only if the result of every comparison edge is `RUN`.
- The researcher does not write GNDCRW files. The `--gndcrw` option of `bvl run` is not accepted in `bvl-study-1` research (`INPUT_LOCK_ONLY` in `PKG\cli\bvl.py`).

Mode B — standalone use of the `gndcrw` command (not connected to a BVL run)

- The researcher writes the six files themselves and runs `gndcrw check manifest.json`.
- The result is **not bound** to a BVL run or to `MANIFEST-4.json`. It is only a separate preflight record.

In **research with only one condition** (a design without factors), there are no comparison edges, so the Mode A GNDCRW call does not happen. The gate opens with only the coverage check and the plugin preflight.

Differences by installation

For Mode A to work, the `gndcrw` package must be importable in the Python environment in which BVL runs.

- BVL runs in its own virtual environment (`...\components\BOOMVALLEY-Lab\venv`), and that environment does not use system packages (`include-system-site-packages = false` in `pyvenv.cfg`). GNDCRW is installed in a separate virtual environment (`...\components\gndcrw\venv`).
- **Toolkit `1.0.0-rc2` :** the BVL environment did not have `gndcrw` . So every design with factors (two or more conditions) was rejected with `BVL-GATE-004 ... engine failed` . The `gndcrw` command itself worked.
- **Toolkit `1.0.0-rc3` or later (including rc6 and `1.0.0`):** Setup also installs the GNDCRW 0.2.0 wheel into the BVL environment offline, and provides the commands only after confirming it can be imported (`packaging/toolkit/INSTALL_CONTRACT.md` section 15). The `gndcrw package inside boomvalley-lab` line of `bvt --version` shows this connection.

On the rc6 installation, the official factor designs `EX\conformance\b3_model_as_factor.json` , `EX\conformance\b2_summary_raw_access.json` and `EX\k0_study\study_2cell.json` passed `bvl check` in every cell, with a GNDCRW `RUN` verdict recorded in `evidence-v2\gndcrw_results\` of the run, and each cell ran to a FULLMT `VERIFIED` package **[Run-confirmed]**. If you are using rc2, upgrade. Research with only one condition is not affected in any version.

1.7 Supported and unsupported features

Supported

Feature	Evidence
Real model calls to an OpenAI-compatible server (e.g. Ollama's `/v1`)	`PKG\providers\openai_chat.py` , `EX\provider\local_model_study.json` **[Run-confirmed]**
Running with predefined responses (checking the procedure without a model)	`bvl.scripted-adapter:1` , `PKG\lifecycle\plugins.py`
Four decision rules: single, majority, plurality, veto	`PKG\kernel\decisions.py` (single and majority in the official examples and veto in Template D were run **[Run-confirmed]**; plurality was not run)
Passing actor output within one round	`actor_output` source, `PKG\kernel\information.py` **[Run-confirmed]**
Round repetition and three termination rules	`PKG\kernel\rounds.py`
Memory of World results across rounds	`memory_update` , `PKG\kernel\memory.py` **[Run-confirmed]**
Multiple episodes and carrying memory across episodes	`schedule` , `carry` , `EX\longitudinal\` **[Run-confirmed]**
Draft-review-revise block (up to 3 revisions)	`bounded_revision` , `PKG\kernel\revision.py` (`EX\k0_study\study_revision.json` **[Run-confirmed]**)
Human actors, tool actors	`humans` , `tools` , `EX\human\` , `EX\tool\` (the official examples pass `validate` and `check` ; running them was not checked **[Unverified]**)

Feature	Evidence
Declaring and running factor designs	`design`, `PKG\kernel\coverage.py` (requires Toolkit rc3 or later, section 1.6) **[Run-confirmed]**

Not supported

What you cannot do	Reason (source evidence)
Writing prompt sentences or a system message directly in the declaration	There is only one renderer, `bvl.json-renderer:1`, and an actor receives only one block of JSON as the user message. Instructions must be put in a source file.
Showing another actor's output directly across rounds	The producing node of an `actor_output` must be a preceding node in the same round (`BVL-INFO-012`).
Accumulating conversation history	Each call is a single independent message. Earlier content is passed only through `memory_artifact`, and what it holds is only the results of World transitions.
Retrying on call failure, repairing responses	The provider is called only once. `RETRY` and `REPAIR` family policies are rejected at the declaration stage (`BVL-PROP-008`, `BVL-DEC-008`).
Continuing a run after a model call failure	A failed run ends as `FAILED` and cannot be continued with `bvl resume`. It also cannot be packaged.
Statistical comparison between conditions	`analysis` only aggregates within one run (one cell). There are no cross-condition comparison metrics.
Replicates of a design, selecting only some cells, per-cell exceptions	The cells of `design` are the full product of the factors.
`design.exclusions`	The grammar accepts it, but declaring it in this build gives `BVL-COV-005`.
`termination` node kind	It is in the grammar, but no handler is registered. Termination is done with `protocol.round.termination`.
`world_observation` source kind	Cannot be run (`BVL-INFO-002`).
Adding user-defined parsers, renderers, providers, or decision rules	Externally installed plugins accept only World kinds (`INSTALLED_TYPES` in `PKG\lifecycle\plugins.py`).
`SCIENTIFIC` grade packages	Cannot be made or verified. Only `CONFORMANCE` is possible.
Package signing	There is no signing feature. `NOT_SIGNED` is always attached to FULLMT results.
Running `bvl-experiment-2` input	Retired. Only `validate` and `check` work.

Part 2. Files the researcher must create

2.1 Map of all files

File	Category	Who creates it	Validation command
`study.json`	**Required**	Researcher	`bvl validate`, `bvl check`
Source files such as `sources\*.json`	Conditional (if `frozen_resource` is declared)	Researcher	`bvl validate`
Consent file for human actors	Conditional (if `humans` is declared)	Researcher	`bvl human enroll`
Sources in the tool work folder (site)	Conditional (if `tools` is declared)	Researcher	`bvl run --tool-site`
The five GNDCRW documents (inside BVL)	Automatic	**BVL** (for factor designs)	`bvl check`
The whole `RUN\` folder	Automatic	BVL `bvl run`	`bvl status`
`RUN\MANIFEST-4.json` and the evidence added at sealing	Automatic	BVL `bvl package`	`bvl verify`, `fullmt`
FULLMT result JSON	Automatic (when `--out` is given)	FULLMT	—
The six GNDCRW files (standalone use)	Optional	Researcher	`gndcrw check`

Recommended folder structure:

```
my_study\
  study.json
  sources\
    brief_a.json
    brief_b.json
```

The `path` of a `frozen_resource` is resolved **relative to the folder that contains `study.json`** (`PKG\lifecycle\study.py`). The run folder (`RUN`) may be placed inside or outside this folder.

2.2 `study.json` (`bvl-study-1`)

2.2.1 Common rules

- UTF-8 JSON. A key that appears twice is rejected. `NaN` and `Infinity` are not allowed (`PKG\core\strict_json.py`).
- **Every section is closed.** A key outside the defined keys, or a missing required key, gives `BVL-LIFE-001`.
- Do not use `null` values. For optional items, omit the key itself.
- Items defined as lists must not be empty (exceptions are noted in each section).

2.2.2 Top-level keys

Key	Category	Content
`schema`	Required	Exactly `"bvl-study-1"`. **Do not change it.**
`experiment_id`	Required	Non-empty string. The run name becomes `<experiment_id>.<cell>`.
`episode_key`	Required	Non-empty string. If `schedule` exists, it must equal its first item.
`root_seed`	Required	Integer of 0 or more. Changing it makes a different plan.
`protocol`	Required	Section 2.2.3
`initial_world_state`	Required	Object. Section 2.2.8
`roles`	Required	Section 2.2.4
`profiles`	Required	Section 2.2.4
`actors`	Required	Section 2.2.4
`proposals`	Required	Section 2.2.6
`decisions`	Required	Section 2.2.7
`information`	Optional	Section 2.2.5
`world`	Optional (effectively required in some conditions)	Section 2.2.8
`analysis`	Optional	Section 2.2.10
`design`	Optional	Section 2.2.11
`plugin_config`	Optional (effectively required for some providers)	Section 2.2.12
`schedule`	Optional	Section 2.2.13
`humans`	Optional	Section 2.2.13
`tools`	Optional	Section 2.2.13

Basis: `REQUIRED` and `OPTIONAL` in `PKG\lifecycle\study.py`.

2.2.3 `protocol` — procedure, rounds, termination

`protocol` has `nodes` (required), **exactly one of `rounds` or `round`**, and the optional item `constructs`. Writing both `rounds` and `round`, or omitting both, gives `BVL-LIFE-001`.

Repetition method 1: `rounds` (simple repetition)

```
"protocol": {"rounds": 3, "nodes": [ ... ]}
```

A positive integer. Exactly that many rounds run.

Repetition method 2: `round` (round contract)

```
"round": {
  "max_rounds": 5,
```

```
  "termination": {"rule": "...", "timing": "...", "config": { ... }}
}
```

`max_rounds` is a positive integer and has no default (`BVL-ROUND-002`). `termination` is one of three registered rules (`PKG\kernel\rounds.py`).

`rule`	`config`	`timing`	Behavior
`builtin.fixed-count:1`	`{"count": N}`	Omit, or `after_round_close`	Ends after N rounds. If `count` is greater than `max_rounds` , `BVL-ROUND-010` .
`builtin.state-predicate:1`	`{"path": [...], "op": "...", "value": ...}`	**Required**	Ends when the value at `path` in the World state satisfies the condition. `op` is `eq` , `ne` , `lt` , `le` , `gt` , `ge` .
`builtin.terminal-state:1`	`{"path": [...], "success": [...], "failure": [...]}`	**Required**	Ends when the value at `path` in the World state is one of the values in `success` or `failure` .

Allowed values of `timing` are `before_observation` , `after_decision` , `after_world_transition` , `after_round_close` . The last two rules read the World state, so a `world` section must exist and `path` must actually exist in `initial_world_state` (`BVL-ROUND-001`). If round number `max_rounds` closes without a termination verdict, the run ends by the upper limit.

Examples: `EX\external\b10_external_negotiation.json` (terminal-state), `EX\longitudinal\study_rounds.json` (fixed-count), `EX\longitudinal\study_termination_timing.json` (state-predicate).

`nodes[]` — the content of one round

Each node requires `id` and `kind` ; `depends_on` (list of strings) and `config` (object) are optional. `id` starts with an alphanumeric character, uses only alphanumerics, `_` , `.` , `-` , and must be unique. Dependencies must not form a cycle (`PKG\kernel\protocol.py`).

`kind`	`config`	Dependency rule
`actor_call`	`{"actor": "<actor_id>"}`	Free. Only sets call order.
`proposal_parse`	None	Depends on exactly one `actor_call` . `proposals.contracts` must have exactly one contract for this node.
`decision`	None (adding one gives `BVL-DEC-005`)	Depends on `proposal_parse` nodes. Exactly one proposal must come in from each authority member's actor. `decisions[]` must have an entry for this node.
`world_transition`	`{"world": "<World plugin id>"}`	Depends on exactly one `decision` .
`memory_update`	See below	Depends on exactly one `world_transition` .

A decision node does not learn whose proposal is whose from call order or names. It derives this by following `proposal_parse` → the `actor_call` it depends on → that node's `config.actor` (`PKG\kernel\decisions.py`).

A round can have two or more `world_transition` nodes (`EX\external\b10_external_negotiation.json` applies two in one round).

`memory_update` — memory

There are two forms, and they cannot be mixed within one study (`PKG\kernel\memory.py`).

Form	`config`	Delivery
Simple	`{"owner": "<actor_id>", "key": "<name>"}`	The renderer puts it into the message as the `memory` item.
Lifecycle	All five keys `{"owner", "key", "scope", "retention", "transform"}`	Declared as a `memory_artifact` source; only actors granted access see it.

Values for the lifecycle form:

- `scope` : `"episode"` or `"cross_episode"`
- `retention` : integer of 1 or more. Memory created in round r is valid from round r+1 through round r+ `retention` .
- `transform` : only `"builtin.outcome-memory-v2:1"` exists. The content of the memory is the `outcome` of that World transition.
- `owner` must be a declared actor, and the (`owner` , `key`) combination must not overlap across nodes.

Memory holds **only the results produced by the World**. It does not hold what actors said or thought.

Carry-over between episodes (`cross_episode`) additionally requires the keys `carry` and `regime_change` . For syntax and examples, follow `EX\longitudinal\README.md` and `EX\longitudinal\study_cross_episode_regime.json` .

`constructs[]` — draft-review-revise blocks

There is only one kind, `bounded_revision` (`PKG\kernel\revision.py`).

```
"constructs": [
  {"construct": "bounded_revision", "id": "allocation", "max_revisions": 1,
   "authors": ["analyst", "reviser"], "reviewers": ["controller", "director"],
   "schema": "allocation:1", "parser": "bvl.json-proposal-parser:1"}
]
```

- `max_revisions` : integer from 0 to 3 (`BVL-REV-002`).
- `authors` and `reviewers` each have exactly `max_revisions + 1` members, and **every step must be a different actor**.
- Optional keys: `depends_on` , `on_veto` (`FAIL_EPISODE` by default, or `NO_ACTION`).
- The block expands into the nodes `<id>.draft.0` , `<id>.draft.0.parse` , `<id>.review.0` , ..., `<id>.decision` . The final decision is `builtin.veto:1` . Make the World node depend on `<id>.decision` .
- The block creates its own contracts and decisions, so when you use only blocks, `proposals.contracts` and `decisions` can be `[]` . `proposals.schemas` is still required.

Examples: `EX\conformance\b6_veto_one_revision.json` , `EX\k0_study\study_revision.json` .

2.2.4 `roles` , `profiles` , `actors`

`roles` — list of role names. Starts with an alphanumeric character, uses only alphanumerics, `_` , `.` , `-` , no duplicates (`BVL-ACTOR-002`).

`profiles[]` — the means of response (`PKG\kernel\actors.py`)

Key	Category	Content
`profile_id`	Required	Unique name.
`provider`	Required	A registered provider. List in section 4.1.
`model`	Required	Scripted provider: the same name as a `responses` key in `plugin_config`. OpenAI-compatible provider: **must be exactly the same as the model name the server reports in its response** (if different, `BVL-ACTOR-007` after the call).
`renderer`	Required	`"bvl.json-renderer:1"` (the only one).
`options`	Optional	Options accepted by the provider. `seed` and `random_seed` cannot be included (`BVL-ACTOR-008`). Seeds are derived from `root_seed`.
`seed_policy`	Optional	`"bvl-seed-actor-1"` (default) or `"bvl-seed-shared-round-1"`. Writing the default gives the same plan hash as leaving the key out; the other value gives a different plan hash **[Run-confirmed]**. What difference it makes to real model responses was not checked **[Unverified]**. No official example uses this key.

Allowed `options` values for `bvl.openai-chat-adapter:1` (`OPTIONS` in `PKG\providers\openai_chat.py`):

Option	Range
`temperature`	0 to 2 inclusive
`top_p`	Greater than 0, up to 1 inclusive
`max_tokens`	Positive integer
`frequency_penalty`, `presence_penalty`	-2 to 2 inclusive
`response_format`	`"text"` or `"json_object"`

Other options are rejected at the declaration stage. Give no options to `bvl.scripted-adapter:1`.

`actors[]` — exactly three keys: `actor_id`, `role`, `profile`.

`profile` is either a profile name string or an object whose profile varies with a factor.

```
{"actor_id": "worker", "role": "worker",
 "profile": {"factor": "worker_model", "cases": {"a": "model-a", "b": "model-b"}}}
```

`cases` must contain every level of that factor (`BVL-ACTOR-004`). Example: `EX\conformance\b3_model_as_factor.json`.

No rule prevents several actors from using the same profile. Two actors sharing one profile, and one actor called twice in one round (two `actor_call` nodes with the same `actor`), both passed `check` and ran to `COMPLETED` with the scripted provider **[Run-confirmed]**. All official examples give each actor its own profile.

2.2.5 `information` — sources and access rights

It has both keys `sources` and `access`, and neither may be empty. If you do not need information control, omit the `information` section itself.

`sources[]`

`kind`	Keys	Content
`frozen_resource`	`source_id`, `kind`, `path`	A fixed source file. The plan binds **the SHA-256 and size of the content**, not the path. Changing the file content makes a different plan.
`actor_output`	`source_id`, `kind`, `producer`	Another actor's output. `producer` is an `actor_call` node (raw response) or a `proposal_parse` node (parsed proposal).
`memory_artifact`	`source_id`, `kind`, `producer`	The memory of a lifecycle-form `memory_update` node.

Constraint on `actor_output`: **every** call node of an actor granted that source must have `producer` as a preceding node in the same round. Otherwise `BVL-INFO-012`. There is no way to receive another actor's output from a previous round directly.

`access[]`

Key	Category	Content
`actor`	Required	A declared actor.
`source`	Required	A declared source.
`effect`	Required	`"ALLOW"` or `"DENY"`.
`when`	Optional	`{"<factor>": "<level>"}`. Applies only under that condition.
`transform`	Optional (`ALLOW` only)	`{"transform": "<id>", "config": {...}}`.

There is one registered transform, `builtin.redact-json-keys:1`. Its `config` is `{"keys": ["...", "..."]}`, and it removes those top-level keys from a JSON object source. Example: `EX\conformance\b2_summary_raw_access.json`.

2.2.6 `proposals` — proposal schemas and parsers

`schemas[]` — all five keys required.

```
{"schema": "bvl-proposal-schema-1", "schema_id": "order", "version": 1,
 "fields": {"order_qty": "integer"}, "allow_no_action": false}
```

- `schema` is exactly `"bvl-proposal-schema-1"`. **Do not change it.**
- The only types for `fields` are `integer`, `number`, `string`, `boolean`. Ranges, choice lists, and nested objects cannot be defined (`FIELD_TYPES` in `PKG\kernel\proposals.py`).

`contracts[]`

Key	Category	Content
`node_id`	Required	A `proposal_parse` node.
`schema`	Required	Format `"<schema_id>:<version>"`. Example: `"order:1"`.
`parser`	Required	`"bvl.json-proposal-parser:1"` (the only one).
`on_invalid`	Optional	`"PARSE_FAILURE"` (default) or `"NO_ACTION"`.

If `on_invalid` is `NO_ACTION`, the schema's `allow_no_action` must be `true` (`BVL-PROP-007`). Values such as `REPAIR`, `RETRY`, `REPROMPT`, `FALLBACK`, `COERCE` are rejected with `BVL-PROP-008`.

How strictly responses are parsed

- The response must be **one JSON object and nothing else**. Any explanatory text before or after it, or code block markers, cause failure (`BVL-PROP-004`).
- A missing field, an undeclared field, a wrong type, or a `null` value causes failure (`BVL-PROP-005`).
- There is no type conversion. `4.0` or `"4"` in an `integer` field causes failure.
- Under `PARSE_FAILURE` (default), a failure leads to **episode failure**. Under `NO_ACTION` , that proposal is recorded as "no action" and the procedure continues.

There is no separate valid response syntax that means "no action". A `NO_ACTION` proposal arises only from the combination of a parse failure and `on_invalid: NO_ACTION` .

When using a real model, `on_invalid: "NO_ACTION"` is recommended. With the default, a single format violation by the model fails the whole run, and a failed run can be neither resumed nor packaged.

2.2.7 `decisions[]` — decision rules

Key	Category	Content
`node_id`	Required	A `decision` node.
`rule`	Required	Table below.
`authority`	Required	List of authority actors.
`on_unresolved`	Optional	`"FAIL_EPISODE"` (default) or `"NO_ACTION"` .

`rule`	Number of authority members	Decision	When it is unresolved
`builtin.single-authority:1`	Exactly 1	That actor's valid proposal	That actor gives no action (`BVL-DEC-003`)
`builtin.majority:1`	2 or more	The same action proposed by a majority of authority members. Abstentions count in the denominator.	No majority (`BVL-DEC-006`)
`builtin.plurality:1`	2 or more	The single action proposed most often	Tie or no proposals (`BVL-DEC-006`)
`builtin.veto:1`	2 or more	The same action proposed by **all** authority members	Any one differs or abstains (`BVL-DEC-007`)

- None of the four rules accepts `config` .
- "The same action" means all field values of the proposals are equal.
- The three voting rules reject schemas that have a `number` type field (`BVL-DEC-011`).
- All authority proposals for one decision must follow the same schema.
- Using `FIRST` , `LAST` , `CALL_ORDER` (order-dependent) or `FALLBACK` , `DEFAULT` , `RANDOM` , `RETRY` (invented actions) for `on_unresolved` is rejected (`BVL-DEC-012` , `BVL-DEC-008`).
- For a `NO_ACTION` decision to reach the World, a `world` section must exist and that World must declare the meaning of no action (`BVL-WORLD-008`).

Expressing an approval structure: If you set [proposer, approver] as the authority of `builtin.veto:1` , the decision holds only when the approver answers with **exactly the same action** as the proposer. For the approver to see the draft, declare the draft's `proposal_parse` as `actor_output` and grant it to the approver **[Run-confirmed]** (section 4.6).

2.2.8 `world` and `initial_world_state`

`world`

Key	Category	Content
`plugin`	Required	A registered World. List in section 4.1.
`state_schema`	Required	`"<id>:<version>"`. Must equal that World's.
`action_schema`	Required	`"<id>:<version>"`. Must equal that World's.
`no_action`	Optional	If used, must equal that World's value.
`exogenous_events`	Optional	If used, must equal that World's list.
`rng_policy`	Optional	`"DETERMINISTIC"` (default) or `"SEEDED"`. If it differs from the World's value, `BVL-WORLD-013`.
`regime_events`, `regime_fields`	Optional	If used, must equal that World's values.

A mismatch gives `BVL-WORLD-002`; an unknown plugin gives `BVL-WORLD-001`.

When the `world` section is required

- When using a domain World (marked "domain" in the table in section 4.1): without it, `BVL-WORLD-004`.
- When using a `SEEDED` World (`bvl.shock-world:1`).
- When the termination rule is `state-predicate` or `terminal-state`.
- When a `NO_ACTION` decision can reach the World.

`bvl.ledger-world:1` and `bvl.tally-world:1` can be used without a `world` section. However, declaring `world` is recommended, because only then are state and action validated against schemas and World recomputation verification (`bvl verify --replay-world`) possible.

If `world` exists: there must be at least one `world_transition` node, and every `world_transition` must point to the same plugin.

`initial_world_state`

- Always an object.
- If `world` exists, every field of its state schema must be present with the correct type, and with a closed schema there must be no extra fields (`BVL-WORLD-003`).
- If the World defines value ranges (e.g., counts are 0 or more), those must also be respected (`BVL-WORLD-003`).

The state fields of each World are in the table in section 4.1. When writing an actual initial state, start from the `initial_world_state` of the official example in the table's "Example" column.

World verdicts: For each transition, one of the following is recorded: `ADMISSIBLE` (executed as is), `CLIPPED` (adjusted and executed), `INADMISSIBLE` (not executed), or `NO_ACTION` for a no-action input (`PKG\kernel\world.py`).

2.2.9 The message an actor actually receives

`bvl.json-renderer:1` sends one user message to the actor. Its content is the following JSON (`JsonRendererV1` in `PKG\lifecycle\plugins.py`).

```
{"actor": "<actor_id>", "round": 1, "world": { ...entire World state... },
 "information": {"<source_id>": "<content string of that source>", ...}}
```

- `information` is present only when there are granted sources. The value is the source content inserted **as a string**.
- If there is simple-form memory, a `memory` item is added.

Therefore **the instructions for the model must be written inside a `frozen_resource` source file.** Write in the source file what to do and what JSON format to answer in, and grant it to that actor. The approach of the official examples is in `EX\provider\sources\order_brief.json`.

```
{"task": "You decide how many units to add to a shared running total this round.",
 "reply": "Answer with one JSON object and nothing else: {\"amount\": <integer between 0 and 20>}.",
 "rule": "The target for the total is 12 units. Add what is still missing; add 0 once the total is 12
or more."}
```

2.2.10 `analysis` — pre-bound aggregation

```
"analysis": {
  "analysis_id": "order-summary",
  "metrics": [
    {"metric_id": "decisions", "kind": "unit_count", "table": "decisions"},
    {"metric_id": "mean_amount", "kind": "action_mean", "field": "amount"}
  ]
}
```

`kind`	Additional keys	Computation
`unit_count`	`table`: one of `calls`, `decisions`, `proposals`	Number of admitted units
`action_mean`	`field`	Mean of that numeric field in the actions requested by decisions
`action_share`	`field`, `equals` (string, boolean, integer)	Share of decisions where that field equals `equals`
`proposal_agreement`	None	Share of candidate proposals equal to the decision's requested action

- Every metric can take the optional key `by`: `episode_key`, `node`, `round`.
- Values come out as **reduced fraction strings** `"p/q"` (e.g., `"679/8"`). If there is nothing to compute over, `UNDEFINED`.
- Metrics read **the action requested by the decision**. Not the action the World actually executed (the adjusted value) (`EX\conformance\README.md`).
- Aggregation is only within one run (one cell). **There are no cross-condition comparison metrics.** The result always carries the `SINGLE_RUN_POPULATION` limitation (`PKG\kernel\bound_analysis.py`).
- Running `bvl analyse` on a study without `analysis` gives `BVL-LIFE-006`.

The key point is that analysis is declared **before** the run and bound into the plan. Changing metrics after the run makes a different plan.

2.2.11 `design` — experimental conditions

```
"design": {
  "factors": {"worker_model": ["a", "b"]},
  "contrasts": {"worker_model": ["actor.worker.profile.model"]}
}
```

`factors`: factor name → list of levels. Names use only alphanumerics, `_`, `-` (no dots). There are 2 or more levels, and **their order has meaning** (`BVL-COV-008`).

`contrasts`: for each factor, the list of **names of plan items that factor changes**. The set of factors and the set of keys must be the same, and each list must not be empty (`BVL-COV-009`).

cell: the full combination of all factor levels. Its name is `cell` followed by `.<factor>-<level>` in factor-name order. Example: `cell.worker_model-a` . A study without factors has only one `cell` .

A factor level can actually change only two things.

1. An actor's profile: the `{"factor", "cases"}` form of `actors[].profile`
2. Access rules: `when` in `information.access[]`

There is no syntax to change the initial World state, number of rounds, decision rules, and so on by factor.

Comparison edges: The researcher does not write them. They are created automatically between **adjacent levels** of each factor, for every combination of the other factors (`bvl-adjacent-isolation-1`). A 2×2 design has four comparison edges.

Names used in `contrasts` : write the exact plan item name, or write `prefix.*` (`PKG\kernel\coverage.py`). Common ones:

What changes	Name
An actor's model	`actor.<actor_id>.profile.model`
An actor's provider, options	`actor.<actor_id>.profile.provider` , `actor.<actor_id>.profile.options.<option>`
Sources granted to an actor	`information.grant.<actor_id>`
Access transform	`information.transform.<actor_id>.<source_id>.*`
Schema version of the access set	`information.schema`

For each comparison edge, BVL creates a GNDCRW declaration stating "items in `contrasts` **must differ**, and every other item **must be equal**". Therefore `contrasts` must list **everything** that factor actually changes. For example, adding a transform also changes the schema version of the access set, so `EX\conformance\b2_summary_raw_access.json` also lists `information.schema` . If you omit something, it is judged as "an item that must be equal differs", and the gate closes.

Running: One `bvl run` runs only one cell. If there are several conditions, specify them one at a time with `--cell` and run each in a different folder. Whichever cell you run, the gate checks all cells and all comparison edges.

For whether this runs depending on the Toolkit version, see section 1.6 (rc3 or later required).

2.2.12 `plugin_config`

Only two providers accept configuration (`PKG\lifecycle\plugins.py`).

`bvl.scripted-adapter:1`

```
"plugin_config": {
  "bvl.scripted-adapter:1": {
    "responses": {
      "scripted:manager:1": {"1": {"amount": 15}, "*": {"amount": 2}}
    }
  }
}
```

Model name → {round number string or `"*"` → response}. String responses are sent as is; other values are sent as JSON.

`bvl.openai-chat-adapter:1`

```
"plugin_config": {
  "bvl.openai-chat-adapter:1": {"endpoint": "http://127.0.0.1:11434/v1", "timeout_seconds": 300}
}
```

Key	Category	Content
`endpoint`	Required	API base address. Requests go to `<endpoint>/chat/completions` . The address cannot contain account information or a query string.
`timeout_seconds`	Optional	Positive number. Default 120.
`api_key_env`	Optional	**Name of the environment variable** that holds the token. Do not write the token itself. Over plain http, allowed only for local addresses.

If a profile uses this provider and there is no `endpoint` , it is rejected at the declaration stage.

2.2.13 `schedule` , `humans` , `tools`

`schedule` : `{"episodes": ["episode-1", "episode-2", ...]}` . The first item must equal `episode_key` . Every episode starts from `initial_world_state` . Example: `EX\longitudinal\study_cross_episode_regime.json` .

`humans[]` : actors answered by a person. All six keys are required: `actor` , `channel` , `response` , `provenance` , `consent` , `privacy` . That actor's profile must have `provider` `"bvl.operator-entry-channel:1"` , `model` `"bvl:human-participant:1"` , and no options. Running requires `bvl run --human-desk` and `bvl human enroll / pending / answer` . For the full procedure follow `EX\human\README.md` , and for a declaration example follow `EX\human\b11_human_manager.json` .

`tools[]` : actors answered by a tool. There are three registered tools: `bvl.table-lookup-tool:1` , `bvl.record-append-tool:1` , `bvl.notice-dispatch-tool:1` . To run, declare `argument_selection` and give `bvl run --tool-site` . For the full procedure follow `EX\tool\README.md` , and for a runnable declaration example follow `EX\tool\b12_tool_actor.json` .

This manual covers only a syntax summary of these two features. Read the READMEs above before using them.

2.2.14 Common errors while writing `study.json`

Symptom	Cause
`BVL-LIFE-001 ... has keys [...]; required [...], optional [...]`	A misspelled key or a missing required key. The message tells you the allowed keys.
`BVL-LIFE-001 ... protocol declares exactly one of rounds ... and round`	You wrote both `rounds` and `round`, or omitted both.
`BVL-PROP-001`	A `proposal_parse` node has no contract, or the node a contract points to does not exist.
`BVL-DEC-005`	An authority member's proposal does not come into the decision node. Check that the decision node's `depends_on` contains that actor's `proposal_parse`.
`BVL-INFO-012`	You granted `actor_output` to an actor that does not run after it in the same round.
`BVL-WORLD-003`	A field is missing from `initial_world_state`, or a type is wrong.
`BVL-WORLD-004`	You used a domain World without declaring a `world` section.
`BVL-PROP-007`	`on_invalid: NO_ACTION` but the schema's `allow_no_action` is `false`.
`BVL-ROUND-001`	The termination rule's `path` is not in the initial state, or you used a state-based termination rule without `world`.
Source file not found (`BVL-INFO-002`)	`path` is relative to the folder of `study.json`.

Validation commands: `bvl validate --experiment study.json`, then `bvl check --experiment study.json`. Some problems (unregistered parser, rule, or transform names; extra configuration keys) appear as gate reasons in `check`, not in `validate`. For example, an unregistered parser name passes `validate` and is refused by `check` with `BVL-PROP-001` and `BVL-COV-010`; `options` given to the scripted provider pass `validate` and are refused with `BVL-COV-002` **[Run-confirmed]**. Pass both commands before running.

2.3 `sources\*.json` — fixed source files

Item	Content
Path	Relative path from the `study.json` folder. By convention `sources\<name>.json`.
Category	Conditional. If a `frozen_resource` source is declared, that file must exist.
Creator	Researcher
Purpose	Fixed information and instructions for actors.
Format	The kernel does not enforce a format; it reads the bytes as is. However, the renderer inserts it as a UTF-8 string, so it must be UTF-8, and to use a transform (`redact-json-keys`) or tool argument selection it must be a JSON object.
Relation to other files	Pointed to by `information.sources[].path` in `study.json`. The plan binds the SHA-256 and size of the content.

Minimal example (`EX\conformance\sources\field_report.json`):

```
{"stock": 10, "demand": 12, "defect_log": "unit-17 cracked housing; unit-22 seal leak",
"complaint_count": 5}
```

Cautions:

- Changing even one character of the file changes the plan hash. Do not edit it after the run.
- The line ending style (CRLF/LF) is also part of the content. Make sure your editor does not change it on save.
- Write response-format instructions for a real model precisely. They must pass the strict parsing rules in section 2.2.6.
- **Keep source files inside the study folder.** BVL does not refuse a `path` with `..` or an absolute path: both passed `validate` and `check` and read the file outside the study folder **[Run-confirmed]**. Such a study cannot be moved or handed to someone else as one folder. A path with `\` (`sources\brief.json`) gives the same plan hash as `/`; `/` is recommended.

2.4 GNDCRW — inside a BVL study (generated automatically)

In a `bvl-study-1` study, **the researcher does not write** the documents passed to GNDCRW. The only thing the researcher writes is `design` in `study.json` (section 2.2.11).

Document	Who creates it	From what
declaration (`gndcrw-declaration/v0.1`)	BVL	`contrasts` items for the comparison edge's factor are `MUST_DIFFER`; all other plan items are `MUST_EQUAL`
target catalog (`gndcrw-target-catalog/v0.1`)	BVL	All items in the plans of the two cells
base surface, candidate surface (`gndcrw-surface/v0.1`)	BVL	Compiled plan values of the lower-level cell and the higher-level cell
pair context (`gndcrw-pair/v0.1`)	BVL	Fixed: `comparison_mode: CONTROL`, `status: ESTABLISHED`
manifest	Not created	Passed directly in memory, without going through files

Basis: `edge_documents` in `PKG\kernel\coverage.py`, `run_comparison_set` in `PKG\kernel\comparison.py`.

Conditions under which BVL accepts a result (`PKG\kernel\comparison.py`): the result is a `CONTROL` comparison under the `GNDCRW-STRICT-0.1` policy, the result's subjects match the documents BVL created, and `outcome` is `RUN`. `BLOCK` or `UNVERIFIABLE` becomes the gate reason `BVL-GATE-003:<comparison edge>:<result>`.

Studies without comparison conditions (no factors): there are no comparison edges, so GNDCRW is not called. An empty result set is recorded in `gndcrw_results` in the run evidence. In this case you cannot say "GNDCRW returned `RUN`". What you received is gate `CLEARED`.

Gate reason codes are in Part 7. The check command is `bvl check`.

2.5 GNDCRW — standalone use (written by the researcher)

Write the six files in this section only when you use the `gndcrw check` command **separately from BVL**. Its result is not bound to a BVL run or to `MANIFEST-4.json`.

GNDCRW answers one question. **In the assembled results of the two conditions, base (BASE) and candidate (CANDIDATE), is what was declared as must-differ actually different, and is what was**

declared as must-equal actually equal? The package's own description is "Known violation is BLOCK. Missing proof is UNVERIFIABLE. Broken input is an error." (`gndcrw-0.2.0.dist-info\METADATA`).

The installation does not include GNDCRW's normative specification documents. This section is based on reading the validation code (`GND\contracts\*.py` , `GND\manifest.py`).

2.5.1 File layout and cross-references

```
my_preflight\
  manifest.json      binds the five files by path, SHA-256, and size
  declaration.json   what must differ and what must be equal
  catalog.json       list of items that can be compared
  base.json          values observed under the base condition
  candidate.json     values observed under the candidate condition
  pair.json          how the two conditions are paired
```

This value in this file	Must equal this
`provider_id` , `catalog_id` , `profile_id` , `catalog_digest` of `declaration.catalog_ref`	The fields of the same name in `catalog`
`declaration.catalog_ref.catalog_version`	`"gndcrw-target-catalog/v0.1"`
`provider.provider_id` of `base` and `candidate` (if not null)	`catalog.provider_id`
The key set of `targets` in `base` and `candidate`	Exactly the same as the key set of `catalog.targets`
The key set of `entity_values` in an `ENTITY_FIELD` item	`composition.entity_ids` of that surface
`pair.base_surface_id` , `pair.candidate_surface_id`	`base.surface_id` , `candidate.surface_id`
`base.role` , `candidate.role`	`"BASE"` , `"CANDIDATE"`
`sha256` , `size` in `manifest.subjects.<role>`	The actual bytes of that file
`manifest.external_subject.digest` (if used)	`provenance.input_digests[<scheme>]` of both surfaces

Common to the five files: UTF-8 (no BOM), strict JSON, no duplicate keys. Almost all objects are closed, so undefined keys are rejected. A violation is `SUBJECT_CONTRACT_INVALID` (exit code 3).

2.5.2 `catalog.json` — `gndcrw-target-catalog/v0.1`

All seven keys required (`GND\contracts\catalog.py`).

Key	Content
`catalog_version`	Exactly `"gndcrw-target-catalog/v0.1"`
`catalog_id` , `profile_id` , `provider_id`	Non-empty strings
`catalog_digest`	The canonical JSON digest of this document excluding `catalog_digest` (section 2.5.7)
`targets`	Item name → description object
`composition_targets`	Exactly two keys, `entity_count` and `entity_set` . Values are fixed (exactly as in the example below)

Item description object (all four keys required):

- `level`: `SURFACE`, `ENTITY_FIELD`, `SUBSTRUCTURE`
- `value_kind`: `STRING`, `INTEGER`, `NUMBER`, `BOOLEAN`, `NULL`, `JSON`
- `availability`: `SUPPORTED`, `KNOWN_UNSUPPORTED`
- `equality`: only `JSON_STRUCTURAL`

Official example (`EX\bvl_input_v2\gndcrw\catalog.json`, whitespace reduced only):

```
{"catalog_version": "gndcrw-target-catalog/v0.1",
 "catalog_id": "fixture-catalog", "profile_id": "default", "provider_id": "fixture-provider",
 "catalog_digest": "sha256:1924b54a7f0d66dcbab94758d1313a70aa51d6bcbc9df76acb328e6db0f64463",
 "targets": {
   "manager.raw_state.recency": {"level": "ENTITY_FIELD", "value_kind": "INTEGER", "availability":
"SUPPORTED", "equality": "JSON_STRUCTURAL"},
   "manager.role_instruction": {"level": "SUBSTRUCTURE", "value_kind": "JSON", "availability":
"SUPPORTED", "equality": "JSON_STRUCTURAL"},
   "manager.payload.length": {"level": "SURFACE", "value_kind": "INTEGER", "availability":
"SUPPORTED", "equality": "JSON_STRUCTURAL"}},
 "composition_targets": {
   "entity_count": {"value_kind": "INTEGER", "availability": "SUPPORTED", "equality":
"JSON_STRUCTURAL"},
   "entity_set": {"value_kind": "STRING_SET", "availability": "SUPPORTED", "equality": "SET_EXACT"}}}
```

2.5.3 `base.json`, `candidate.json` — `gndcrw-surface/v0.1`

All nine keys required (`GND\contracts\surface.py`).

Key	Content
`surface_version`	Exactly `"gndcrw-surface/v0.1"`
`surface_id`	Non-empty string
`role`	`"BASE"` for the base file, `"CANDIDATE"` for the candidate file
`provider`	`provider_id`, `provider_version`, `adapter_id`, `adapter_version`. Each a string or null
`assembly`	`mode` (`DIRECT` or `RECONSTRUCTED`), `assembly_entrypoint` (string or null), `equivalence`
`targets`	Exactly one record per catalog item
`composition`	`entity_kind`, `entity_count`, `entity_ids`, `identity_namespace`
`provenance`	Open object. `input_digests` (name → digest) is required
`premises`	List (may be empty)

Rules:

- If `assembly.mode` is `DIRECT`, `equivalence` must be null.
- `composition.entity_count` must equal the number of `entity_ids`, and `entity_ids` must have no duplicates.
- Each record in `targets` requires `observability`: `OBSERVED`, `UNOBSERVABLE`, `MISSING`, `UNSUPPORTED`, `AMBIGUOUS`, `ASSERTED_ONLY`.
- `OBSERVED` records: `ENTITY_FIELD` items have `entity_values` (an object whose key set equals `entity_ids`); `SURFACE` and `SUBSTRUCTURE` items have `value`.
- Omitting a catalog item, or including an item not in the catalog, is rejected.

The following are valid as input but the verdict becomes `UNVERIFIABLE` (`REQUIRED_PROVENANCE_MISSING`): any field of `provider` is null, `assembly_entrypoint` is null, or `input_digests` is an empty object.

Official example (`EX\bvl_input_v2\gndcrw\base.json` , whitespace reduced only):

```
{"surface_version": "gndcrw-surface/v0.1", "surface_id": "base", "role": "BASE",
 "provider": {"provider_id": "fixture-provider", "provider_version": "1.0.0", "adapter_id": "fixture-
adapter", "adapter_version": "1.0.0"},
 "assembly": {"mode": "DIRECT", "assembly_entrypoint": "fixture.assemble", "equivalence": null},
 "targets": {
   "manager.raw_state.recency": {"level": "ENTITY_FIELD", "observability": "OBSERVED",
"entity_values": {"slot-01": 1, "slot-02": 2, "slot-03": 3}},
   "manager.role_instruction": {"level": "SUBSTRUCTURE", "observability": "OBSERVED", "value":
{"items": ["observe", "order"]}},
   "manager.payload.length": {"level": "SURFACE", "observability": "OBSERVED", "value": 400}},
 "composition": {"entity_kind": "slot", "entity_count": 3, "entity_ids": ["slot-01", "slot-02",
"slot-03"], "identity_namespace": "fixture.slot/v1"},
 "provenance": {"input_digests": {
   "config": "sha256:c56c132ac01336a9750e33b9242257ac78252348344eecfaaf609ed1497d01b6",
   "seed": "sha256:3cd0084c6c90c9a4d69ffd29f184cd42fd3cc1d370ac15c73e52d5c5a7322b7c",
   "boomvalley-lab-contract-identity":
"sha256:3e9b8277d1fb5e9de9c14060751aa2d6c4e7c49aadd9c4db417f0f551029febf"}},
 "premises": []}
```

`candidate.json` in the same folder differs only in `surface_id` (`candidate`), `role` (`CANDIDATE`), the recency values (11, 12, 13), and the `config` digest.

2.5.4 `pair.json` — `gndcrw-pair/v0.1`

All five keys required (`GND\contracts\pair.py`).

Key	Content
`pair_version`	Exactly `"gndcrw-pair/v0.1"`
`comparison_mode`	`CONTROL` , `CONFIG_PERTURBATION` , `REFERENCE`
`base_surface_id` , `candidate_surface_id`	The `surface_id` of the two surfaces
`entity_comparability`	`status` , `method` , `authority` , `mapping` , `limitations` all required

- `status` : `ESTABLISHED` , `NOT_ESTABLISHED` , `UNKNOWN` , `NOT_APPLICABLE`
- `method` : `STABLE_DOMAIN_ID` , `EXPLICIT_MAPPING` , `DECLARED_PREMISE` , `NONE` . If `ESTABLISHED` , one of the first three; for any other status, `NONE` .
- `authority` : null if `method` is `NONE` ; otherwise `{"kind": "RESEARCHER"|"PROVIDER"|"CONTRACT", "id": "..."}` .
- `mapping` : an object (base id → candidate id) for `EXPLICIT_MAPPING` ; null for `STABLE_DOMAIN_ID` and `NONE` .

If `comparison_mode` is `REFERENCE` , the result is always `BLOCK` . This is because the declaration's `required_mode` allows only `CONTROL` and `CONFIG_PERTURBATION` .

Official example (`EX\bvl_input_v2\gndcrw\pair.json` , complete):

```
{"pair_version": "gndcrw-pair/v0.1", "comparison_mode": "CONTROL",
 "base_surface_id": "base", "candidate_surface_id": "candidate",
 "entity_comparability": {"status": "ESTABLISHED", "method": "STABLE_DOMAIN_ID",
   "authority": {"kind": "PROVIDER", "id": "fixture-provider"}, "mapping": null, "limitations": []}}
```

2.5.5 `declaration.json` — `gndcrw-declaration/v0.1`

All ten keys required. Lists may be empty, but the keys must be present (`GND\contracts\declaration.py`).

Key	Content
`declaration_version`	Exactly `"gndcrw-declaration/v0.1"`
`declaration_id`	Non-empty string
`intent`	`{"intent_id": "...", "intent_digest": "sha256:..."}`
`catalog_ref`	The five identifying values of the catalog
`comparison`	`{"required_mode": "CONTROL" or "CONFIG_PERTURBATION"}`
`expected_changes`	List of rules for what must differ
`invariants`	List of rules for what must be equal
`declared_consequences`	What you state in advance may differ as a consequence
`premises`	List of premises
`context`	List of context items

Rule object (`expected_changes`, `invariants`; all five keys required):

- `rule_id`: alphanumerics and `.`, `_`, `-`. Unique across all three rule lists.
- `target`: `{"kind": "TARGET" or "COMPOSITION", "id": "<item name>"}`
- `operation`: `MUST_DIFFER` in `expected_changes`, `MUST_EQUAL` in `invariants`
- `scope`: `{"kind": "WHOLE_TARGET"}`, `{"kind": "ALL_COMPARABLE"}`, or `{"kind": "EXPLICIT_ENTITY_IDS", "entity_ids": [...]}`. The latter two for `ENTITY_FIELD` items; `WHOLE_TARGET` for other items.
- `premise_refs`: list of `premise_id` values in this declaration's `premises`

The following come out as a **verdict (`BLOCK`)**, not as input errors: a nonexistent item name, an `operation` that does not fit the section, a `scope` that does not fit the item level, conflicting rules on the same target, and **an empty `expected_changes`** (`NO_FOCAL_EXPECTED_CHANGE`).

Official example (`EX\bvl_input_v2\gndcrw\declaration.json`, whitespace reduced only):

```
{"declaration_version": "gndcrw-declaration/v0.1", "declaration_id": "EXP-FIXTURE-001",
 "intent": {"intent_id": "fixture-intent", "intent_digest":
"sha256:083fb0188cac4cd74db152f93d5aabce13242fd9ea5d69929e20b43ce8ec27d6"},
 "catalog_ref": {"provider_id": "fixture-provider", "catalog_id": "fixture-catalog",
   "catalog_version": "gndcrw-target-catalog/v0.1", "profile_id": "default",
   "catalog_digest": "sha256:1924b54a7f0d66dcbab94758d1313a70aa51d6bcbc9df76acb328e6db0f64463"},
 "comparison": {"required_mode": "CONTROL"},
 "expected_changes": [{"rule_id": "CHG-001", "target": {"kind": "TARGET", "id":
"manager.raw_state.recency"},
   "operation": "MUST_DIFFER", "scope": {"kind": "ALL_COMPARABLE"}, "premise_refs": []}],
 "invariants": [{"rule_id": "INV-001", "target": {"kind": "TARGET", "id":
"manager.role_instruction"},
   "operation": "MUST_EQUAL", "scope": {"kind": "WHOLE_TARGET"}, "premise_refs": []}],
 "declared_consequences": [], "premises": [], "context": []}
```

2.5.6 `manifest.json` — `gndcrw-manifest/v0.2`

Key	Category	Content
`manifest_version`	Required	Exactly `"gndcrw-manifest/v0.2"`
`subjects`	Required	Exactly five roles: `declaration`, `target_catalog`, `base_surface`, `candidate_surface`, `pair_context`
`subjects.<role>`	Required	All three keys `{"path", "sha256", "size"}`
`external_subject`	Optional	`{"scheme", "digest"}`

File rules (`GND\manifest.py`):

- The file name must end with `.json` (lowercase). Size is 65,536 bytes or less.
- `path`: a relative path separated only by `/`. Backslashes, drive letters, absolute paths, and `..` are not allowed. It is **relative to the folder containing the manifest file**, and files can be only in that folder or its subfolders. It must end with `.json`.
- `sha256`: `sha256:` followed by 64 lowercase hexadecimal characters.
- `size`: integer from 1 to 16,777,216 inclusive.
- The five files must be distinct files, and must be regular files, not links or hard links. File name case must match the disk exactly.

Official example (`EX\b2b_frontline_internal_exchange_qwen_100_158\stage_a\gndcrw\manifest.json`, complete):

```
{"manifest_version": "gndcrw-manifest/v0.2",
 "subjects": {
   "base_surface": {"path": "base.json", "sha256":
"sha256:7fb2ea1f2229d0d646ddf762e2dbc562b32eeba8eb89eb3524659c64f8a26f59", "size": 2893},
   "candidate_surface": {"path": "candidate.json", "sha256":
"sha256:c40e3008e1a241d58a2e21af0cef06e7174bb2ea6e140f2a2ca882d4202ba4b3", "size": 2846},
   "declaration": {"path": "declaration.json", "sha256":
"sha256:0a0fc69fb0a1ff6a09fd53a382489257ca73fe17cd8ec4c6ff687ca63d125ae8", "size": 1829},
   "pair_context": {"path": "pair.json", "sha256":
"sha256:77ef007ae973e11d1a6021f282374d063773bb2630bd1993f56681c77e9549d6", "size": 378},
   "target_catalog": {"path": "catalog.json", "sha256":
"sha256:e701adb04f5b6c56047a6b2abfd899de63478a627f014ae3e45f6accc7c28e46", "size": 1282}}}
```

The hash values in this example are for the files in that folder. They match the installed files byte for byte, and `gndcrw check` on that folder gives `RUN` **[Run-confirmed]**. The folder `EX\bvl_input_v2\gndcrw\` used for the other examples in this section has no `manifest.json`; to run `gndcrw check` on an official example, use `EX\b2b_frontline_internal_exchange_qwen_100_158\stage_a\gndcrw\`. Neither folder is included in the anonymous review edition (section 0.1); there, run `gndcrw check` on `examples\manual_negotiation_demo\gndcrw\` (section 2.5.9).

`external_subject` is an optional item that indicates which external experiment declaration this preflight is for. GNDCRW only checks that `provenance.input_digests[<scheme>]` of both surfaces equals that value; it does not interpret what the value means (header of `GND\manifest.py`).

2.5.7 Computing SHA-256 and size

GNDCRW has **two kinds of digest**. Do not confuse them.

Kind	Where used	What is computed	Effect of file formatting
Raw byte digest	`sha256`, `size` in `manifest.json`	The file bytes on disk as is	Whitespace, key order, line endings (CRLF/LF), and trailing blank lines all affect it
Canonical JSON digest	`catalog_digest`, `intent_digest`	JSON with keys sorted and whitespace removed	No effect

Both have the format `sha256:` + 64 lowercase hexadecimal characters.

Raw byte digest (Windows CMD)

```
certutil -hashfile base.json SHA256
for %I in (base.json) do @echo %~zI
```

Prefix the single line of hexadecimal printed by the first command with `sha256:` and write it to `sha256`; write the number printed by the second command to `size`. On Windows versions where `certutil` prints uppercase letters or spaces, convert to lowercase and remove the spaces. On the check PC (Windows 11) `certutil` printed a header line, the hash as one line of lowercase hexadecimal without spaces, and a closing line, both in the display language of Windows; take only the middle line **[Run-confirmed]**.

Canonical JSON digest

It is hard to compute by hand. The procedure below was followed on a copy of the official example with both digests set to the placeholder, and it ended in `RUN` **[Run-confirmed]**. The way to get it without code is to copy down the value GNDCRW tells you. If a digest is wrong, the error message shows the declared value and the computed value together (`GND\contracts\catalog.py`, `GND\contracts\declaration.py`).

```
catalog_digest does not recompute (declared X, computed Y)
intent_digest does not recompute (declared X, computed Y)
```

Procedure:

1. Put a placeholder value (`sha256:` followed by 64 `0`s) in both digest positions.
2. Compute and enter the raw byte values in the manifest, and run `gndcrw check manifest.json`.
3. The catalog is validated first. Put the `computed` value from the message into `catalog_digest` in `catalog.json` and into `catalog_ref.catalog_digest` in `declaration.json`.
4. The files have changed, so recompute `sha256` and `size` in the manifest.
5. Run again, enter the `computed` value for `intent_digest`, and fix the manifest once more.

Ordering principle: compute the manifest hashes after all content is finalized. Every time you edit a file, its raw byte digest changes.

What goes into `intent_digest` is `catalog_ref`, `comparison`, the three rule lists, `premises`, and the `context` items whose `normative` is true. `declaration_id` and `intent_id` do not go in.

2.5.8 Common errors

Error code	Cause
`REF_DIGEST_MISMATCH`, `REF_SIZE_MISMATCH`	You edited a file after creating the manifest, or the editor changed the line endings.

Error code	Cause
`SUBJECT_CONTRACT_INVALID`	The content of one of the five files does not match its contract. The message gives `<role>:<location>: <reason>`.
`MANIFEST_PATH_INVALID`	You used a backslash, an absolute path, or `..`.
`MANIFEST_UNKNOWN_FIELD`, `MANIFEST_FIELD_INVALID`	The manifest has an extra key or a wrong format.
`EXTERNAL_SUBJECT_NOT_IN_SURFACE`	`external_subject.digest` is not in the `input_digests` of the two surfaces.

Validation command: `gndcrw check manifest.json` (Part 5).

2.5.9 Building the files from scratch — worked example

This section walks through writing the six files, using a real worked example: `examples\manual_negotiation_demo\gndcrw\` in the repository. It checks the negotiation study of section 4.6 for one thing: **the buyer and the supplier organization differ only in their mandate and briefs, and all other conditions are the same.** The folder is in the repository only. It is not in the `EX` folder of the rc6 installation.

Check status: [Run-confirmed]. On 2026-10-17, `gndcrw check` on this folder on the rc6 installation gave `Outcome RUN`, `PASS=7 FAIL=0 WARNING=0`, exit code 0.

① **Decide what to compare.** First write down what BASE and CANDIDATE are, which items to compare, and whether each item must differ or must be equal. The table of the worked example:

Item (name in `targets`)	Level	Value	Rule
`organization.mandate`	`SUBSTRUCTURE`	objective, opening price, concession step, price limit	must differ (`CHG-MANDATE`)
`organization.brief_files`	`SUBSTRUCTURE`	path, SHA-256 and size of the source files each organization receives	must differ (`CHG-BRIEF-FILES`)
`negotiation.protocol`	`SURFACE`	maximum rounds, termination rule, World, deadline	must be equal (`INV-PROTOCOL`)
`organization.members`	`SUBSTRUCTURE`	roles, who drafts, who approves	must be equal (`INV-MEMBERS`)
`organization.approval`	`SUBSTRUCTURE`	decision rule, authority, unresolved policy, how to approve and reject	must be equal (`INV-APPROVAL`)
`organization.model_profile`	`SUBSTRUCTURE`	provider, model, renderer and options of negotiator and manager	must be equal (`INV-MODEL-PROFILE`)
`organization.information_access`	`SUBSTRUCTURE`	kinds of sources negotiator and manager receive, memory settings	must be equal (`INV-INFORMATION-ACCESS`)

- BASE is the buyer organization (`base.json`) and CANDIDATE is the supplier organization (`candidate.json`).
- With a rule on every item, no undeclared difference (`UNDECLARED_DIFFERENCE`, advisory) can arise.
- With no item that must differ, the outcome is `BLOCK` (`NO_FOCAL_EXPECTED_CHANGE`).
- For objects or lists, use `value_kind` `JSON`. Do not use non-integer numbers such as `0.5`. The worked example writes `temperature` as the integer `0`.

② **Write `catalog.json`.** For each item of the table, add `{"level": ..., "value_kind": "JSON", "availability": "SUPPORTED", "equality": "JSON_STRUCTURAL"}`. Copy `composition_targets` from the example in section 2.5.2. Put the placeholder `sha256:` followed by 64 `0` into `catalog_digest` for now.

③ **Write `base.json` and `candidate.json`.** The key set of `targets` in both files must be exactly that of the catalog. Each item is `{"level": ..., "observability": "OBSERVED", "value": <value>}`. **Copy the values from `study.json` and the source files.** The worked example's value of `organization.model_profile` (buyer organization):

```
{"negotiator": {"provider": "bvl.openai-chat-adapter:1", "model": "qwen2.5:7b-instruct",
                "renderer": "bvl.json-renderer:1",
                "options": {"temperature": 0, "max_tokens": 96, "response_format": "json_object"}},
 "manager":    {"provider": "bvl.openai-chat-adapter:1", "model": "qwen2.5:7b-instruct",
                "renderer": "bvl.json-renderer:1",
                "options": {"temperature": 0, "max_tokens": 96, "response_format": "json_object"}}}
```

In items that must be equal, avoid values that contain the organization's name. If the actor names `buyer_manager` and `supplier_manager` were written as they are, the two values would differ and the result would be `INVARIANT_VIOLATED`. The worked example uses role names such as `manager`, `negotiator` and `own_manager_brief`.

The other keys:

Key	Value in the worked example	Note
`provider`	`boomvalley-lab`, `1.0.0rc3`, `manual-negotiation-demo`, `1.0.0`	Any null gives `UNVERIFIABLE` (`REQUIRED_PROVENANCE_MISSING`)
`assembly`	`DIRECT`, `manual-negotiation-demo.study-json`, `null`	A null `assembly_entrypoint` gives `UNVERIFIABLE`
`composition`	`role`, 2, `["manager", "negotiator"]`, `bvl.manual-negotiation-demo.role/v1`	The same in both files
`provenance.input_digests`	raw-byte SHA-256 of `bvl-study-json` (study.json), `negotiator-brief`, `manager-brief`	An empty object gives `UNVERIFIABLE`
`premises`	`[]`	

④ **Write `pair.json`.** Take the example of section 2.5.4 and change only the two `surface_id` values. The worked example uses `{"kind": "RESEARCHER", "id": "manual-negotiation-demo"}` as `authority`.

⑤ **Write `declaration.json`.** Following the table of ①, put `MUST_DIFFER` rules in `expected_changes` and `MUST_EQUAL` rules in `invariants`. For items at level `SURFACE` or `SUBSTRUCTURE`, `scope` is `{"kind": "WHOLE_TARGET"}`. Copy the five catalog values into `catalog_ref`, and put the placeholder into `intent_digest`.

⑥ **Compute the two canonical digests.** GNDCRW has no command that prints them. Call GNDCRW's own functions with the Python of the installed GNDCRW. Run in CMD, in the folder of the six files **[Run-confirmed]**:

```
set GPY="C:\Program Files\BOOMVALLEY Research Toolkit\components\gndcrw\venv\Scripts\python.exe"
%GPY% -c "import json,sys; from gndcrw.contracts.catalog import compute_digest;
print(compute_digest(json.load(open(sys.argv[1],encoding='utf-8'))))" catalog.json
```

Put the printed value in two places: `catalog_digest` of `catalog.json` and `catalog_ref.catalog_digest` of `declaration.json`. Then compute `intent_digest`. Keep this order, because `intent_digest` covers `catalog_ref`.

```
%GPY% -c "import json,sys; from gndcrw.contracts.declaration import compute_intent_digest;
print(compute_intent_digest(json.load(open(sys.argv[1],encoding='utf-8'))))" declaration.json
```

Put the printed value into `intent.intent_digest` of `declaration.json`. From Toolkit rc7, the one line `bvt gndcrw-digest manifest.json --write` does this step ⑥ and the next step ⑦ (the manifest's hashes and sizes) together; without the option it only checks that the values are right and changes nothing. Neither value depends on whitespace or key order, so reformatting the files does not change them. In the worked example they are `sha256:61619e4f…` (catalog) and `sha256:319acf48…` (intent). Reading them from GNDCRW's error messages as in section 2.5.7 gives the same values.

⑦ Compute the raw-byte hashes and write `manifest.json`. Do this only after the content of all five files is final. For each file:

```
certutil -hashfile base.json SHA256
for %I in (base.json) do @echo %~zI
```

To bind the check to `study.json`, add `external_subject`. Its `scheme` is the name used in `input_digests` in ③ (`bvl-study-json`), and its `digest` is the raw-byte SHA-256 of `study.json`. It must equal the value in both surfaces (otherwise `EXTERNAL_SUBJECT_NOT_IN_SURFACE`).

⑧ Check.

```
gndcrw check manifest.json
```

From another folder, give the path, such as `gndcrw check <folder>\manifest.json`. The five files are looked up relative to the folder of the manifest **[Run-confirmed]**.

Points to note after writing.

- **GNDCRW does not check that the copied values are right.** If the values in `base.json` and `candidate.json` differ from `study.json` but agree with each other as declared, the result is still `RUN`. `external_subject` is only the author's claim that "this check is about this `study.json`" (result limitation `EXTERNAL_SUBJECT_PROVIDER_ATTESTED`). Comparing it with the actual SHA-256 of `study.json` is the researcher's job; in PowerShell, `Get-FileHash study.json`.
- **The result is not bound to a BVL run** (section 1.6). In a study with one condition, `gndcrw_results` in the BVL package is an empty result set.
- **When `study.json` or a source file changes, redo ③ and ⑦.** An editor that changes line endings also changes the raw-byte hashes. In the repository, the worked example folder has the `-text` attribute so Git does not change line endings.
- The six files of the worked example were generated by a script that reads the values from `study.json`. Writing them by hand follows the same content and order.

2.6 FULLMT — input package

2.6.1 What you give it

What you give FULLMT is **one folder**. It is the run folder (`RUN`) created by `bvl run` and sealed by `bvl package` , as is.

FULLMT decides the package type by which marker file is at the top level of the folder (`FMT\app\epc_reader.py`).

What is at the top level of the folder	Result
Only `MANIFEST-4.json`	MANIFEST-4 verification (current format)
Only `EVIDENCE.json`	Old format (EPC) verification
Both	`ERROR AMBIGUOUS_PACKAGE_FORMAT`
Neither	`ERROR NOT_AN_EPC_PACKAGE`

A run folder on which `bvl package` was not run has no `MANIFEST-4.json` , so it cannot be given to FULLMT.

2.6.2 `MANIFEST-4.json`

Item	Content
Path	`RUN\MANIFEST-4.json`
Created by	`bvl package RUN`
Schema identifier	`boomvalley-lab-manifest-4`
Top-level keys	Exactly `schema` , `package_class` , `plan` , `members` , `limitations`

- `package_class` : `"CONFORMANCE"` . `"SCIENTIFIC"` is defined but cannot be created, and FULLMT rejects it.
- `plan` : the plan's identity `{"kind": "plan", "digest": "sha256:..."}` .
- `members` : `{"path", "role", "digest", "size"}` for every evidence file in the folder. Sorted by path. `digest` is the SHA-256 of the file's raw bytes, and `size` is the number of bytes.
- `limitations` : always the following four codes.

```
CONFORMANCE_PACKAGE_NOT_SCIENTIFIC_EVIDENCE
FULLMT_MANIFEST4_SPIKE_READER_REQUIRED
GATE2_INPUTS_SUPPLIED_NOT_DERIVED
PLUGIN_SET_NOT_ENGINE_RESOLVED
```

Basis: `PKG\kernel\manifest4.py` , `PKG\kernel\package4.py` , `FMT\app\manifest4_reader.py` .

The researcher does not write or edit `MANIFEST-4.json` . Once created, it cannot be created again ("manifest-4 is immutable and already exists").

2.6.3 What the researcher creates and what is created automatically

File	Created by
`study.json` , `sources\*`	Researcher
`RUN\journal-2.jsonl`	`bvl run`

File	Created by
`RUN\requests-v2\request-*.json`	`bvl run`
`RUN\artifacts\call-*.bin`	`bvl run`
`RUN\custody-v2\entries.jsonl`	`bvl run`
`RUN\evidence-v2\<role>\<hash>.json`	`bvl run`
`RUN\evidence-v2\conformance_attestation\`, `dataset_units\`, `analysis_output\`	`bvl package`
`RUN\MANIFEST-4.json`	`bvl package`
FULLMT result JSON	`fullmt --out`

After sealing, do not put any file into the `RUN` folder. If even one file not in the list exists, both `bvl verify` and `fullmt` fail (FULLMT rule `M4-UNL`). Keep notes, result JSON, and screenshots outside `RUN`. The target of `fullmt --out` must also be outside `RUN`.

2.6.4 FULLMT result JSON

The schema of the result document is `fullmt-manifest4-verification-result/1` (`FMT\app\manifest4_reader.py`). By default the whole result is printed as JSON to standard output; if `--out` is given, it is also written to a file.

Key	Meaning
`outcome`	`VERIFIED`, `REJECTED`, `ERROR`
`meaning`	`CONFORMANCE` or `MEMBER_INVENTORY` (absent for `ERROR`)
`failed_rules`	List of names of failed rules. Empty list if `VERIFIED`
`rules`	`{rule_id, layer, title, status, problems, note}` for each of the 20 rules. `status` is `PASS`, `FAIL`, `NOT_EVALUATED`
`summary`	`{"PASS": n, "FAIL": n, "NOT_EVALUATED": n}`
`package_identity`, `manifest_identity`	Package identity. Same as the value printed by `bvl package`
`package_class`, `plan`	The manifest's values as is
`observed`	Observed facts: `members` (file count), `evidence_level`, `dataset_units`, `analysis`, `attested`, etc.
`limitations.producer_declared`	The manifest's `limitations` as is
`limitations.consumer`	List of `{code, detail}` — **limitations FULLMT states about itself**
`error`, `error_code`	Only for `ERROR`
`verifier`	FULLMT's name and version. In the installation, `code_digest` is null

Interpretation is in Part 6, and failure handling is in Part 7.

Part 3. Order of writing files

This assumes you start from an empty folder. The requested 12 steps have been adjusted to match the actual implementation. The adjustments are as follows.

- "Composing GNDCRW input" is not a separate file-writing step. In `bvl-study-1`, writing `design` in `study.json` serves that role (Step 7).
- Studies with two or more conditions require Toolkit rc3 or later; Toolkit `1.0.0` qualifies (section 3.2).

3.1 Steps

Step 1. Define the research question

First check with the "Not supported" table in section 1.7 whether it is in a form BVL can answer. In particular, if you need free conversation, statistical comparison across conditions, or writing prompts directly, the current implementation does not fit.

Step 2. Define experimental conditions and comparison targets

Decide what separates the conditions. Only two things can be declared as factors: **an actor's profile** and **information access rules** (section 2.2.11). To change anything else, you must create a separate `study.json` for each condition (section 3.2).

Step 3. Design the World and organizational structure

Choose a World that fits the study from the World list in section 4.1. Creating a new World is code development, so it is outside the scope of this manual. Check the state fields and action fields of the chosen World, and open the official example that uses that World.

Then draw the flow of one round on paper: who is called, whose proposal goes to which decision, and which decision is applied to the World.

Step 4. Set up actors and information access rights

Write `roles` and `actors`. Make a list of what each actor must see and transfer it to `information.sources` and `information.access`. The default is deny, so anything you omit is not visible. Write the instruction source files (`sources\*.json`) in this step.

Step 5. Set up AI provider / model

Write `profiles` and `plugin_config`. It is recommended to first check that the procedure runs with `bvl.scripted-adapter:1`, then switch to a real model (`bvl.openai-chat-adapter:1`). With scripted responses you can repeat Steps 6–8 without model cost.

What to check when using a real model: Is the server up? Is the `model` name exactly the same as the name the server reports? Does `endpoint` include `/v1` ?

Step 6. Define proposal / decision / protocol

Write `proposals.schemas` to match the World's action schema (field names and types must match for decisions to be applied to the World). Write `proposals.contracts`, `decisions`, `protocol.nodes`. Decide repetition and termination (`rounds` or `round`). If memory is needed, add a `memory_update` node and a `memory_artifact` source. If aggregation is needed, write `analysis`.

Step 7. Declare experimental conditions (`design`)

Skip this if there is only one condition. If there are two or more conditions, write `design.factors` and `design.contrasts` . BVL creates the GNDCRW documents.

Step 8. Pre-validation

```
bvl validate --experiment study.json
bvl check --experiment study.json
```

`validate` checks the format, and `check` checks the gate. `check` must give `gate-2 CLEARED` before you can run. Neither writes anything, so repeat until they pass.

Step 9. Run BVL

```
bvl run --experiment study.json --into RUN
```

`RUN` must be a nonexistent folder or an empty folder. Check that the run ended with `COMPLETED` . If `FAILED` , discard that folder (it cannot be resumed), fix the cause, and run again in a new folder.

Step 10. Package evidence

```
bvl package RUN
```

`MANIFEST-4.json` is created. After this, do not put anything into `RUN` .

Step 11. Verify

```
bvl verify RUN
fullmt RUN --out fullmt-result.json
```

`bvl verify` is the producer's own recomputation; `fullmt` is an independent recomputation. The target of `--out` must be outside `RUN` .

Step 12. Analyze and interpret results

```
bvl analyse RUN
```

The pre-bound metrics are output. For the limits of interpretation, follow Part 6. The tools do not compare across conditions, so the researcher must compare the results of each condition separately, and that comparison is outside the verification scope of these tools.

3.2 Studies with two or more conditions

In Toolkit rc3 or later, BVL can load the `gndcrw` package, so a factor design receives a GNDCRW verdict at the gate (section 1.6). On the rc6 installation, all official factor designs passed in every cell **[Run-confirmed]**. Still check your own installation first:

```
bvt --version
bvl check --experiment study.json --cell cell.<factor>-<level>
```

If `bvt --version` shows `yes` for `gndcrw package inside boomvalley-lab` and `bvl check` gives `gate-2 CLEARED` , you can run. If the reasons include `BVL-GATE-004` , the installation is rc2 or earlier, or

the connection is broken. Run Setup again to repair it.

If you still cannot use a factor design, the options available without code changes are as follows.

Alternative	Method	What you lose
Single study per condition	Create a separate `study.json` without `design` for each condition and run each.	**The tools do not confirm** that the two conditions differ only in what was declared. The researcher must show the differences between the two files directly and take responsibility for them. Each package is verified as a separate, unrelated run.
Add a standalone GNDCRW check	In addition to the above, write the six files in section 2.5 yourself and run `gndcrw check`.	That result is not bound to the BVL run. The researcher writes the surface values directly, so it is a check of "values claimed by the provider".

Part 4. Templates you can copy and use

4.1 List of registered components

The list below is everything that `study.json` can point to by name. Names must be written **exactly as shown**.

Provider (Adapter) — `profiles[].provider`

Name	Purpose	Configuration
`bvl.scripted-adapter:1`	Returns predefined responses. No external calls.	`responses` in `plugin_config`
`bvl.openai-chat-adapter:1`	One `POST <endpoint>/chat/completions` to an OpenAI-compatible server. No retries.	`endpoint` etc. in `plugin_config`

Human and tool actors use `bvl.operator-entry-channel:1` and the tool name, respectively, as `provider` (section 2.2.13).

Renderer — `profiles[].renderer`

Name	Behavior
`bvl.json-renderer:1`	Builds one JSON object from the actor name, round, World state, and allowed sources, and sends it as the user message.

Parser — `proposals.contracts[].parser`

Name	Behavior
`bvl.json-proposal-parser:1`	Reads the response as one UTF-8 JSON object. No configuration.

Decision rules — `decisions[].rule`

`builtin.single-authority:1`, `builtin.majority:1`, `builtin.plurality:1`, `builtin.veto:1` (section 2.2.7)

Termination rules — `protocol.round.termination.rule`

`builtin.fixed-count:1`, `builtin.state-predicate:1`, `builtin.terminal-state:1` (section 2.2.3)

Memory transform — `transform` of `memory_update`

`builtin.outcome-memory-v2:1`

Information transform — `information.access[].transform`

`builtin.redact-json-keys:1`

World — `world.plugin`, `config.world` of `world_transition`

Name	Kind	State schema and fields	Action schema and fields	Meaning of no action	Rule summary	Example (relative to `EX`)
`bvl.ledger-world:1`	Reference	`bvl.ledger-state:1` (open, free fields)	`bvl.ledger-action:1` (open)	Not declared	Merges the action into the state as is. Always admissible.	`k0_study\study.json`
`bvl.tally-world:1`	Reference	`bvl.tally-state:1`: `applied` integer, `total` integer	`bvl.tally-action:1`: `amount` integer	`NO_TRANSITION`	Adds `amount` to `total` and increments `applied` by 1.	`k0_study\study_world.json`
`bvl.shock-world:1`	Reference	`bvl.shock-state:1`: `applied`, `total` integers	`bvl.shock-action:1`: `amount` integer	Not declared	Tally with a random shock of -3 to 3. Requires `rng_policy: SEEDED`.	`k0_study\study_stochastic.json`
`bvl.route-world:1`	Domain	`bvl.route-state:1`: `closed` array, `costs` object, `total_cost` integer, `trips` integer	`bvl.route-action:1`: `route` string	`NO_TRANSITION`	An open route is admissible; a closed route is adjusted to the cheapest open route; an unknown route is inadmissible.	`conformance\b7_route_choice_world.json`
`bvl.restock-world:1`	Domain	`bvl.restock-state:1`: `capacity`, `demand`, `period`, `sold`, `stock`, `unmet` integers	`bvl.restock-action:1`: `order_qty` integer	`EXOGENOUS_PROGRESSION`	An order within spare capacity is admissible, an excess is adjusted, a negative is inadmissible. Demand is processed afterward.	`conformance\b7_second_domain_restock.json`
`bvl.vending-world:1`	Domain	`bvl.vending-state:1`: `slots` object	`bvl.vending-action:1`: `quantity` integer, `slot_id` string	`NO_TRANSITION`	A refill within slot spare capacity is admissible, an excess is adjusted.	`conformance\vending_legacy_oracle.json`
`bvl.vending-world:2`	Domain	`bvl.vending-state:2`: `cash`, `day`, `holding_cost`, `lead_time`, `order_fee`, `sold`, `unmet` integers, `pipeline` array, `slots` object	`bvl.vending-action:1`	`EXOGENOUS_PROGRESSION`	Daily steps: ordering, delivery, demand, holding cost.	`longitudinal\b8_longitudinal_vending.json`
`bvl.seasonal-restock-world:1`	Domain	`bvl.seasonal-restock-state:1`: restock fields + `shift_at`, `shift_demand` integers	`bvl.seasonal-restock-action:1`: `order_qty` integer	`EXOGENOUS_PROGRESSION`	Restock with a demand regime shift.	`longitudinal\study_cross_episode_regime.json`
`bvl.seasonal-vending-world:1`	Domain	`bvl.seasonal-vending-state:1`: vending:2 fields + `calendar` array, `demand` object, `season` string	`bvl.vending-action:1`	`EXOGENOUS_PROGRESSION`	vending:2 with seasons.	`longitudinal\b9_memory_regime_vending.json`

Name	Kind	State schema and fields	Action schema and fields	Meaning of no action	Rule summary	Example (relative to `EX`)
`bvl.negotiation-world:1`	Domain	`bvl.negotiation-state:1`: `deadline`, `moves`, `price` integers, `offers` object, `status` string, `turn` string	`bvl.negotiation-action:1`: `move` string, `price` integer, `side` string	`EXOGENOUS_PROGRESSION`	One move by one side at a time. Ruling table in `EX\external\README.md`.	`external\b10_external_negotiation.json`

- "Domain" Worlds require a `world` section declaration (`BVL-WORLD-004`).
- For the internal structure of state objects (`slots`, `pipeline`, `calendar`, etc.), start from the `initial_world_state` of the example file in the table as is.
- Basis: `CATALOG` in `PKG\lifecycle\plugins.py`, `PKG\domains\__init__.py`, `PKG\domains\*.py`.

4.2 How to read the templates

Each part of a template is marked as follows.

Mark	Meaning
Edit	Must be changed for every study.
Fixed	A schema identifier or a registered component name. Changing it causes rejection.
Optional	Add or remove as needed.
Reference	Must exactly match a name elsewhere.

BVL fills in no values automatically in `study.json`. The plan hash, seed-derived values, cell names, and run names are all computed and output by BVL, and are not written in the file.

Reference relationships inside the file:

```
actors[].role              → roles[]
actors[].profile           → profiles[].profile_id
profiles[].provider        → registered provider, key of plugin_config
profiles[].model           → (scripted) key of plugin_config.responses / (real) name the server
reports
nodes[].config.actor       → actors[].actor_id
nodes[].depends_on[]       → nodes[].id
nodes[].config.world       → world.plugin
contracts[].node_id        → nodes[].id whose kind is proposal_parse
contracts[].schema         → "<schemas[].schema_id>:<schemas[].version>"
decisions[].node_id        → nodes[].id whose kind is decision
decisions[].authority[]    → actors[].actor_id
sources[].producer         → nodes[].id
sources[].path             → file relative to the study.json folder
access[].actor / .source   → actors[].actor_id / sources[].source_id
factors/levels in access[].when → design.factors
proposals.schemas[].fields → names and types match the world's action schema fields
initial_world_state        → the world's state schema fields
analysis.metrics[].field   → names in proposals.schemas[].fields
```

4.3 Template A — minimal study (1 actor, scripted responses)

Verification status: [Run-confirmed]. Newly composed for this manual by combining parts of the official example `EX\conformance\b3_model_as_factor.json` (1-actor structure) and `EX\k0_study\study_world.json` (tally World declaration). Copied unchanged into `study.json`, it went through the whole flow of section 5.12 on the rc6 installation (results after the table). After you edit it, run `bvl validate` and `bvl check` again.

```
{
  "schema": "bvl-study-1",
  "experiment_id": "my-first-study",
  "episode_key": "episode-1",
  "root_seed": 1,
  "protocol": {
    "rounds": 2,
    "nodes": [
      {"id": "lead_call", "kind": "actor_call", "config": {"actor": "lead"}},
      {"id": "lead_parse", "kind": "proposal_parse", "depends_on": ["lead_call"]},
      {"id": "decision", "kind": "decision", "depends_on": ["lead_parse"]},
      {"id": "world", "kind": "world_transition", "depends_on": ["decision"],
       "config": {"world": "bvl.tally-world:1"}}
    ]
  },
  "initial_world_state": {"applied": 0, "total": 0},
  "world": {"plugin": "bvl.tally-world:1", "state_schema": "bvl.tally-state:1",
            "action_schema": "bvl.tally-action:1"},
  "roles": ["lead"],
  "profiles": [
    {"profile_id": "lead-profile", "provider": "bvl.scripted-adapter:1",
     "model": "scripted:lead:1", "renderer": "bvl.json-renderer:1"}
  ],
  "actors": [
    {"actor_id": "lead", "role": "lead", "profile": "lead-profile"}
  ],
  "proposals": {
    "schemas": [
      {"schema": "bvl-proposal-schema-1", "schema_id": "order", "version": 1,
       "fields": {"amount": "integer"}, "allow_no_action": false}
    ],
    "contracts": [
      {"node_id": "lead_parse", "schema": "order:1", "parser": "bvl.json-proposal-parser:1"}
    ]
  },
  "decisions": [
    {"node_id": "decision", "rule": "builtin.single-authority:1", "authority": ["lead"]}
  ],
  "analysis": {
    "analysis_id": "order-summary",
    "metrics": [
      {"metric_id": "decisions", "kind": "unit_count", "table": "decisions"},
      {"metric_id": "mean_amount", "kind": "action_mean", "field": "amount"}
    ]
  },
  "plugin_config": {
    "bvl.scripted-adapter:1": {
      "responses": {"scripted:lead:1": {"*": {"amount": 3}}}
    }
  }
}
```

Part	Mark
`experiment_id`, `root_seed`, `protocol.rounds`	Edit
Node `id`, `actor_id`, `roles`, `profile_id`, `schema_id`, `analysis_id`, `metric_id`	Edit (keep reference relationships)
`"schema": "bvl-study-1"`, `"schema": "bvl-proposal-schema-1"`	Fixed
`kind` values, provider/renderer/parser/rule/World names, schema names	Fixed
Field names in `initial_world_state`, `amount` in `fields`	Fixed by the World (those of the tally World)
`analysis`	Optional
Response values in `plugin_config`	Edit

Results of the unchanged template on the rc6 installation **[Run-confirmed]**:

Command	Result
`bvl validate`	`valid: my-first-study (1 design cell(s))`, plan `sha256:69e95409e052114aa73ccd989b9e772c742f7289cf1512ace82169fa9e8d9e55`
`bvl check`	`gate-2 CLEARED`
`bvl run`	`COMPLETED`, `calls: 2 custody-accepted, 0 pending`
`bvl package`	`sealed CONFORMANCE package: 25 members`
`bvl verify`	`VERIFIED`; L1 met; L2 not met (`NOT_L2`, no `information` section); L5 not met (no human or tool actor)
`bvl verify --l1 --l3 --replay-world --experiment study.json`	`VERIFIED`, exit code 0 (L3: 2 decision-to-World transitions)
`bvl verify --l1 --l2 ...` or `--l4`	first line `VERIFIED`, but **exit code 4**: the template has no actor-to-actor hand-over (L2) and uses `rounds`, not a `round` contract (L4)
`fullmt RUN --quiet`	`VERIFIED CONFORMANCE ... failed=[]`
`bvl analyse`	`decisions 2/1`, `mean_amount 3/1`

The plan hash does not depend on the layout of `study.json`: the same template saved with another indentation and CRLF line endings gave the same plan hash. Source files are different; their bytes are part of the plan (section 2.3).

4.4 Template B — using a real model

Use the official example `EX\provider\local_model_study.json` as is as the starting point. It is a 3-round study in which a scripted worker reports and a real-model manager decides. It differs from Template A in the following three places.

1. Profile — an actor that uses a real model

```
{"profile_id": "manager-profile", "provider": "bvl.openai-chat-adapter:1",
 "model": "qwen2.5:7b-instruct", "renderer": "bvl.json-renderer:1",
 "options": {"temperature": 0, "max_tokens": 64, "response_format": "json_object"}}
```

2. Provider configuration

```
"plugin_config": {
  "bvl.openai-chat-adapter:1": {"endpoint": "http://127.0.0.1:11434/v1", "timeout_seconds": 300}
}
```

3. Instruction source and access permission

```
"information": {
  "sources": [
    {"source_id": "order_brief", "kind": "frozen_resource", "path": "sources/order_brief.json"}
  ],
  "access": [
    {"actor": "manager", "source": "order_brief", "effect": "ALLOW"}
  ]
}
```

Part	Mark
`model`	Edit. Exactly the same as the name the server reports. For Ollama, the name in `ollama list`.
`endpoint`	Edit. For Ollama, `http://127.0.0.1:11434/v1`.
`options`	Optional. Allowed values are in section 2.2.4.
`timeout_seconds`, `api_key_env`	Optional
Instructions in the source file	Edit. Specify the response format exactly.

Verification status: [Run-confirmed]. The official example `EX\provider\local_model_study.json` was run unchanged on the rc6 installation with local Ollama `0.35.1` and `qwen2.5:7b-instruct`: plan `sha256:22bfc463202b20acf234b7c691d5c9dfff13d880b61ca661a289603975fc10c0`, `COMPLETED` with 6 calls, `sealed CONFORMANCE package: 44 members`, `bvl verify` `VERIFIED` with L1 and L2 (3 actor-to-actor hand-overs), FULLMT `VERIFIED CONFORMANCE`, and `bvl analyse` `decisions` `3/1`, `mean_amount` `32/3`. The analysis values depend on what the model answered and can differ in your run. The official document `docs/T4_SECOND_PROVIDER_T4_010.md` (tag `v1.0.0`) also records a run of the same combination as PASS.

Behavior to know when using a real model (`PKG\providers\openai_chat.py`):

- If the response's `finish_reason` is not `"stop"`, the call fails. Give enough `max_tokens` so that the answer is not cut off. With `max_tokens: 4` the run failed with `PROVIDER_INVALID_RESPONSE: ... the completion did not finish (finish_reason 'length')` **[Run-confirmed]**.
- It calls only once. A timeout or connection failure is a failure of that run.
- `temperature: 0` and a fixed seed do not guarantee the same response.

4.5 Template C — a design with two conditions

Two official examples each show one of the two ways to use a factor.

C-1. A factor that changes an actor's model (`EX\conformance\b3_model_as_factor.json`)

```
"actors": [
  {"actor_id": "worker", "role": "worker",
   "profile": {"factor": "worker_model", "cases": {"a": "model-a", "b": "model-b"}}}
],
```

```
"design": {
  "factors": {"worker_model": ["a", "b"]},
  "contrasts": {"worker_model": ["actor.worker.profile.model"]}
}
```

C-2. A factor that changes information access (`EX\conformance\b2_summary_raw_access.json`)

```
"information": {
  "sources": [
    {"source_id": "field_report", "kind": "frozen_resource", "path": "sources/field_report.json"}
  ],
  "access": [
    {"actor": "lead", "source": "field_report", "effect": "ALLOW",
     "when": {"report_access": "summary"},
     "transform": {"transform": "builtin.redact-json-keys:1",
                   "config": {"keys": ["complaint_count", "defect_log"]}}},
    {"actor": "lead", "source": "field_report", "effect": "ALLOW",
     "when": {"report_access": "raw"}}
  ]
},
"design": {
  "factors": {"report_access": ["summary", "raw"]},
  "contrasts": {"report_access": ["information.schema", "information.transform.lead.*"]}
}
```

Part	Mark
Factor names, level names	Edit (alphanumerics, `_`, `-` only)
Keys of `cases`	Reference. Same as the levels in `factors`.
Factors/levels in `when`	Reference. Same as `factors`.
Item names in `contrasts`	Edit. Every plan item that the factor changes, without omission. Table in section 2.2.11.

Run each cell separately.

```
bvl check --experiment study.json --cell cell.worker_model-a
bvl run   --experiment study.json --cell cell.worker_model-a --into RUN_a
bvl run   --experiment study.json --cell cell.worker_model-b --into RUN_b
```

Verification status: [Run-confirmed]. Rejected in Toolkit rc2. On the rc6 installation both examples passed in both cells:

Example	Cell	Plan	`run`	Package	FULLMT
C-1	`cell.worker_model-a`	`sha256:d2b65260…`	`COMPLETED`	23 members	`VERIFIED`
C-1	`cell.worker_model-b`	`sha256:6e1abaab…`	`COMPLETED`	24 members	`VERIFIED`
C-2	`cell.report_access-summary`	`sha256:c62669a0…`	`COMPLETED`	23 members	`VERIFIED`
C-2	`cell.report_access-raw`	`sha256:d17a18f3…`	`COMPLETED`	23 members	`VERIFIED`

The GNDCRW verdict for the comparison edge (`edge.worker_model.a-b` and `edge.report_access.summary-raw`) is `RUN` and is kept in `evidence-v2\gndcrw_results\` of each run.

What a missing contrast does [Run-confirmed]. When the `contrasts` of C-1 named only `actor.worker.profile.provider` instead of the model, `bvl check` refused with `BVL-GATE-003:edge.worker_model.a-b:BLOCK` (exit code 8): the model differed between the cells but was

not declared. A name that matches no plan item (`actor.worker.profile.nonsense`) added `BVL-COV-007:design:actor.worker.profile.nonsense`. `bvl check` without `--cell` on a 2-cell design is `BVL-LIFE-001 ... the design has 2 cells; name one with --cell (...)` (exit code 2).

4.6 Template D — the official 4-agent B2B negotiation demo (negotiation between two organizations with manager approval, real model)

This template is BOOMVALLEY's **official 4-agent B2B negotiation demo**. The buyer organization and the supplier organization each have one negotiator and one manager; all four actors respond with `qwen2.5:7b-instruct` on local Ollama.

Verification status.

- **[Run-confirmed]** On 2026-10-13, it passed `bvl validate`, `bvl check`, and `bvl run` on a Toolkit `1.0.0-rc2` installation (BVL `v1.0.0`). Plan `sha256:f26d8b24bd5ad2f74c742684dcc689a46b7386f0a74961afb6c1581b6bcaa0b3`, 16 calls, agreement in 4 rounds, and a run time of about 35 seconds.
- On 2026-10-13 the same run folder was packaged with `bvl package` and FULLMT `0.12.0` gave `VERIFIED CONFORMANCE` with all 20 rules `PASS`.
- **[Run-confirmed]** On 2026-10-16 it was run again on the rc6 installation from the text of this section, copied as is. The plan hash was the same (`sha256:f26d8b24…`), the run made 16 calls and was `COMPLETED` in about 50 seconds, and `bvl package` produced **the same package identity** `sha256:3befac5077e1dc8cad8b4d3ac9fef0d3349c4db8ad8a97849f9f1ea1247f20fb` (107 members). `bvl verify RUN --l1 --l2 --l3 --l4 --replay-world --experiment study.json` was `VERIFIED` with exit code 0, FULLMT was `VERIFIED CONFORMANCE` with 20 rules `PASS`, and `bvl analyse` gave the values in the table below. That the model answered exactly as before on this PC is an observation, not a guarantee (section 4.4).

The plan hash covers the bytes of the `study.json` below and of the four source files used at that time. If the content of a source file differs by even one character, the hash also differs.

What this template shows:

- All four actors respond with a real model.
- The World is applied twice per round (buyer side move, supplier side move).
- In each organization, the negotiator submits a draft and the manager approves it with `builtin.veto:1`. The manager sees the draft as `actor_output`.
- The ruling on every move is kept as lifecycle-type memory and passed to all four actors in later rounds.
- `builtin.terminal-state:1` terminates on agreement, breakdown, or expiry. The cap is 5 rounds.

`study.json` full text (expand)

```
{
  "schema": "bvl-study-1",
  "experiment_id": "sample-demo-two-org-negotiation-qwen",
  "episode_key": "episode-1",
  "root_seed": 20261013,
  "protocol": {
    "round": {
      "max_rounds": 5,
      "termination": {
```

```
        "rule": "builtin.terminal-state:1",
        "timing": "after_round_close",
        "config": {"path": ["status"], "success": ["agreed"], "failure": ["broken", "expired"]}
      }
    },
    "nodes": [
      {"id": "buyer_draft_call", "kind": "actor_call", "config": {"actor": "buyer_negotiator"}},
      {"id": "buyer_draft_parse", "kind": "proposal_parse", "depends_on": ["buyer_draft_call"]},
      {"id": "buyer_approval_call", "kind": "actor_call", "depends_on": ["buyer_draft_parse"],
       "config": {"actor": "buyer_manager"}},
      {"id": "buyer_approval_parse", "kind": "proposal_parse", "depends_on":
["buyer_approval_call"]},
      {"id": "buyer_decision", "kind": "decision",
       "depends_on": ["buyer_draft_parse", "buyer_approval_parse"]},
      {"id": "buyer_move", "kind": "world_transition", "depends_on": ["buyer_decision"],
       "config": {"world": "bvl.negotiation-world:1"}},
      {"id": "supplier_draft_call", "kind": "actor_call", "depends_on": ["buyer_move"],
       "config": {"actor": "supplier_negotiator"}},
      {"id": "supplier_draft_parse", "kind": "proposal_parse", "depends_on":
["supplier_draft_call"]},
      {"id": "supplier_approval_call", "kind": "actor_call", "depends_on": ["supplier_draft_parse"],
       "config": {"actor": "supplier_manager"}},
      {"id": "supplier_approval_parse", "kind": "proposal_parse",
       "depends_on": ["supplier_approval_call"]},
      {"id": "supplier_decision", "kind": "decision",
       "depends_on": ["supplier_draft_parse", "supplier_approval_parse"]},
      {"id": "supplier_move", "kind": "world_transition", "depends_on": ["supplier_decision"],
       "config": {"world": "bvl.negotiation-world:1"}},
      {"id": "buyer_move_minutes", "kind": "memory_update", "depends_on": ["buyer_move"],
       "config": {"owner": "buyer_negotiator", "key": "buyer_move_minutes", "scope": "episode",
                  "retention": 4, "transform": "builtin.outcome-memory-v2:1"}},
      {"id": "supplier_move_minutes", "kind": "memory_update", "depends_on": ["supplier_move"],
       "config": {"owner": "supplier_negotiator", "key": "supplier_move_minutes", "scope": "episode",
                  "retention": 4, "transform": "builtin.outcome-memory-v2:1"}}
    ]
  },
  "initial_world_state": {"status": "open", "turn": "buyer", "offers": {}, "price": 0, "moves": 0,
                          "deadline": 10},
  "world": {
    "plugin": "bvl.negotiation-world:1",
    "state_schema": "bvl.negotiation-state:1",
    "action_schema": "bvl.negotiation-action:1",
    "no_action": "EXOGENOUS_PROGRESSION",
    "exogenous_events": ["deadline", "turn_passed"]
  },
  "roles": ["negotiator", "manager"],
  "profiles": [
    {"profile_id": "buyer-negotiator-profile", "provider": "bvl.openai-chat-adapter:1",
     "model": "qwen2.5:7b-instruct", "renderer": "bvl.json-renderer:1",
     "options": {"temperature": 0, "max_tokens": 96, "response_format": "json_object"}},
    {"profile_id": "buyer-manager-profile", "provider": "bvl.openai-chat-adapter:1",
     "model": "qwen2.5:7b-instruct", "renderer": "bvl.json-renderer:1",
     "options": {"temperature": 0, "max_tokens": 96, "response_format": "json_object"}},
    {"profile_id": "supplier-negotiator-profile", "provider": "bvl.openai-chat-adapter:1",
     "model": "qwen2.5:7b-instruct", "renderer": "bvl.json-renderer:1",
     "options": {"temperature": 0, "max_tokens": 96, "response_format": "json_object"}},
    {"profile_id": "supplier-manager-profile", "provider": "bvl.openai-chat-adapter:1",
     "model": "qwen2.5:7b-instruct", "renderer": "bvl.json-renderer:1",
     "options": {"temperature": 0, "max_tokens": 96, "response_format": "json_object"}}
  ],
  "actors": [
    {"actor_id": "buyer_negotiator", "role": "negotiator", "profile": "buyer-negotiator-profile"},
    {"actor_id": "buyer_manager", "role": "manager", "profile": "buyer-manager-profile"},
    {"actor_id": "supplier_negotiator", "role": "negotiator",
```

```
     "profile": "supplier-negotiator-profile"},
    {"actor_id": "supplier_manager", "role": "manager", "profile": "supplier-manager-profile"}
  ],
  "information": {
    "sources": [
      {"source_id": "buyer_negotiator_brief", "kind": "frozen_resource",
       "path": "sources/buyer_negotiator_brief.json"},
      {"source_id": "buyer_manager_brief", "kind": "frozen_resource",
       "path": "sources/buyer_manager_brief.json"},
      {"source_id": "supplier_negotiator_brief", "kind": "frozen_resource",
       "path": "sources/supplier_negotiator_brief.json"},
      {"source_id": "supplier_manager_brief", "kind": "frozen_resource",
       "path": "sources/supplier_manager_brief.json"},
      {"source_id": "buyer_draft", "kind": "actor_output", "producer": "buyer_draft_parse"},
      {"source_id": "supplier_draft", "kind": "actor_output", "producer": "supplier_draft_parse"},
      {"source_id": "buyer_move_minutes", "kind": "memory_artifact",
       "producer": "buyer_move_minutes"},
      {"source_id": "supplier_move_minutes", "kind": "memory_artifact",
       "producer": "supplier_move_minutes"}
    ],
    "access": [
      {"actor": "buyer_negotiator", "source": "buyer_negotiator_brief", "effect": "ALLOW"},
      {"actor": "buyer_negotiator", "source": "buyer_move_minutes", "effect": "ALLOW"},
      {"actor": "buyer_negotiator", "source": "supplier_move_minutes", "effect": "ALLOW"},
      {"actor": "buyer_manager", "source": "buyer_manager_brief", "effect": "ALLOW"},
      {"actor": "buyer_manager", "source": "buyer_draft", "effect": "ALLOW"},
      {"actor": "buyer_manager", "source": "buyer_move_minutes", "effect": "ALLOW"},
      {"actor": "buyer_manager", "source": "supplier_move_minutes", "effect": "ALLOW"},
      {"actor": "supplier_negotiator", "source": "supplier_negotiator_brief", "effect": "ALLOW"},
      {"actor": "supplier_negotiator", "source": "buyer_move_minutes", "effect": "ALLOW"},
      {"actor": "supplier_negotiator", "source": "supplier_move_minutes", "effect": "ALLOW"},
      {"actor": "supplier_manager", "source": "supplier_manager_brief", "effect": "ALLOW"},
      {"actor": "supplier_manager", "source": "supplier_draft", "effect": "ALLOW"},
      {"actor": "supplier_manager", "source": "buyer_move_minutes", "effect": "ALLOW"},
      {"actor": "supplier_manager", "source": "supplier_move_minutes", "effect": "ALLOW"}
    ]
  },
  "proposals": {
    "schemas": [
      {"schema": "bvl-proposal-schema-1", "schema_id": "negotiation-move", "version": 1,
       "fields": {"side": "string", "move": "string", "price": "integer"},
       "allow_no_action": true}
    ],
    "contracts": [
      {"node_id": "buyer_draft_parse", "schema": "negotiation-move:1",
       "parser": "bvl.json-proposal-parser:1", "on_invalid": "NO_ACTION"},
      {"node_id": "buyer_approval_parse", "schema": "negotiation-move:1",
       "parser": "bvl.json-proposal-parser:1", "on_invalid": "NO_ACTION"},
      {"node_id": "supplier_draft_parse", "schema": "negotiation-move:1",
       "parser": "bvl.json-proposal-parser:1", "on_invalid": "NO_ACTION"},
      {"node_id": "supplier_approval_parse", "schema": "negotiation-move:1",
       "parser": "bvl.json-proposal-parser:1", "on_invalid": "NO_ACTION"}
    ]
  },
  "decisions": [
    {"node_id": "buyer_decision", "rule": "builtin.veto:1",
     "authority": ["buyer_negotiator", "buyer_manager"], "on_unresolved": "NO_ACTION"},
    {"node_id": "supplier_decision", "rule": "builtin.veto:1",
     "authority": ["supplier_negotiator", "supplier_manager"], "on_unresolved": "NO_ACTION"}
  ],
  "analysis": {
    "analysis_id": "sample-demo-negotiation-summary",
    "metrics": [
      {"metric_id": "decisions", "kind": "unit_count", "table": "decisions"},
```

```
      {"metric_id": "accepted", "kind": "action_share", "field": "move", "equals": "accept"},
      {"metric_id": "mean_price", "kind": "action_mean", "field": "price"}
    ]
  },
  "plugin_config": {
    "bvl.openai-chat-adapter:1": {"endpoint": "http://127.0.0.1:11434/v1", "timeout_seconds": 300}
  }
}
```

The four source files (`sources\`). Their content is exactly what was used in the reference run. Each file is one-line JSON.

`sources\*.json` four files (expand)

`sources\buyer_negotiator_brief.json`

```
{"organization": "buyer", "position": "negotiator", "good": "1000 control boards; price is the unit price", "task": "You negotiate the unit price with the supplier. world.offers.buyer is your own bid on the table, world.offers.seller is the ask of the supplier on the table (a missing member means no offer yet). The memory sources list the rulings on earlier moves. Propose your next move; your manager must approve it before it reaches the table.", "limits": "Never offer or accept a price above 90. Never lower a bid you already made.", "policy": "Open with a bid of 60. Each later round raise your bid by 8, never above 90. If the ask of the supplier is at or below the bid you would make next, accept that ask instead.", "reply": "Answer with one JSON object and nothing else: {\"side\": \"buyer\", \"move\": \"offer\" or \"accept\" or \"walk_away\", \"price\": <integer>}. For accept, price is exactly world.offers.seller."}
```

`sources\buyer_manager_brief.json`

```
{"organization": "buyer", "position": "manager", "good": "1000 control boards; price is the unit price", "task": "You approve or reject the move your negotiator drafted. The draft is the information source buyer_draft. world.offers.seller is the ask of the supplier on the table.", "approval_rule": "Approve when side is buyer and either move is offer with a price from 1 to 90, or move is accept with a price equal to world.offers.seller and not above 90, or move is walk_away. Reject anything else.", "reply": "Answer with one JSON object and nothing else. To approve, repeat the draft exactly: the same side, move and price. To reject, answer {\"side\": \"buyer\", \"move\": \"reject\", \"price\": 0}."}
```

`sources\supplier_negotiator_brief.json`

```
{"organization": "supplier", "position": "negotiator", "good": "1000 control boards; price is the unit price", "task": "You negotiate the unit price with the buyer. world.offers.seller is your own ask on the table, world.offers.buyer is the bid of the buyer on the table (a missing member means no offer yet). The memory sources list the rulings on earlier moves. Propose your next move; your manager must approve it before it reaches the table.", "limits": "Never offer or accept a price below 75. Never raise an ask you already made.", "policy": "Open with an ask of 110. Each later round lower your ask by 10, never below 75. If the bid of the buyer is at or above the ask you would make next, accept that bid instead.", "reply": "Answer with one JSON object and nothing else: {\"side\": \"seller\", \"move\": \"offer\" or \"accept\" or \"walk_away\", \"price\": <integer>}. For accept, price is exactly world.offers.buyer."}
```

`sources\supplier_manager_brief.json`

```
{"organization": "supplier", "position": "manager", "good": "1000 control boards; price is the unit price", "task": "You approve or reject the move your negotiator drafted. The draft is the information source supplier_draft. world.offers.buyer is the bid of the buyer on the table.", "approval_rule": "Approve when side is seller and either move is offer with a price of 75 or more, or move is accept with a price equal to world.offers.buyer and not below 75, or move is walk_away. Reject anything else.", "reply": "Answer with one JSON object and nothing else. To approve, repeat the draft exactly:
```

```
the same side, move and price. To reject, answer {\"side\": \"seller\", \"move\": \"reject\",
\"price\": 0}."}
```

The negotiator sources contain `limits` (limits), `policy` (opening price and concession step), and `reply` (response format). The manager sources contain `approval_rule` (approval criteria) and `reply`.

Run order (folder layout: `study.json`, the four files in `sources\`):

```
ollama list
bvl validate --experiment study.json
bvl check    --experiment study.json
bvl run      --experiment study.json --into RUN
bvl package  RUN
fullmt       RUN --out fullmt_result.json
```

If `qwen2.5:7b-instruct` is not in `ollama list`, get it with `ollama pull qwen2.5:7b-instruct`. Give `--into` a folder that does not exist yet. Create `fullmt_result.json` outside `RUN`.

Results of the reference run.

Moves that reached the table (`evidence-v2\world_transition\`, in World ruling order) **[Run-confirmed]**:

Order	Side	Requested move	Price	Admissibility ruling	Ruling	Status
1	buyer	offer	60	ADMISSIBLE	tabled	open
2	seller	offer	110	ADMISSIBLE	tabled	open
3	buyer	offer	68	ADMISSIBLE	tabled	open
4	seller	offer	100	ADMISSIBLE	tabled	open
5	buyer	offer	76	ADMISSIBLE	tabled	open
6	seller	offer	90	ADMISSIBLE	tabled	open
7	buyer	offer	84	ADMISSIBLE	tabled	open
8	seller	offer	80	CLIPPED	crossed	agreed

On the 8th move the supplier side asked 80, which crossed the buyer side's offer of 84, so the World processed it as an **agreement at the buyer side's price of 84** (`crossed` / `CLIPPED`). All 8 decisions were `AGGREGATE`, with the negotiator and manager answering the same move. The manager never rejected.

Meaning of ruling values:

Value	Meaning
`tabled`	The proposal was placed on the table
`crossed` / `CLIPPED`	The proposal crossed the other side's proposal and was processed as an agreement at the other side's price
`held` / `CLIPPED`	The side tried to retract its own proposal, so the existing proposal was kept
`accepted`	The other side's price was accepted as is
`no_action`	No move was executed in that turn (manager rejection or response format error)

Main values in the FULLMT result file (the same on 2026-10-13 and on the rc6 rerun) **[Run-confirmed]**:

Item	Value
`outcome` / `meaning`	`VERIFIED` / `CONFORMANCE`
`summary`	`PASS` 20, `FAIL` 0, `NOT_EVALUATED` 0
Package identity	`sha256:3befac5077e1dc8cad8b4d3ac9fef0d3349c4db8ad8a97849f9f1ea1247f20fb`
Plan	`sha256:f26d8b24bd5ad2f74c742684dcc689a46b7386f0a74961afb6c1581b6bcaa0b3`
Package file count	107
Evidence level (FULLMT)	`L1`. `bvl verify` with the level options confirms L1 to L4 (`actor-to-actor hand-overs: 8`, `transitions: 8`, `rounds closed: 4`, `memory artifacts: 8`)
Steps	`actor_call` 16, `proposal_parse` 16, `decision` 8, `world_transition` 8, `memory_update` 8
Rounds	5 planned, 4 executed (ended by the termination rule)
Analysis (`COMPUTED`)	`decisions` = `8/1`, `accepted` = `0/1`, `mean_price` = `167/2` (= 83.5)
Reader	`fullmt-manifest4-reader` `0.12.0`, producer contract `boomvalley-lab` `v1.0.0` (`db21f42`)

`mean_price` is the mean of the 8 prices requested by the decisions (60, 110, 68, 100, 76, 90, 84, 80). It is not the agreed price of 84 (section 6.5, "Targets of analysis metrics"). `accepted` being 0 has the same cause. The agreement came from the World's `crossed` ruling, not from an `accept` move.

Limits attached to the result: consumer side `CONFORMANCE_NOT_SCIENTIFIC_EVIDENCE`, `NOT_SIGNED`, `WORLD_CODE_NOT_REPLAYED`, `ACTOR_CALLS_NOT_REEXECUTED`, `BUILD_AND_PLUGIN_SET_SELF_REPORTED`, `GATE_INPUTS_SUPPLIED_NOT_DERIVED`, `OBSERVATION_DIGESTS_ATTESTED_NOT_RECOMPUTED`. Producer declarations `CONFORMANCE_PACKAGE_NOT_SCIENTIFIC_EVIDENCE`, `FULLMT_MANIFEST4_SPIKE_READER_REQUIRED`, `GATE2_INPUTS_SUPPLIED_NOT_DERIVED`, `PLUGIN_SET_NOT_ENGINE_RESOLVED`. Their meanings are in section 6.3.

How to adapt this demo.

What to change	Where to edit
Negotiator's opening price, concession step, limits	`limits`, `policy` in the negotiator source file
Manager's approval criteria	`approval_rule` in the manager source file
Maximum number of rounds	`protocol.round.max_rounds` and `initial_world_state.deadline` (= number of rounds × 2)
Memory retention period	`retention` of the `memory_update` nodes
Model	`profiles[].model` (must exactly match the name the server reports)
Server address	`endpoint` in `plugin_config`

Leave `reply` (the response format instruction) in the source files unchanged. If the format changes, moves are not executed. Any edit changes the plan hash, so run into a new `--into` folder.

Points to note when designing:

- Approval is "answering by repeating the same action". The manager source must instruct this.
- Rejection was set up as answering a different action that still fits the schema. The decision then becomes `NO_ACTION`, and in the negotiation World that turn is lost.

- No rejection occurred in the reference run. In the negotiation demo run on 2026-10-17 with the same source files (`examples\manual_negotiation_demo`), the supplier manager in round 2 answered a different price (110) than the draft (100). The decision became `NO_ACTION` and the supplier lost that turn. That run also reached agreement at 84 in round 4 **[Run-confirmed]**. A deliberate rejection using the reject form (`"move": "reject"`) has not been observed yet. How to read the run record is in section 5.13.
- Because the sources specify the concession step, the result is the execution of an instructed policy. It does not demonstrate free bargaining.
- Even if the negotiation ends on a buyer side move, the supplier side of that round is called once more (the termination check happens after the round closes).

4.7 Template for using GNDCRW alone

The five official examples in section 2.5 are the templates as is. How to design the compared items yourself, with a worked example (the two organizations of a negotiation study), is in section 2.5.9. What to change and what to keep when writing your own:

Part	Mark
The five `*_version` values	Fixed
All of `composition_targets`	Fixed (as in the examples)
`equality` value	Fixed (`JSON_STRUCTURAL`)
`catalog_id`, `profile_id`, `provider_id`, `declaration_id`, `intent_id`, `surface_id`	Edit (keep reference relationships)
Item names and descriptions in `targets`	Edit. The key set must be the same in the three files (catalog, base, candidate).
Observed values (`value`, `entity_values`)	Edit. The researcher copies them from the actual assembly result.
`expected_changes`, `invariants`	Edit
`catalog_digest`, `intent_digest`	Write down the values that GNDCRW computes and reports (section 2.5.7)
`sha256`, `size` in the manifest	Compute after all content is finalized (section 2.5.7)

Verification status: [Run-confirmed]. `gndcrw check` on the official folder `EX\b2b_frontline_internal_exchange_qwen_100_158\stage_a\gndcrw\` gives `RUN` with 4 findings `PASS`. Changing one invariant value in `candidate.json` and binding the manifest again gives `BLOCK` (`INV-CUSTOMER-REQUEST` `FAIL`, `INVARIANT_VIOLATED`). Section 5.10 shows the commands. That folder is not included in the anonymous review edition (section 0.1); the included `examples\manual_negotiation_demo\gndcrw\` gives `RUN` with 7 findings `PASS` (section 2.5.9).

Part 5. Windows CMD Run Guide

5.1 Preparation

Command location. Toolkit Setup adds `C:\Program Files\BOOMVALLEY Research Toolkit\bin` to the system PATH. That folder holds `bvl.cmd`, `boomvalley-lab.cmd`, `gndcrw.cmd` and `fullmt.cmd`. In rc4 and later it also holds `bvt.cmd` and `boomvalley-toolkit.cmd`. Check with the following in a CMD window **opened after** the Toolkit installation.

```
where bvl
where gndcrw
where fullmt
where bvt
bvt --version
```

Each `where` prints one line under `C:\Program Files\BOOMVALLEY Research Toolkit\bin\` **[Run-confirmed]**. If they are not found, open a new window or run with the full path.

```
"C:\Program Files\BOOMVALLEY Research Toolkit\bin\bvl.cmd" validate --experiment study.json
```

`bvl` and `boomvalley-lab` are different commands. Every procedure in this manual uses `bvl`. `boomvalley-lab` is the command of the old approach, and its `run` is retired.

Working folder. The examples below assume you run them in the folder that contains `study.json`.

```
cd /d C:\work\my_study
```

Keep this path short. Unless long paths are enabled in Windows, a full path may have at most 260 characters. The deepest files in a run folder are about 115 characters below the run folder (for example `evidence-v2\conformance_attestation\<64 hex>.json`, and a temporary `.<64 hex>.json.partial` while writing). Keep the full path of the run folder under about 130 characters. On the check PC **[Run-confirmed]**: run folders with a path of 155 to 173 characters failed at once with `execution: [Errno 2] No such file or directory: '...\.<hash>.json.partial'` (exit code 5); a 148-character run folder ran to `COMPLETED`, but `bvl package` then failed with exit code 5; a 142-character run folder worked through FULLMT. The same studies all worked in short folders. A folder such as `C:\work\<study>\RUN` is safe; a folder deep inside `Documents`, OneDrive or a long project tree may not be.

Checking the exit code. Run this right after a command.

```
echo %ERRORLEVEL%
```

Saving output. Redirect standard output to a file. Do not save it inside the `RUN` folder.

```
bvl check --experiment study.json --json > check-result.json
```

Display language. `bvl` and `gndcrw` accept `--locale ko-KR` **as the first argument**. Only human-readable sentences change. Codes, exit codes and `--json` output stay the same.

```
bvl --locale ko-KR validate --experiment study.json
```

With `bvl`, the sentence lines are translated (for example the `valid:` line of `validate`) **[Run-confirmed]**. With `gndcrw`, only the error messages change; the text report of `gndcrw check` stays in English **[Run-confirmed]**.

When using a real model. Check that Ollama is running and the model is present.

```
ollama list
```

5.2 Common points for `bvl`

- Every subcommand accepts `--json`. The result is printed as JSON.
- **Some detailed error reasons appear only in `--json`.** If the text output does not show a reason, run again with `--json` and look at `problems`.
- Errors go to standard error in the form `"<class>: <message>"`.

Exit codes (`PKG\core\errors.py`, `PKG\cli\bvl.py`):

Code	Meaning	Typical case
0	Success	
2	Declaration or usage is wrong	`BVL-LIFE-001`, invalid option
3	Identity problem	The `--into` folder already holds evidence (`BVL-LIFE-002`)
4	A claim was rejected in verification	`bvl verify` failure, `BVL-LIFE-007`
5	Execution failure	Run is `FAILED`, file I/O error
6	Evidence package problem	Packaging refused, already sealed
8	Gate did not open	`REFUSED` in `check` or `run`. Not an error; the result goes to standard output.
9	Stopped, waiting for a human answer	Study with a human actor
70	Internal error	Defect in the tool

5.3 `bvl validate`

```
bvl validate --experiment study.json
```

- Reads: `study.json` and the source files it points to.
- Writes: nothing.
- Success output **[Run-confirmed]**:

```
valid: sample-demo-two-org-negotiation-qwen (1 design cell(s))
  cell: plan sha256:f26d8b24bd5ad2f74c742684dcc689a46b7386f0a74961afb6c1581b6bcaa0b3
```

The plan hash of each cell is shown. If you change the declaration or a source, this value changes. If you keep this value in your research records, you can fix "which declaration was run".

- Exit codes: 0, 2 (declaration error), 5 (file could not be read).

5.4 `bvl check`

```
bvl check --experiment study.json
bvl check --experiment study.json --cell cell.worker_model-a
```

- `--cell` is required when there are two or more cells. Without it: `BVL-LIFE-001 study declaration is invalid: the design has 2 cells; name one with --cell (...)`, exit code 2 **[Run-confirmed]**.
- Writes: nothing.
- What it does: runs the coverage check, the GNDCRW judgment on comparison edges (if any), and plugin preflight, then issues the gate decision.
- Success output **[Run-confirmed]**:

```
gate-2 CLEARED for cell (plan sha256:f26d8b24…)
  execution: NOT_STARTED
```

- On refusal, `gate-2 REFUSED` is printed with reasons on `- <reason>` lines, and the exit code is 8 **[Run-confirmed]**. Reason codes are in section 7.3.

```
gate-2 REFUSED for cell.worker_model-a (plan sha256:d2b65260…)
    - BVL-GATE-003:edge.worker_model.a-b:BLOCK
  execution: NOT_STARTED
```

- Exit codes: 0 (`CLEARED`), 8 (`REFUSED`), 2, 5.

5.5 `bvl run`

```
bvl run --experiment study.json --into RUN
bvl run --experiment study.json --cell cell.worker_model-a --into RUN_a
```

- `--into` : the run folder. It must be **a folder that does not exist or an empty folder**. Giving a folder that holds evidence yields `BVL-LIFE-002` (exit code 3).
- With a human actor, `--human-desk <folder>` is required. With a tool actor, `--tool-site <folder>` is required. Both must be outside the run folder.
- Success output **[Run-confirmed]**:

```
gate-2 CLEARED for cell (plan sha256:f26d8b24…)
run sample-demo-two-org-negotiation-qwen.cell: COMPLETED
  calls: 16 custody-accepted, 0 pending
  execution: COMPLETED
```

- In `calls` , `custody-accepted` is the number of calls whose response was accepted as evidence. `pending` is the number of calls whose request was recorded but whose response was not accepted.
- Failure output **[Run-confirmed]** (exit code 5):

```
gate-2 CLEARED for cell (plan sha256:ac938047…)
run my-first-study.cell: FAILED
  calls: 0 custody-accepted, 1 pending
  execution: FAILED
```

- **The text output does not say why a run failed.** The reason is printed only in the `failure` field of `bvl run ... --json`, for example `"failure": "ProviderError: PROVIDER_UNREACHABLE: POST /chat/completions: cannot reach http://127.0.0.1:1/v1 (ConnectionRefusedError)"`. `journal-2.jsonl` records only the error class (`"reason": "ProviderError"`), and `bvl status --json` has no `failure` field **[Run-confirmed]**. For a run with a real model, keep the JSON output: `bvl run --experiment study.json --into RUN --json > run-result.json` (the file is written next to `study.json`, outside `RUN`). If you ran without `--json`, run the same study again into a new folder with `--json` to see the reason.
- On a gate refusal the folder is still created and keeps the refusal record: `run <name>: REFUSED`, `calls: 0 custody-accepted, 0 pending`, exit code 8 **[Run-confirmed]**.
- Exit codes: 0 (`COMPLETED`), 8 (gate refusal; the folder is still created and the refusal record is kept), 9 (waiting for a human), 5 (`FAILED`), 3, 2.

Run states (`PKG\lifecycle\gated.py`):

State	Meaning
`REFUSED`	The gate refused. It stays closed. Fixing the declaration means a new run.
`OPENED`	Opened, but no episode has started.
`INTERRUPTED`	The process stopped in the middle of an episode.
`WAITING`	Waiting for a human answer.
`COMPLETED`	All episodes finished.
`FAILED`	An episode was recorded as failed.
`SEALED`	`COMPLETED`, plus a valid `MANIFEST-4.json` exists.

A failed run cannot be continued. A run that became `FAILED` from a model call failure or a parse failure (`PARSE_FAILURE`) does not continue with `bvl resume` and cannot be packaged. Fix the cause and run again in a **new folder**.

Contents of the run folder:

Path	Contents
`journal-2.jsonl`	Record of all events. Append-only.
`requests-v2\request-*.json`	The request sent for each call. Recorded before the call.
`artifacts\call-*.bin`	The raw response bytes of each call.
`custody-v2\entries.jsonl`	Response receipt records.
`evidence-v2\plan\`	The plan that was run (one).
`evidence-v2\gate\`	Gate decision.
`evidence-v2\coverage\`	Coverage check record.
`evidence-v2\gndcrw_results\`	The GNDCRW result set for comparison edges.
`evidence-v2\execution_binding\`	Binding of the plan, installation build and plugin set.
`evidence-v2\call_binding\`	Per call: who called, and what it could see.
`evidence-v2\proposal\`, `decision\`, `memory\`	Proposals, decisions, memory.
`evidence-v2\world_state\`, `world_transition\`	World states and transitions.

The name of each file under `evidence-v2` is the SHA-256 of its content.

5.6 `bvl status` and `bvl resume`

```
bvl status RUN
bvl resume RUN --plan
bvl resume RUN --experiment study.json
```

- `status`: read-only. Shows the gate decision, run state and call counts.
- `resume --plan`: read-only. Tells you what can be done (`resume_action`).
- `resume --experiment`: continues a run whose process was interrupted (`INTERRUPTED`). It judges the declaration again, and the same gate decision as the original must result (`BVL-LIFE-004`).
- Cases that cannot be continued: `FAILED`, `COMPLETED` and `SEALED` do nothing. A run that stopped after a call was sent but before its result was confirmed is refused with `BVL-LIFE-005`. The tool does not call the provider again on its own.

Confirmed on the rc6 installation **[Run-confirmed]**: `status` prints the gate line, the run state and the call counts (exit code 0) for `COMPLETED`, `SEALED`, `FAILED` and `REFUSED` runs. `resume --plan` and `resume --experiment` on a `COMPLETED`, `SEALED` or `FAILED` run print `resume: NONE` with exit code 0 and change nothing. `resume --experiment` on a refused run is `BVL-LIFE-003 ... was refused by gate-2 and stays closed; a changed study is a new run` (exit code 5). Continuing an `INTERRUPTED` run was not checked **[Unverified]**.

5.7 `bvl package`

```
bvl package RUN
```

- Precondition: a run whose gate was `CLEARED` and whose episodes all completed normally. If even one refused, failed or unknown event is recorded, it is refused with `manifest-4 package refused: ...` (exit code 6).
- Writes (inside the `RUN` folder):
  1. `evidence-v2\conformance_attestation\<hash>.json`
  2. `evidence-v2\dataset_units\<hash>.json` (runs with call binding evidence)
  3. `evidence-v2\analysis_output\<hash>.json` (when `analysis` is declared)
  4. `MANIFEST-4.json`
- Success output **[Run-confirmed]**:

```
sealed CONFORMANCE package: 107 members, manifest
sha256:3befac5077e1dc8cad8b4d3ac9fef0d3349c4db8ad8a97849f9f1ea1247f20fb
```

- **It can be done only once.** Running it again gives `evidence: manifest-4 is immutable and already exists` (exit code 6) **[Run-confirmed]**.
- On a `FAILED` run it is refused with the list of reasons, for example `evidence: manifest-4 package refused: journal-2 #5 is CALL_OUTCOME_UNKNOWN; journal-2 #6 is EPISODE_FAILED; ...` (exit code 6) **[Run-confirmed]**.
- Exit codes: 0, 6, 3.

Note: the description in `bvl package --help` ("package a SEALED run as an EPC-v1 evidence package") is wording from the old format. A `bvl-study-1` run produces a MANIFEST-4 package.

5.8 `bvl verify`

```
bvl verify RUN
bvl verify RUN --l1 --l2
bvl verify RUN --l1 --l2 --l3 --l4 --replay-world --experiment study.json
```

- Writes: nothing. **It does not call the model or the provider.**
- Without options: checks that the package is internally consistent. The manifest format, the size and hash of every file, that no file is missing from the list, and that the attestation, dataset and analysis are the same when recomputed.
- With options, success requires that level to be met.

Option	What it requires (wording from `PKG\kernel\l*_conformance.py`)
`--l1`	The run was produced only from the declaration and registered plugins, through the gate; every provider call is accounted for; and official figures were read only from pre-bound analysis.
`--l2`	An actor's committed output was passed as authorized input to the next actor in the same episode, and that hand-over is traceable through evidence.
`--l3`	A decision changed an explicit environment, and the side holding the World code recomputed it. Requires `--replay-world`.
`--l4`	Rounds proceeded until the declared stop rule stopped them, and memory is evidence. Requires a round contract (`round`). Requires `--replay-world`.
`--l5`	There is a human or tool actor, and only declared responders responded. A study whose actors are all models is not L5.
`--calls`	Every model call, human response and tool call belongs to a declared caller.
`--replay-world --experiment study.json`	Recomputes every recorded World transition with the World code on this PC. A study without a `world` section is `NOT_APPLICABLE` and is treated as a failure.

- Option dependencies: `--l3` and `--l4` must be used with `--replay-world --experiment`. `--replay-world` and `--experiment` must be used together.
- Output: `VERIFIED` or `REFUSED` on the first line, followed by a list of problems and per-level lines. Actual output for Template D with `--l1 --l2 --l3 --l4 --replay-world --experiment study.json` **[Run-confirmed]**:

```
VERIFIED
P18 L1 conformance: L1 (hidden calls: 0)
P18 L2 conformance: L2 (actor-to-actor hand-overs: 8)
World replay: REPLAYED (transitions: 8)
P18 L3 conformance: L3 (decision-to-World transitions: 8)
P18 L4 conformance: L4 (rounds closed: 4, memory artifacts: 8)
Call provenance: ALL_DECLARED (hidden calls: 0; model 16, human 0, tool 0)
P18 L5 conformance: NOT_L5 (human exchanges closed: -, tool calls resolved: -)
    - Y2 the plan contracts no human and no tool actor: every caller is a model (not an L5 run)
```

The World recomputation, L3 and L4 lines appear in the text output only when `--replay-world` is given. A level that is not met is shown as `NOT_L<n>` with its reason on a `-` line, even when you did not ask for

that level. See the full content with `--json`.

- **The first line is about the package, not about the levels you asked for.** When a requested level is not met, the first line still says `VERIFIED`, and only the exit code (4) tells you that the requirement failed **[Run-confirmed]**. In a batch file, branch on `%ERRORLEVEL%`, not on the first line.
- A package that does not verify prints `REFUSED` with the problems, for example `- unlisted file notes.txt`, or `size mismatch` and `digest mismatch` lines for an edited file (exit code 4) **[Run-confirmed]**.
- Exit codes: 0, 4 (verification failure or requested level not met).

Which levels you can require depends on the structure of the study. A study that uses `rounds` (simple repetition) does not pass `--l4` (`EX\k0_study\README.md`).

5.9 `bvl analyse`

```
bvl analyse RUN
```

- Preconditions: the package must pass verification (`BVL-LIFE-007`, exit code 4), and the study must have declared `analysis` (`BVL-LIFE-006`, exit code 5).
- Writes: nothing. **It does not compute anything new.** It reads and shows the analysis result recorded at sealing and recomputed at verification.
- Output for Template D **[Run-confirmed]**:

```
analysis sample-demo-negotiation-summary: COMPUTED
  accepted: [{"group":"*","hits":0,"n":8,"value":"0/1"}]
  decisions: [{"group":"*","n":8,"value":"8/1"}]
  mean_price: [{"group":"*","missing":0,"n":8,"value":"167/2"}]
```

Values are reduced fractions: `167/2` is 83.5, and a count appears like `"8/1"`. Metrics are printed in alphabetical order of `metric_id`. On a study without `analysis` it is `execution: BVL-LIFE-006 this plan pre-binds no analysis, so none may be reported` (exit code 5) **[Run-confirmed]**.

5.10 `gndcrw capabilities` and `gndcrw check`

`gndcrw` subcommands have no `--help`. `gndcrw check --help` is an error (exit code 2). Help works only as `gndcrw --help`.

```
gndcrw --version
gndcrw capabilities
```

`capabilities` takes no arguments and prints the supported contract versions and policies as one line of JSON. `gndcrw --version` prints `gndcrw 0.2.0`. Actual output, shown here broken into lines **[Run-confirmed]**:

```
{"capabilities_version":"gndcrw-capabilities/v0.2",
 "code_digest_method":"gndcrw-code-digest/v0.2",
 "contracts":{"declaration":"gndcrw-declaration/v0.1","pair":"gndcrw-pair/v0.1","surface":"gndcrw-
surface/v0.1","target_catalog":"gndcrw-target-catalog/v0.1","verification":"gndcrw-
verification/v0.1"},
 "legacy":{"result_version":"gndcrw-result/v0.1"},
 "manifest":{"manifest_versions":["gndcrw-manifest/v0.2"],"result_version":"gndcrw-result/v0.2"},
```

```
  "policy_ids":["GNDCRW-STRICT-0.1"],
  "tool":{"name":"gndcrw","version":"0.2.0"}}
```

Running a check (in the folder with the six files from section 2.5):

```
gndcrw check manifest.json
gndcrw check manifest.json --format json --output ..\gndcrw-result.json
gndcrw check manifest.json --gate
```

Option	Meaning
`--format text` / `json`	Output format. Default is `text`.
`--gate`	Returns different exit codes by result: 10 for `BLOCK`, 11 for `UNVERIFIABLE`.
`--output <path>`	Writes the result to a file. If it already exists, `OUTPUT_EXISTS` (exit code 6).
`--overwrite`	With `--output`. Allows overwriting.

Options cannot be abbreviated, and the same option cannot be given twice. Specifying an input file with `--output` is refused.

Exit codes (`GND\cli.py`):

Code	Meaning
0	A result was produced. **Without `--gate`, it is 0 even for `BLOCK` or `UNVERIFIABLE`.**
2	Command usage error
3	Invalid input (manifest, references, content)
5	Internal error
6	Failed to write the result
10	`--gate` and the result is `BLOCK`
11	`--gate` and the result is `UNVERIFIABLE`

To branch on the result in a batch file, always give `--gate`. Without it, the exit code alone cannot distinguish `RUN` from `BLOCK`.

The text output shows, in order, a header (tool, policy, `Outcome`, comparison, declaration, digests and the hash and size of each input file), `Findings` (judgment and reason per rule), `Premises`, `Limitations`, `Summary`, and the `Outcome` line again at the end **[Run-confirmed]**. Main keys of the JSON output (`gndcrw-result/v0.2`): `outcome`, `findings[]` (each with `finding_id`, `status`, `policy_class`, `reason_code`, `expected`, `observed`), `limitations[]`, `summary`, `binding` (the actual hashes and sizes of the input files).

Errors appear on the first line of standard error in the form `gndcrw: error: <code>: <description>`.

Confirmed on the rc6 installation **[Run-confirmed]**:

Command	Result
`gndcrw check manifest.json` (official example)	`Outcome RUN`, `PASS=4`, exit code 0
the same with an invariant changed in `candidate.json`	`Outcome BLOCK`, `FAIL=1` (`INVARIANT_VIOLATED`), exit code 0

Command	Result
the same with `--gate`	`Outcome BLOCK`, exit code 10
`--format json --output ..\gndcrw-result.json`	nothing on standard output, the file is written, exit code 0
the same again	`gndcrw: error: OUTPUT_EXISTS: ..\gndcrw-result.json exists; pass --overwrite`, exit code 6
`--output base.json`	`OUTPUT_IS_INPUT`, exit code 6
`--form json`	`OPTION_NOT_SUPPORTED: --form: not a manifest-mode option`, exit code 2
`--gate --gate`	`OPTION_REPEATED: --gate given twice`, exit code 2
`gndcrw check --help`	`OPTION_NOT_SUPPORTED: --help: not a manifest-mode option`, exit code 2
a folder without `manifest.json`	`MANIFEST_NOT_FOUND: manifest.json: no such file`, exit code 3

5.11 `fullmt`

```
fullmt RUN
fullmt RUN --quiet
fullmt RUN --out ..\fullmt-result.json
fullmt RUN --quiet --locale ko-KR
```

Argument	Meaning
`<folder>`	Package folder. `MANIFEST-4.json` must be at its top level.
`--out <path>`	Also writes the result JSON to a file. It must be **outside the `RUN` folder**.
`--quiet`	Prints only a one-line result.
`--locale en-US` / `ko-KR`	Language of help and diagnostic sentences. Standard output is the same regardless of language.
`--log-level`	Diagnostic level. Default `WARNING`.

- There are no subcommands. In the usage line of `fullmt --help`, the program name appears as `fullmt-epc-verify`. It is the same program.
- The package folder is only read.
- Default output: the whole result document is printed to standard output as JSON (section 2.6.4).
- `--quiet` output (example from the documentation, `FMT\README_WINDOWS.md`):

```
VERIFIED CONFORMANCE sha256:<package identity> failed=[]
```

On failure, the failed rules appear, as in `REJECTED CONFORMANCE sha256:... failed=['M4-MEM']`.

- In the installation, each run prints a one-line warning to standard error, `WARNING fullmt.reader: reader code identity unavailable: git rev-parse --show-toplevel: fatal: not a git repository ...`, and `verifier.code_digest` in the result is null **[Run-confirmed]**. This does not affect the judgment (`FMT\README_WINDOWS.md`).
- **`--out` inside `RUN` is not refused.** `fullmt RUN --out RUN\x.json` wrote the file into the package folder and reported `VERIFIED`; after that, `bvl verify RUN` was `REFUSED` (`unlisted file x.json`)

and FULLMT is `REJECTED` (`M4-UNL`) **[Run-confirmed]**. If it happens, delete that file from `RUN` .

- A path that is not a folder gives `ERROR None failed=[]` on standard output and `PACKAGE_NOT_A_DIRECTORY` on standard error (exit code 2) **[Run-confirmed]**.

Exit codes (`FMT\app\fullmt_codes.py`):

Code	Meaning
0	`VERIFIED`
1	`REJECTED`
2	`ERROR` ; or a judgment was made but the `--out` file could not be written; or a usage error
9009	The executable or the linked Python was not found (installation problem)

Exit codes 0, 1 and 2 and the `--quiet` lines above were confirmed; `--locale ko-KR` gave the same standard output **[Run-confirmed]**. Exit code 9009 was not produced.

5.12 The whole flow at a glance

For a study with one condition:

```
cd /d C:\work\my_study
bvl validate --experiment study.json
bvl check    --experiment study.json
bvl run      --experiment study.json --into RUN
bvl package  RUN
bvl verify   RUN
fullmt       RUN --out fullmt-result.json
bvl analyse  RUN
```

In the last three lines, `fullmt-result.json` is created next to `study.json` (that is, outside `RUN`). After each line, check the exit code with `echo %ERRORLEVEL%` before going on. This exact sequence ran with exit code 0 on every line for Templates A, B and D on the rc6 installation **[Run-confirmed]**.

5.13 Reading the run record — who received what, who answered what, what was decided

The text output of `bvl run` reports only `COMPLETED` and the number of calls. How the negotiation went is found by reading the records in the run folder. The commands below are **PowerShell** commands (in CMD, type `powershell` first). They only **read** the `RUN` folder, so they do not affect the package, before or after sealing. Run them in the folder that holds the run folder (next to `study.json`). If the run folder is not named `RUN` , change `RUN` in the commands.

Check status: [Run-confirmed]. On 2026-10-17, the commands below were run on the `RUN` of the negotiation demo (`examples\manual_negotiation_demo` , the same source files as section 4.6), executed on the rc6 installation. The example outputs are from that run. The same commands also worked on the `RUN` of Template D.

What is recorded where.

What you want to know	Where it is recorded	Event in `journal-2.jsonl`
Sources an actor was granted and denied in each call	`access.granted`, `access.denied` in `evidence-v2\call_binding\*.json`	`call_binding.path` of `CALL_INTENT_RECORDED`
The full message the actor actually received	`messages[0].content` in `requests-v2\request-*.json`	`request.name` of `CALL_INTENT_RECORDED`
The actor's raw answer	`artifacts\call-*.bin`	`artifact.name` of `CALL_PERSISTED`
How the answer was parsed	—	`proposal.verdict`, `proposal.value` of `PROPOSAL_PARSED`
The result of the decision rule	`evidence-v2\decision\*.json`	`decision.outcome`, `decision.requested_action` of `DECISION_RECORDED`
World ruling and the state of the table	`evidence-v2\world_transition\*.json`	`memory.value` of `MEMORY_RECORDED` (ruling summary)
The end-of-round termination check	—	`terminate`, `termination.reason` of `TERMINATION_EVALUATED`

`journal-2.jsonl` holds one JSON event per line, in the order the events happened. File names under `evidence-v2` are hashes of their content and carry no order. Follow the journal to read them in order.

① Decisions — did the manager approve?

```
Get-Content RUN\journal-2.jsonl -Encoding UTF8 | ConvertFrom-Json | Where-Object event -eq
DECISION_RECORDED | Select-Object round, @{n='decision';e={$_.decision.outcome}}, @{n='move';e=
{$_.decision.requested_action | ConvertTo-Json -Compress}}
```

```
round decision  move
----- --------  ----
    1 AGGREGATE {"move":"offer","price":60,"side":"buyer"}
    1 AGGREGATE {"move":"offer","price":110,"side":"seller"}
    2 AGGREGATE {"move":"offer","price":68,"side":"buyer"}
    2 NO_ACTION {}
    3 AGGREGATE {"move":"offer","price":76,"side":"buyer"}
    3 AGGREGATE {"move":"offer","price":100,"side":"seller"}
    4 AGGREGATE {"move":"offer","price":84,"side":"buyer"}
    4 AGGREGATE {"move":"offer","price":80,"side":"seller"}
```

Under `builtin.veto:1`, `AGGREGATE` means negotiator and manager answered the same move: an approval. `NO_ACTION` means the two answers differed and no move was adopted (`on_unresolved: NO_ACTION`). That organization makes no move in that turn.

② The table — how did the World rule?

```
Get-Content RUN\journal-2.jsonl -Encoding UTF8 | ConvertFrom-Json | Where-Object event -eq
MEMORY_RECORDED | Select-Object round, @{n='side';e={$_.memory.value.side}}, @{n='ruling';e=
{$_.memory.value.ruling}}, @{n='status';e={$_.memory.value.status}}, @{n='price';e=
{$_.memory.value.price}}, @{n='table';e={$_.memory.value.offers | ConvertTo-Json -Compress}}
```

```
round side   ruling    status price table
----- ----   ------    ------ ----- -----
    1 buyer  tabled    open       0 {"buyer":60}
    1 seller tabled    open       0 {"buyer":60,"seller":110}
    2 buyer  tabled    open       0 {"buyer":68,"seller":110}
    2 seller no_action open       0 {"buyer":68,"seller":110}
```

```
    3 buyer  tabled    open       0 {"buyer":76,"seller":110}
    3 seller tabled    open       0 {"buyer":76,"seller":100}
    4 buyer  tabled    open       0 {"buyer":84,"seller":100}
    4 seller crossed   agreed    84 {"buyer":84,"seller":100}
```

These are the memories recorded for each move (`*_move_minutes`). They are passed to all four actors in the next round. The meaning of each ruling is in the table of section 4.6.

③ Raw answers — who answered what?

```
$ev = Get-Content RUN\journal-2.jsonl -Encoding UTF8 | ConvertFrom-Json
$af = @{}; $ev | Where-Object event -eq CALL_PERSISTED | ForEach-Object { $af[$_.call] =
$_.artifact.name }
$ev | Where-Object event -eq CALL_INTENT_RECORDED | ForEach-Object { $b = Get-Content (Join-Path RUN
$_.call_binding.path) -Raw -Encoding UTF8 | ConvertFrom-Json; [pscustomobject]@{round=$_.round;
actor=$b.access.actor_id; answer=((Get-Content (Join-Path RUN\artifacts $af[$_.call]) -Raw -Encoding
UTF8) -replace '\s+',' ').Trim()} }
```

Round 2:

```
round actor              answer
----- -----              ------
    2 buyer_negotiator   { "side": "buyer", "move": "offer", "price": 68 }
    2 buyer_manager      {"side":"buyer","move":"offer","price":68}
    2 supplier_negotiator { "side": "seller", "move": "offer", "price": 100 }
    2 supplier_manager   { "side": "seller", "move": "offer", "price": 110 }
```

This shows the cause of the round 2 `NO_ACTION` in ①. The supplier negotiator drafted 100, but the supplier manager did not repeat the draft and answered 110. It did not use the reject form of its brief (`"move": "reject"`) either. Both answers were well formed (`verdict` `VALID` in `PROPOSAL_PARSED`), so the veto rule left the decision unresolved. In round 3 the supplier drafted 100 again and it was approved. In round 4 the supplier's 80 crossed the buyer's 84, and the price was agreed at 84. The message this manager received (⑤ with `-Index 7`) held the draft 100 together with the organization's own earlier ask of 110 still on the table (`world.offers.seller`). **The decision record alone cannot tell a deliberate rejection from a failure to follow the brief.** To tell them apart, read the raw answers as here.

④ Information — who received which sources?

```
Get-Content RUN\journal-2.jsonl -Encoding UTF8 | ConvertFrom-Json | Where-Object event -eq
CALL_INTENT_RECORDED | ForEach-Object { $b = Get-Content (Join-Path RUN $_.call_binding.path) -Raw -
Encoding UTF8 | ConvertFrom-Json; [pscustomobject]@{round=$_.round; actor=$b.access.actor_id;
granted=$b.access.granted -join ', '} }
```

```
round actor              granted
----- -----              -------
    1 buyer_negotiator   buyer_move_minutes, buyer_negotiator_brief, supplier_move_minutes
    1 buyer_manager      buyer_draft, buyer_manager_brief, buyer_move_minutes, supplier_move_minutes
    1 supplier_negotiator buyer_move_minutes, supplier_move_minutes, supplier_negotiator_brief
    1 supplier_manager   buyer_move_minutes, supplier_draft, supplier_manager_brief,
supplier_move_minutes
```

The same four lines repeat in every round. To see the denied sources, replace `granted=$b.access.granted -join ', '` with `denied=$b.access.denied -join ', '`. A manager receives its organization's draft (`*_draft`) but not its own negotiator's brief. No actor receives the other

organization's sources. The other side's **offered price** is known from the World state (`world.offers`), which every actor receives (section 2.2.9).

⑤ The full message — what the actor actually saw

```
$ev = Get-Content RUN\journal-2.jsonl -Encoding UTF8 | ConvertFrom-Json
$c = $ev | Where-Object event -eq CALL_INTENT_RECORDED | Select-Object -Index 0
(Get-Content (Join-Path RUN\requests-v2 $c.request.name) -Raw -Encoding UTF8 | ConvertFrom-
Json).messages[0].content
```

`-Index 0` is the first call (the buyer negotiator in round 1). Change the number to pick another call, counting from 0 in call order; for example, `-Index 7` is the supplier manager in round 2. The output has the format of section 2.2.9 (`actor` , `information` , `round` , `world`). Each value in `information` is the content of a source, as a string.

⑥ Termination

```
Get-Content RUN\journal-2.jsonl -Encoding UTF8 | ConvertFrom-Json | Where-Object event -eq
TERMINATION_EVALUATED | Select-Object round, terminate, @{n='reason';e={$_.termination.reason}}
```

In the last round, `terminate` is `True` and the reason is `TERMINAL_STATE_SUCCESS` (agreement). The other rounds show `False` and `STATE_NOT_TERMINAL` .

Part 6. Research Interpretation and Limits

6.1 Distinguish six things

Each "success" indication from the tools means something different. Do not mix them in a paper or report.

Category	What it means	Which indication confirms it	What it does not mean
Run success	The declared procedure ran to the end.	`COMPLETED` from `bvl run`	That the result is correct. That the model understood the instructions.
Declaration and experimental condition verification	The declaration is well-formed, and conditions differ only in what was declared.	`valid` from `bvl validate`, `gate-2 CLEARED` from `bvl check`, (with two or more conditions) GNDCRW `RUN`	That the condition difference is appropriate for the research question.
Evidence integrity verification	The package files match the list, the records fit together, and the run followed the declared plan.	`VERIFIED` from `bvl verify`, `VERIFIED` from `fullmt`	Who produced it (no signature). Whether the model server was honest.
Analysis recomputation	Pre-bound metrics are the same when recomputed from the evidence.	`bvl analyse`, FULLMT rule `M4-ANA`	That the metric measures the research question. Statistical significance.
Scientific validity	The design can answer the research question.	**The tools do not judge this.**	—
Proof of causation	The difference in conditions produced the difference in results.	**The tools do not judge this.**	—

The last two are the researcher's responsibility. This is made explicit by the fact that the package's `limitations` always contains `CONFORMANCE_PACKAGE_NOT_SCIENTIFIC_EVIDENCE`.

6.2 What a GNDCRW `RUN` guarantees and what it does not

What it guarantees (`gndcrw-0.2.0.dist-info\METADATA`):

> A RUN means only that the assembled experiment satisfied the declared preflight integrity checks on the supported observable surface.

That is, in the observed values of the two given conditions, every item declared as must-differ differed, every item declared as must-be-equal was equal, and no basis needed for the judgment was missing.

What it does not guarantee:

- It does not run the experiment. It says nothing about results.
- It does not claim scientific validity.
- It does not check whether the observed values themselves are true. It takes the values written in the surface as given.

- It accepts premises (`premises`) without verifying them (limitation code `ATTRIBUTED_PREMISE_NOT_INDEPENDENTLY_VERIFIED`).
- Its own code identity is self-reported (`TOOL_CODE_SELF_REPORTED` , always included in the result).
- For items that are not observed, it does not look for differences outside the declaration.
- It cannot be used as independent proof: "FULLMT must never treat a GNDCRW RUN as independent proof."

Meaning of the three outcomes:

Outcome	Condition
`BLOCK`	There is at least one known violation. Something that should differ is the same, something that should be equal differs, or the declaration itself is defective.
`UNVERIFIABLE`	There is no violation, but a basis needed for the judgment is missing. Not observable, pairing not established, missing provenance information, and so on.
`RUN`	Neither of the above applies. Advisory-level warnings do not change the outcome.

Additional limit for GNDCRW inside BVL (approach A): the documents BVL passes are ones BVL built from its own plan. FULLMT states this as `GATE_INPUTS_SUPPLIED_NOT_DERIVED` : gate inputs are bound as supplied and re-checked, but GNDCRW itself is not run again.

Study with one condition: GNDCRW is not called. You cannot claim "GNDCRW `RUN` ".

6.3 What FULLMT `VERIFIED` guarantees and what it does not

What it guarantees (`FMT\docs\FULLMT_EPC_READER.md`):

> `VERIFIED` says the run conformed to its frozen plan and protocol.

Specifically, FULLMT does not use BVL code. It recomputes the following from the package bytes alone.

- The SHA-256 and size of every file match the list. No file is outside the list.
- The identities of the plan, episodes and steps are the same when recomputed.
- The event record has no refused, failed or unknown events, and the order is correct.
- Each step was committed after its preceding steps were committed.
- The links among calls, proposals, decisions, World transitions and memory match the dependencies in the plan.
- The results of the built-in decision rules (single, majority, plurality) are the same when recomputed.
- Dataset units and pre-bound analysis results are byte-for-byte the same when recomputed.
- The conformance attestation exactly matches FULLMT's own re-derivation.

What it does not guarantee — FULLMT writes these itself into `limitations.consumer` of every result (`FMT\app\manifest4_reader.py`):

Code	Meaning
`CONFORMANCE_NOT_SCIENTIFIC_EVIDENCE`	It only means the run conformed to the frozen plan and protocol. It is not scientific evidence and does not support any scientific claim.
`NOT_SIGNED`	There is no signature. Who produced these bytes is not established.
`ACTOR_CALLS_NOT_REEXECUTED`	Responses are only bound by hash. Model or provider calls are not repeated.

Code	Meaning
`WORLD_CODE_NOT_REPLAYED`	For World transitions, it checks the identities, links and permission judgments as recorded. It does not run World code.
`BUILD_AND_PLUGIN_SET_SELF_REPORTED`	The identities of the build and plugin set are recomputed from documents the producer wrote itself.
`GATE_INPUTS_SUPPLIED_NOT_DERIVED`	Gate inputs are checked as supplied. GNDCRW is not run again.
`OBSERVATION_DIGESTS_ATTESTED_NOT_RECOMPUTED`	Observation hashes are values attested by the producer.

So `VERIFIED` does **not** tell you:

- Whether the model actually gave that response (whether the model server honestly reported its name and output).
- Whether running the same declaration again gives the same result.
- Whether the World rules were implemented correctly (this is the area that `bvl verify --replay-world` checks by recomputing with the World code on this PC).
- Whether the research design is valid, or what the results mean.

Difference between `REJECTED` and `ERROR`: `REJECTED` is a judgment (one or more rules failed). `ERROR` is not a judgment (the input could not be processed).

`meaning`: the result of a normal `bvl package` is `CONFORMANCE`. If it comes out as `MEMBER_INVENTORY`, only "these files, these bytes" are confirmed, and conformance is not claimed.

6.4 Relationship between `bvl verify` and `fullmt`

	`bvl verify`	`fullmt`
Who computes	The tool that ran it	A separate program that shares no code with BVL
World recomputation	Possible with `--replay-world`	Not done
Conformance level (L1–L5) judgment	Done	Not done (only checks that the attestation's re-derivation matches)
Significance	Consistency from the producer's view	Independent check against defects or tampering in the producer tool

Neither replaces the other. When reporting, write exactly which command was run with which options.

6.5 Points to note in studies with a real model

- **Reproducibility.** `temperature: 0` and a fixed seed do not guarantee the same response. Do not generalize from the result of a single run.
- **Influence of instructions.** The model follows the instructions in the source files. If the instructions fix the behavior concretely, the result is the execution of those instructions, not the model's judgment.
- **Interpreting no action.** With `on_invalid: NO_ACTION`, a response that broke the format and a deliberate refusal are recorded as the same `NO_ACTION`. To tell them apart, look at the raw responses

in `artifacts\`.

- **What analysis metrics count.** Metrics count the actions that decisions requested. Anything the World adjusted or disallowed is not reflected.
- **Comparison between conditions.** The tools do not do this. State that comparisons made by the researcher are outside the verification scope of these tools.

6.6 Wording you can use in a report

May write	Must not write
"Declaration X (plan hash …) was run and produced `COMPLETED`."	"The experiment was verified."
"The package was `VERIFIED CONFORMANCE` under FULLMT 0.12.0: the run conformed to the frozen plan and protocol."	"FULLMT verified the results."
"The two conditions were `RUN` under GNDCRW-STRICT-0.1: only the declared items differed."	"GNDCRW confirmed the validity of the experiment."
"In a single run, … was observed."	"The model does …."

Part 7. Troubleshooting

All codes below are defined in the installation's source. Symptom wording is taken from the messages in the source. Codes marked **[Run-confirmed]** were produced on purpose on the rc6 installation, with the message and exit code shown.

7.1 Schema errors in input JSON

Code / symptom	Cause	Action
`BVL-LIFE-001` `study declaration is invalid: ...`	Key typo, missing required key, undefined key, empty list.	The message lists the allowed keys. Compare with the relevant part of section 2.2. **[Run-confirmed]**: a top-level typo gives `study has keys [...]; required [...], optional [...]`, exit code 2.
`INVALID_JSON`, `DUPLICATE_KEY`, `INVALID_UTF8`, `BOM`, `TRAILING_CONTENT`	JSON syntax error, same key twice, encoding problem.	Save as UTF-8 (no BOM). Check commas and brackets. **[Run-confirmed]**: `declaration: SERIALIZATION_ERROR BOM: the input starts with a UTF-8 byte-order mark` and `SERIALIZATION_ERROR DUPLICATE_KEY: member 'root_seed' is named twice`, exit code 2. Windows Notepad saves UTF-8 without BOM by default; "UTF-8 with BOM" is refused.
`BVL-PROP-003`	Malformed proposal schema, disallowed field type.	Types are only `integer`, `number`, `string`, `boolean` (**[Run-confirmed]**: `field 'amount' type 'float' is not one of [...]`).
`BVL-PROP-006`, `BVL-PROP-008`	Unknown value in `on_invalid`, or a repair/retry type value.	Only `PARSE_FAILURE` or `NO_ACTION`.
`BVL-DEC-008`, `BVL-DEC-012`	A value in `on_unresolved` that invents an action or depends on order.	Only `FAIL_EPISODE` or `NO_ACTION`.
`BVL-ROUND-001`, `-002`, `-003`, `-010`	Malformed round contract, missing `max_rounds`, unknown stop rule, `count` over the limit.	Section 2.2.3.
`BVL-REV-001`, `-002`	Malformed `bounded_revision`, `max_revisions` outside 0–3.	Section 2.2.3.
`BVL-WORLD-001`, `-002`, `-003`, `-004`, `-013`	Unknown World, schema name mismatch, initial state mismatch, missing `world` section, random policy mismatch.	Compare with the table in section 4.1.
`BVL-ANA-001`	Malformed `analysis`.	Section 2.2.10.
`BVL-COV-008`, `-009`	Malformed `design.factors` (fewer than 2 levels, etc.), keys of `contrasts` differ from the factors.	Section 2.2.11.

7.2 Actor / Information Source reference errors

Code	Cause	Action
`BVL-ACTOR-001`, `-002`, `-003`, `-004`	Duplicate actor name, undeclared role, nonexistent profile, level mismatch in per-factor profiles.	Compare the names in `actors`, `roles` and `profiles`. **[Run-confirmed]** for `-003`: `actor 'lead' names undeclared profile 'no-such-profile'`.
`BVL-ACTOR-008`	`seed` was put in `options`.	Remove it. The seed is set only with `root_seed`. **[Run-confirmed]**
`BVL-ACTOR-009`	A node or access rule points to an undeclared actor.	Check `config.actor` and `access[].actor`.
`BVL-INFO-001`	An access rule points to an undeclared source.	Compare `source_id`. **[Run-confirmed]**
`BVL-INFO-002`	A source file cannot be read, or the source kind cannot be executed.	`path` is relative to the `study.json` folder.
`BVL-INFO-003`	Both `ALLOW` and `DENY` apply to the same actor and source, or a source name is duplicated.	Consolidate into one rule. Split conditions with `when`.
`BVL-INFO-004`	`effect` is not `ALLOW` or `DENY`.	
`BVL-INFO-012`	The producing node of an `actor_output` is not a same-round predecessor of the receiving actor's call.	Make the receiving actor's call node depend (directly or indirectly) on the producing node. Pass across rounds with memory.
`BVL-PROP-001`	The parser is not registered, or `proposal_parse` nodes and contracts are not one-to-one.	
`BVL-DEC-001`, `-005`, `-011`	Decision rule not registered, authority composition error, authorities have different schemas, or a voting rule has a `number` field.	Section 2.2.7.
`BVL-ROUND-006`, `-007`	Memory scope, retention or owner error; the producing node of a memory source is not a lifecycle-type `memory_update`.	Section 2.2.3.

7.3 Gate refusal (`gate-2 REFUSED`, exit code 8)

Look at the reason lines from `bvl check`. Running with `--json` shows them all in `gate_decision.reasons`.

Reason code	Meaning	Action
`BVL-GATE-003:<comparison edge>:BLOCK`	GNDCRW found a violation on that comparison edge. Usually an item not listed in `contrasts` differs between the two conditions.	List in `contrasts` every plan item the factor actually changes. **[Run-confirmed]** (section 4.5)
`BVL-GATE-003:<comparison edge>:UNVERIFIABLE`	A basis needed for the judgment is missing.	Look at that result in `gndcrw_results` in the `--json` output.

Reason code	Meaning	Action
`BVL-GATE-004`	The result is malformed, or **the GNDCRW engine failed or is not installed.**	Sections 1.6 and 3.2. This was the reason given for factorial designs in Toolkit rc2 and earlier. Check the link with `bvt --version` and repair with Setup.
`BVL-GATE-001`, `-002`, `-005`	The result set does not match this study's comparison edges.	It should not appear in normal use. Record the reproduction conditions and report it.
`BVL-GATE-006`	Plugin preflight failed. A parser or decision rule name is not registered.	Check that it matches the names in section 4.1 character for character. On the rc6 installation an unregistered parser name was refused with the reasons `BVL-COV-010:cell:proposal.<node>.config` and `BVL-PROP-001:<node>:<name> is not registered` instead **[Run-confirmed]**.
`BVL-COV-002`	There is a config key or option that the plugin does not accept.	Check whether you gave `options` to the script provider or put `config` in a parser contract. **[Run-confirmed]**: `BVL-COV-002:cell:actor.lead.profile.options.temperature`.
`BVL-COV-005`	`design.exclusions` was declared.	Remove it. It cannot be used in this build.
`BVL-COV-007`	An item name in `contrasts` matches no item in the plan.	Check the name format in section 2.2.11. **[Run-confirmed]**
`BVL-COV-009:<comparison edge>`	There is no result for the comparison edge.	Often appears together with `BVL-GATE-004`.
`BVL-COV-010`	Unregistered transform name, or the configuration of an unregistered parser.	Only `builtin.redact-json-keys:1` exists as a transform. It also appeared together with `BVL-PROP-001` for an unregistered parser **[Run-confirmed]**.

A run folder refused by the gate stays closed (`BVL-LIFE-003`, **[Run-confirmed]**). Fix the declaration, then run in a new folder.

7.4 GNDCRW Manifest errors (standalone use, exit code 3)

Code	Cause
`MANIFEST_NOT_JSON`, `MANIFEST_YAML_NOT_SUPPORTED`	The file name does not end with `.json`.
`MANIFEST_NOT_FOUND`	The file does not exist, or the letter case of the name differs.
`MANIFEST_TOO_LARGE`, `MANIFEST_BOM`, `MANIFEST_INVALID_UTF8`, `MANIFEST_INVALID_JSON`, `MANIFEST_DUPLICATE_KEY`, `MANIFEST_NOT_OBJECT`	Format problems in the file itself.
`MANIFEST_VERSION_MISSING`, `MANIFEST_VERSION_UNSUPPORTED`	`manifest_version` is missing or is not `gndcrw-manifest/v0.2`.
`MANIFEST_UNKNOWN_FIELD`	There is an undefined key.
`MANIFEST_FIELD_INVALID`	Missing key, type error, malformed `sha256`, `size`, `scheme` or `digest`.
`MANIFEST_PATH_INVALID`, `MANIFEST_DUPLICATE_PATH`	Path rule violation, two roles with the same path.
`REF_MISSING`, `REF_NOT_REGULAR`, `REF_LINK`, `REF_HARDLINK`, `REF_CASE_ALIAS`, `REF_ALIAS`	The referenced file does not exist, is not a regular file, is a link, differs in letter case, or is the same file as another input.

Code	Cause
`REF_SIZE_MISMATCH`, `REF_DIGEST_MISMATCH`	The file's size or hash differs from the manifest. The file changed after the manifest was made. Converting `pair.json` to CRLF line endings gave `REF_SIZE_MISMATCH: pair_context: declared 378 bytes, read 379` **[Run-confirmed]**.
`SUBJECT_CONTRACT_INVALID`	One of the five documents does not conform to its contract.
`EXTERNAL_SUBJECT_NOT_IN_SURFACE`	`external_subject.digest` is not recorded in both surfaces.

Command usage errors (exit code 2): `MANIFEST_ARGUMENT_COUNT`, `OPTION_NOT_SUPPORTED`, `OPTION_REPEATED`, `MIXED_MODE`, `STDIN_NOT_SUPPORTED`. Result write errors (exit code 6): `OUTPUT_IS_INPUT`, `OUTPUT_EXISTS`, `OUTPUT_WRITE_FAILED`.

7.5 GNDCRW `BLOCK` / `UNVERIFIABLE` (standalone use)

In the result's `findings`, look at the `reason_code` of items whose `status` is `FAIL` or `UNVERIFIABLE` (`GND\core\vocabulary.py`).

Reasons that produce `BLOCK` (`INVARIANT_VIOLATED` **[Run-confirmed]**, section 4.7):

`reason_code`	Meaning	Action
`EXPECTED_CHANGE_MISSING`	An item declared as must-differ is the same in both conditions.	Check that the manipulation was actually applied.
`INVARIANT_VIOLATED`	An item declared as must-be-equal differs.	Remove the unintended difference, or if the difference is intended, fix the declaration.
`COMPOSITION_CHANGE_MISSING`, `COMPOSITION_INVARIANT_VIOLATED`	The same problems for composition (entity count, entity set).	
`NO_FOCAL_EXPECTED_CHANGE`	`expected_changes` is empty.	Declare at least one thing that must differ.
`INVALID_DECLARATION_TARGET`	A rule points to an item not in the catalog.	
`INVALID_DECLARATION_OPERATION`	An `operation` that does not fit the section.	`expected_changes` takes `MUST_DIFFER`, `invariants` takes `MUST_EQUAL`.
`INVALID_DECLARATION_SCOPE`	A `scope` that does not fit the item level.	Section 2.5.5.
`CONTRADICTORY_DECLARATION`	Opposing rules on the same target.	
`INVALID_COMPARISON_MODE`	The pair's `comparison_mode` differs from the declaration's `required_mode`.	
`ASSEMBLY_EQUIVALENCE_MISMATCH`	In the evidence for a reconstructed surface, expected and observed values differ.	

Reasons that produce `UNVERIFIABLE`:

`reason_code`	Meaning	Action
`TARGET_MISSING`, `TARGET_UNOBSERVABLE`, `TARGET_UNSUPPORTED`, `TARGET_AMBIGUOUS`	The rule's target item was not observed on one side.	Observe the item and record it as `OBSERVED`.
`TARGET_ASSERTED_ONLY`	The value is only asserted, not observed.	Asserted values do not satisfy a rule.
`ENTITY_COMPARABILITY_UNKNOWN`, `ENTITY_COMPARABILITY_NOT_ESTABLISHED`, `ENTITY_MAPPING_MISSING`	There is no basis for pairing the entities of the two conditions.	Establish the pair's `entity_comparability`.
`ASSEMBLY_EQUIVALENCE_MISSING`	Equivalence of the reconstructed surface is not proven.	Use `DIRECT` assembly or provide `EXACT_BYTES` evidence.
`REQUIRED_PROVENANCE_MISSING`	Something is missing among the surface's provider information, assembly entry point and input digests.	Fill in the nulls and empty objects.

`UNDECLARED_DIFFERENCE` (a difference outside the declaration) is a warning and does not change the outcome. Still, it may be an unintended difference, so check it.

7.6 AI Provider connection errors

Provider errors take the form `"<code>: <description>"` and fail that run (`PKG\providers\openai_chat.py`). **The text output of `bvl run` shows only `FAILED`; the code and description are in the `failure` field of `bvl run ... --json`** (section 5.5).

Code	Cause	Action
`PROVIDER_UNCONFIGURED`	`plugin_config` has no `endpoint`.	Section 2.2.12. Caught at the declaration stage: `BVL-LIFE-001 ... PROVIDER_UNCONFIGURED: bvl.openai-chat-adapter:1 has no endpoint; declare it in plugin_config` from `validate`, exit code 2 **[Run-confirmed]**.
`PROVIDER_UNREACHABLE`	Cannot connect to the server.	Check that the server is running and that the address and port are correct. **[Run-confirmed]**: `cannot reach http://127.0.0.1:1/v1 (ConnectionRefusedError)`.
`PROVIDER_TIMEOUT`	No response within `timeout_seconds`.	Increase the value or reduce other load. The first model load can take a long time.
`PROVIDER_HTTP_ERROR`	The server returned an error. A missing model is common.	Read the server's wording in the message. Check the model name. **[Run-confirmed]** with Ollama: `HTTP 404 (model 'no-such-model:1b' not found)`.
`PROVIDER_INVALID_RESPONSE`	The response is not JSON, does not have exactly one choice, or `finish_reason` is not `stop`.	If the answer was cut off, increase `max_tokens`. **[Run-confirmed]**: `the completion did not finish (finish_reason 'length')`.
`PROVIDER_CREDENTIAL_MISSING`	The environment variable that `api_key_env` points to has no token.	Set the environment variable.
`PROVIDER_REQUEST_UNSUPPORTED`	The request has an option the provider does not accept.	Options table in section 2.2.4.

Code	Cause	Action
`BVL-ACTOR-007`	The model name reported by the server differs from the declaration.	Make `profiles[].model` exactly match the name the server reports.

After these errors, the run becomes `FAILED` with `calls: 0 custody-accepted, 1 pending` for the failed call (exit code 5) **[Run-confirmed]**. It cannot be continued or packaged, so fix the cause and run again in a new folder.

7.7 Model output parse failures

Code	Cause	Action
`BVL-PROP-004`	The response is not a single JSON object. Explanatory text, code block markers, multiple objects.	State in the source file "one JSON object only; write nothing else". Use `response_format: json_object`.
`BVL-PROP-005`	Missing field, extra field, type mismatch, `null`.	Write the exact fields and types in the source file. For an `integer` field, `80.0` or `"80"` fails.
`BVL-WORLD-004` (during run)	The decided action does not fit the World's action schema.	Align the proposal schema's fields with the World's action schema.
`BVL-DEC-003`, `-006`, `-007` (during run)	Authorities gave no action, tied, or used a veto, and `on_unresolved` is `FAIL_EPISODE`.	If this is the intended behavior, use `on_unresolved: NO_ACTION`.
`BVL-WORLD-008` (during run)	A no action decision reached a World that does not declare its meaning.	Use a World that has a meaning for no action (table in section 4.1).

Diagnosis order:

1. Open `RUN\artifacts\call-*.bin` to see what the model actually answered.
2. Open `RUN\requests-v2\request-*.json` to see what the model actually received.
3. Look at the `verdict` and reason in `RUN\evidence-v2\proposal\`.

If `on_invalid` is the default (`PARSE_FAILURE`), the run ends at the first failure. In studies with a real model, using `on_invalid: NO_ACTION` and `allow_no_action: true` records failures as "no action" and lets the procedure continue.

7.8 BVL run failures

Code / symptom	Cause	Action
`BVL-LIFE-002 ... already holds evidence`	The `--into` folder already has content.	Use a new folder name. **[Run-confirmed]**: `identity: BVL-LIFE-002 RUN already holds evidence; a run is opened once, into an empty directory`, exit code 3.
`BVL-LIFE-003 ... was refused by gate-2 and stays closed`	You tried to continue a run the gate refused.	Fix the declaration and run in a new folder.
`BVL-LIFE-004`	`--experiment` was not given for resume, or the declaration changed and a different gate decision resulted.	Use the original declaration and source files unchanged.

Code / symptom	Cause	Action
`BVL-LIFE-005`	It stopped before the result of an external call was confirmed. The tool does not call again on its own.	Run again in a new folder.
`BVL-LIFE-010`	An installed external plugin was refused.	If you installed an external plugin, that is the cause.
`BVL-LIFE-011`	Another `run`, `resume` or `package` is writing to the same folder.	Wait for one to finish.
`BVL-HUMAN-009`, `BVL-TOOL-009`	There is a human actor but no `--human-desk`, or a tool actor but no `--tool-site` or `argument_selection`. Or the reverse.	README in section 2.2.13.
`execution: FAILED`	An episode failed.	Running with `--json` gives the reason in `failure`. The `EPISODE_FAILED` event in `journal-2.jsonl` records only the error class, such as `ProviderError` **[Run-confirmed]**.
`execution: [Errno 2] No such file or directory: '...\.<hash>.json.partial'` (from `run` or `package`, exit code 5)	The path of the run folder is too long for Windows (section 5.1).	Use a short folder such as `C:\work\<study>\RUN`. The folder it leaves is incomplete (`bvl status` shows only `input lock: NOT_INPUT_LOCKED`); delete it and run again in the short folder. **[Run-confirmed]**
`USAGE_ERROR OPTION_NOT_SUPPORTED`	An option not accepted for a `bvl-study-1` study was given (`--gndcrw`, `--stop-after`, `--expect-schedule`, etc.).	Remove that option.
`USAGE_ERROR LEGACY_EXECUTION_RETIRED`	You tried to run `bvl-experiment-2` input.	Declare it as `bvl-study-1`. **[Run-confirmed]**, exit code 2.
`ContainmentViolationV2: <code>` (in the failure reason)	A plugin performed a disallowed action (`HIDDEN_NETWORK`, `HIDDEN_WRITE`, etc.).	It should not appear with the default installation. Suspect an external plugin.

Packaging stage:

Symptom	Cause	Action
`manifest-4 package refused: ...` (exit code 6)	The run did not complete normally. There are refused, failed or unknown events.	That run cannot be packaged. Run again.
`manifest-4 is immutable and already exists`	Already sealed.	No need to do it again. Check with `bvl verify`.
`BVL-LIFE-007`	You tried to read the analysis from a package that does not verify.	See the problems with `bvl verify RUN --json`.
`BVL-LIFE-006`	The study did not declare `analysis`.	Analysis must be declared before the run.

Symptom	Cause	Action
`bvl verify` is `REFUSED`, with `unlisted file ...` in problems	A file was added to the folder after sealing.	Move that file outside `RUN`. **[Run-confirmed]**
`bvl verify` is `REFUSED`, with `<file>: size mismatch` / `digest mismatch`	A file inside `RUN` was edited after sealing.	The package cannot be repaired. Restore the original bytes from a copy, or run again. **[Run-confirmed]**
`bvl verify --l4` fails (`NOT_L4`)	The study uses `rounds`, not a `round` contract.	Match the level requirement to the study structure. It cannot be changed after the run. **[Run-confirmed]**: `X2 the plan's Protocol IR carries no round contract ...`, exit code 4 while the first line says `VERIFIED`.

7.9 FULLMT `REJECTED` / `ERROR`

`REJECTED` (exit code 1). Look at the rules in `failed_rules`, and read that rule's `problems` in the result's `rules[]`.

Rule	What failure means	Common cause
`M4-MEM`	A file's hash or size differs from the list, or a file is missing.	A file was modified or deleted after sealing. Line endings changed during copying. **[Run-confirmed]**: one added byte in an evidence file gave `failed=['M4-MEM', 'M4-ROL']`, because the name of an evidence file is the hash of its content.
`M4-UNL`	There is a file in the folder that is not in the list.	A result JSON, notes or temporary file was put inside `RUN`. **[Run-confirmed]**: `REJECTED CONFORMANCE sha256:... failed=['M4-UNL']`, exit code 1.
`M4-MAN`	The format of `MANIFEST-4.json` is wrong.	The file was edited by hand.
`M4-ROL`	A file's location and role do not match, or its name is not the hash of its content.	A file was moved or renamed.
`M4-CLS`	The package class is not `CONFORMANCE`, or the limitation list differs.	The manifest was edited.
`M4-JRN`	The event record has refused, failed or unknown events, or the order is broken.	A run that did not complete normally.
`M4-AUT`, `M4-PLN`, `M4-EPI`, `M4-CMT`, `M4-ART`, `M4-CUS`, `M4-WLD`, `M4-LIN`, `M4-L1C`, `M4-RQR`	One of the gate, plan, episodes, commit order, evidence identities, receipt records, World transitions, links, call binding or required roles differs from the recomputation.	The content of an evidence file changed.
`M4-ATT`, `M4-L1D`, `M4-ANA`	The attestation, dataset units or analysis result differ from FULLMT's recomputation.	The content of that file changed.
`M4-EXT`	The response records of human or tool actors do not match the declaration.	

When one rule fails, later rules usually stay `NOT_EVALUATED` (in both cases above, 15 rules were `NOT_EVALUATED`). Start from the first failure.

If you get `REJECTED` for a folder produced by a normal `bvl package` that you did not touch, first check whether bytes changed while copying, compressing or transferring the folder. If that still does not explain it, keep the whole result JSON and report it.

`ERROR` (exit code 2). This is not a judgment. The input could not be processed (`FMT\app\fullmt_codes.py`).

`error_code`	Cause	Action
`PACKAGE_NOT_A_DIRECTORY`	The path is not a folder or does not exist.	Check the path. **[Run-confirmed]**
`PACKAGE_ROOT_IS_A_LINK`	The folder is a link.	Specify the real folder.
`NOT_AN_EPC_PACKAGE`	Neither `MANIFEST-4.json` nor `EVIDENCE.json` exists.	Run `bvl package` first.
`AMBIGUOUS_PACKAGE_FORMAT`	Both files exist.	Keep only one format in a folder.
`MANIFEST_NOT_READABLE`	`MANIFEST-4.json` is not a regular file.	
`JSON_PARSE_ERROR`	The manifest is not strict JSON.	The manifest is damaged.
`UNSUPPORTED_MANIFEST_SCHEMA`	The manifest's `schema` is not supported.	
`PERMISSION_DENIED` , `IO_ERROR`	Files cannot be read.	Check permissions and disk.
`CONSUMER_RUNTIME_ERROR` , `INTERNAL_ERROR`	Processing limit or tool defect.	Record the reproduction conditions and report it.

`OUTPUT_WRITE_ERROR` is the case where a judgment was made but the `--out` file could not be written. It appears in `delivery` on standard output, and the exit code is 2. The judgment itself is valid.

`USAGE_ERROR` is an argument error. Nothing is printed to standard output.

Exit code 9009 comes from the shim, not from FULLMT. A component or the linked Python is missing. Repair with Toolkit Setup.

Appendix A. Check results

The final edition was checked on a Toolkit `1.0.0-rc6` installation on 2026-10-16 (Windows 11 Home 10.0.26300; Python 3.13 64-bit; Ollama `0.35.1` with `qwen2.5:7b-instruct`; `bvt --version` `CONSISTENT`). All study folders were copies outside the installation; nothing in `C:\Program Files\BOOMVALLEY Research Toolkit` was changed.

Templates.

Template	`validate` / `check`	`run`	`package`	`verify`	FULLMT	`analyse`
A (minimal, scripted)	pass	`COMPLETED`, 2 calls	25 members	`VERIFIED` (L1; L3 with `--replay-world`)	`VERIFIED`	`2/1`, `3/1`
B (`EX\provider\local_model_study.json`, real model)	pass	`COMPLETED`, 6 calls	44 members	`VERIFIED` (L1, L2)	`VERIFIED`	`3/1`, `32/3`
C-1 (`b3_model_as_factor`, 2 cells)	pass, GNDCRW `RUN`	`COMPLETED` × 2	23 / 24 members	`VERIFIED`	`VERIFIED` × 2	`2/1`, `3/1`
C-2 (`b2_summary_raw_access`, 2 cells)	pass, GNDCRW `RUN`	`COMPLETED` × 2	23 members	`VERIFIED` (L1)	`VERIFIED`	—
D (4-agent negotiation, real model)	pass, same plan as 2026-10-13	`COMPLETED`, 16 calls	107 members, same package identity as 2026-10-13	`VERIFIED` (L1–L4)	`VERIFIED`, 20 `PASS`	`8/1`, `0/1`, `167/2`
GNDCRW standalone (`stage_a\gndcrw`)	`gndcrw check`: `RUN`	—	—	—	—	—
Negotiation demo (`examples\manual_negotiation_demo`, 2026-10-17)	`gndcrw check gndcrw\manifest.json`: `RUN` (`PASS` 7)	`COMPLETED`, 16 calls	107 members	—	`VERIFIED`	—

The negotiation demo row is the result of four commands (`gndcrw check`, `bvl run`, `bvl package`, `fullmt`) run on the same PC on 2026-10-17. `bvl verify` and `bvl analyse` were not run. The commands of section 5.13 were checked on the `RUN` of this run and on the `RUN` of Template D.

All official examples. The 31 `bvl-study-1` studies in `EX` all passed `bvl validate` and `bvl check` in every cell. The 24 studies that need no real model, human or tool (27 cells) were run, packaged and checked with FULLMT; every cell was `COMPLETED` and `VERIFIED CONFORMANCE`. The largest, `EX\longitudinal\realistic_scale_vending.json`, took about 18 minutes from `run` to FULLMT and produced a package of 13,021 files; the others took seconds. The 2 real-model studies were checked up to `check` (one of them is Template B, which was also run). The 2 human and 3 tool studies were checked up to `check` only.

Error cases produced on purpose. Each with the message and exit code given in Part 5 or Part 7:

- Declaration: top-level key typo (`BVL-LIFE-001`), BOM, duplicate key, `float` field type (`BVL-PROP-003`), undeclared profile (`BVL-ACTOR-003`), undeclared source (`BVL-INFO-001`), `seed` in `options` (`BVL-ACTOR-008`), missing `endpoint` (`PROVIDER_UNCONFIGURED`), 2-cell design without `--cell` , `bvl-experiment-2` run (`LEGACY_EXECUTION_RETIRED`).
- Gate: unregistered parser (`BVL-PROP-001` , `BVL-COV-010`), `options` for the scripted provider (`BVL-COV-002`), incomplete `contrasts` (`BVL-GATE-003 ... BLOCK`), unknown contrast item (`BVL-COV-007`), resume of a refused run (`BVL-LIFE-003`).
- Run: unreachable server, unknown model, cut-off answer (`PROVIDER_UNREACHABLE` , `PROVIDER_HTTP_ERROR` , `PROVIDER_INVALID_RESPONSE`); used `--into` folder (`BVL-LIFE-002`); run folder path too long (`[Errno 2]`).
- Package and verification: packaging a `FAILED` run, packaging twice, `analyse` without `analysis` (`BVL-LIFE-006`), a file added to `RUN` (`unlisted file` , `M4-UNL`), an edited evidence file (`size mismatch` , `M4-MEM` , `M4-ROL`), FULLMT on a missing folder (`PACKAGE_NOT_A_DIRECTORY`), `--out` inside `RUN` .
- GNDCRW: `BLOCK` with and without `--gate` , CRLF-converted input (`REF_SIZE_MISMATCH`), placeholder digests (`catalog_digest` / `intent_digest does not recompute`), `OUTPUT_EXISTS` , `OUTPUT_IS_INPUT` , `OPTION_NOT_SUPPORTED` , `OPTION_REPEATED` , `MANIFEST_NOT_FOUND` .

Corrections made in this edition because the run differed from the earlier text.

1. `bvl verify` with a level option that is not met still prints `VERIFIED` on the first line; only exit code 4 shows the failure (section 5.8).
2. The reason for a `FAILED` run is only in `bvl run --json` (`failure`), not in the text output and not in detail in `journal-2.jsonl` (section 5.5).
3. `fullmt --out` inside `RUN` is not refused and breaks the package's verification (section 5.11).
4. Run folders with long paths fail on Windows (sections 0.3 and 5.1).
5. A `path` with `..` or an absolute path in `information.sources` is not refused (section 2.3).
6. An unregistered parser is refused with `BVL-PROP-001` and `BVL-COV-010` , not `BVL-GATE-006` (section 7.3).
7. `gndcrw --locale ko-KR` translates error messages only, not the `check` report (section 5.1).
8. The folder `EX\bvl_input_v2\gndcrw\` has no `manifest.json` ; the runnable standalone example is `EX\b2b_frontline_internal_exchange_qwen_100_158\stage_a\gndcrw\` (section 2.5.6). Neither folder is in the anonymous review edition; there the runnable set is `examples\manual_negotiation_demo\gndcrw\` (sections 0.1 and 2.5.9).

Mismatches with the original request (unchanged from the first edition):

1. **"The researcher creates the GNDCRW input files"** — in `bvl-study-1` , BVL creates them. Writing them directly is only for standalone use, and that is not bound to a BVL run. Accordingly, the "GNDCRW input configuration" step was replaced with the `design` declaration.
2. **"Multiple experimental conditions"** — supported from Toolkit rc3; confirmed on rc6.
3. **Wording of `bvl package --help`** — it says "package a SEALED run as an EPC-v1 evidence package", but it actually produces MANIFEST-4 **[Run-confirmed]**.
4. **`README_WINDOWS.md` of the BVL component** — its install location and first-run example differ from this Toolkit installation (it is based on a standalone installation, and the example is `bvl-experiment-2`).

Appendix B. Unconfirmed items

Not confirmed by running on the rc6 installation, or not traced to the end in the source:

1. The Setup screens (UAC, wizard pages), Setup exit codes other than 0, uninstallation, and use from a second Windows user account.
2. Running studies with human actors (`bvl human` , `--human-desk`) and tool actors (`--tool-site`). The official examples pass `validate` and `check` only.
3. Continuing an `INTERRUPTED` run with `bvl resume --experiment` , and `BVL-LIFE-004` / `BVL-LIFE-005` .
4. The plurality decision rule (`builtin.plurality:1`) in a run.
5. A GNDCRW `UNVERIFIABLE` result, standalone or at the gate (`BLOCK` and `RUN` were confirmed).
6. What difference the two values of `seed_policy` make to real model responses (only the plan hash difference was confirmed).
7. Whether the same real model gives the same answers on another PC or another Ollama version. On the check PC, Template D gave the same package as three days earlier.
8. The exact boundary between problems found by `validate` and those found by `check` beyond the cases in Appendix A.
9. `certutil` output on Windows versions or display languages other than the check PC.
10. The body of FULLMT rules `M4-AUT` – `M4-L1C` and `M4-EXT` . Descriptions were taken from the rule registry and documentation.
11. The normative specification of GNDCRW. It is not in the installation, so only the verification code was read.
12. FULLMT exit code 9009 and the `OUTPUT_WRITE_ERROR` case.
13. That Toolkit `1.0.0` carries the same component versions as rc6. Check with `bvt --version` (section 0.1).

Appendix C. Main source files

Topic	File
`study.json` syntax	`PKG\lifecycle\study.py`
Registered components	`PKG\lifecycle\plugins.py`, `PKG\domains\__init__.py`, `PKG\providers\__init__.py`
Actors, profiles	`PKG\kernel\actors.py`
Information access	`PKG\kernel\information.py`
Proposals, decisions	`PKG\kernel\proposals.py`, `PKG\kernel\decisions.py`, `PKG\kernel\revision.py`
Rounds, memory	`PKG\kernel\rounds.py`, `PKG\kernel\memory.py`
World	`PKG\kernel\world.py`, `PKG\domains\*.py`
Design, coverage, GNDCRW link	`PKG\kernel\coverage.py`, `PKG\kernel\comparison.py`, `PKG\kernel\strict_gate.py`, `PKG\cli\bvl.py`
Analysis	`PKG\kernel\bound_analysis.py`
Real model provider	`PKG\providers\openai_chat.py`
Commands, execution, packaging	`PKG\cli\bvl.py`, `PKG\cli\render.py`, `PKG\lifecycle\gated.py`, `PKG\kernel\package4.py`, `PKG\kernel\manifest4.py`
Verification levels	`PKG\kernel\l1_conformance.py` ~ `l5_conformance.py`, `PKG\kernel\world_replay.py`
Exit codes, errors	`PKG\core\errors.py`, `PKG\core\codes.py`
Official examples and explanations	`EX\k0_study\`, `EX\conformance\`, `EX\longitudinal\`, `EX\external\`, `EX\provider\`, `EX\human\`, `EX\tool\` (`README.md` in each folder; not included in the anonymous review edition, section 0.1)
GNDCRW contracts	`GND\contracts\*.py`, `GND\manifest.py`, `GND\core\vocabulary.py`, `GND\core\engine.py`, `GND\cli.py`, `GND\codes.py`
FULLMT	`FMT\docs\FULLMT_EPC_READER.md`, `FMT\README_WINDOWS.md`, `FMT\app\epc_reader.py`, `FMT\app\manifest4_reader.py`, `FMT\app\fullmt_codes.py`, `FMT\app\fullmt_rules.py`

Appendix D. Related documents

Paths are relative to the `boomvalley-lab` repository.

Document	Contents
`packaging/toolkit/README_INSTALLER.md`	Setup behavior, exit codes, installation layout
`packaging/toolkit/INSTALL_CONTRACT.md`	Install contract (section 15 GNDCRW link, section 16 `bvt --version`, section 17 this manual and the Start menu)
`packaging/toolkit/VERSIONING.md`	Version policy and `bvt --version`
`packaging/toolkit/KNOWN_NONBLOCKERS.md`	Known non-blockers (N-32 no `bvl --version`, etc.)
`packaging/toolkit/TOOLKIT_RELEASE_MANIFEST.json`	Release manifest of the current Toolkit release (component versions, `documentation` section for this manual)
`packaging/toolkit/release/`	Setup and its `.sha256` file, build and acceptance records of the current release
`examples/*/README.md`	Official examples (the `EX` folder in the installation)
`examples/manual_negotiation_demo/`	Declaration, GNDCRW input and guide (`SAMPLE_GUIDE_EN.md`) of the four-step manual demo (GNDCRW → BVL → FULLMT). The GNDCRW input is the worked example of section 2.5.9.
`packaging/toolkit/demo/README.md`	5-second tool tour demo. It is the rc1 demo and differs from the 4-agent B2B negotiation demo (`KNOWN_NONBLOCKERS.md` N-25).